HookSuite

Memoria técnica · Práctica 3 — «Del prototipo al producto»

Máster en Ciberseguridad & IA · Evolve Academy

Grupo **Feet Lovers**

⚠️ **HUECO** — responsable: **José María**. Enlace al vídeo (≥10 min, no listado): es lo único que falta de la portada. Comprobarlo desde una ventana de incógnito antes de entregar.

1. Portada

Producto: **HookSuite** — herramienta web de auditoría de seguridad «tipo Burp Suite», con IA. Grupo: **Feet Lovers**. Entrega: **16 de octubre de 2026**.

Integrantes: Ivan Medina · Macarena Rogerio · Nacho García · Carlos Bañuelos · José María López

Áreas de trabajo de esta práctica, asumidas en común y sin reparto por persona:

Área	Qué se hizo
Frontend	Pulido del Spider y del Repeater, revisión de pantallas, nueva pestaña Manual y retirada de Red
Backend	Alta y acceso por usuario, aislamiento de datos y bus interno de mensajes
IA	Integración del análisis de hallazgos con el panel y control del gasto
Seguridad	Cierre de la API, orígenes permitidos, canal autenticado, gestión de secretos y limpieza del historial
Infraestructura y despliegue	Contenedores, servidor público y despliegue reproducible
Pruebas y evidencias	Plan de pruebas, ejecución y capturas
Documentación	Memoria técnica, README y vídeo

Repositorio: https://github.com/Feet-Lovers/Proyecto-Evolve

2. Resumen ejecutivo

HookSuite resuelve la auditoría web asistida por IA desde el navegador: interceptar y repetir peticiones, fuzzing, utilidades (hash/encoder/regex), análisis de vulnerabilidades y un módulo de IA que prioriza hallazgos. La P3 parte del prototipo de la P1 y lo lleva a un estado ejecutable, seguro y reproducible. **Para quién.** El destinatario es quien audita aplicaciones web y no quiere depender de una herramienta instalada en su equipo: se abre una dirección, se entra con su usuario y trabaja con sus propios datos, aislados de los de los demás.

De dónde se parte, y por qué no era defendible. La verificación en vivo del 29 de septiembre dejó el prototipo de la P1 en evidencia: la API respondía en el puerto 8000 **sin ninguna credencial** desde internet, de modo que cualquiera podía lanzar un rastreo o un ataque de fuerza bruta contra un tercero desde la dirección del grupo; la credencial documentada no abría el frontend; cuatro claves de la API de Anthropic estaban dentro del historial de git; y el módulo de IA no conseguía hablar con el backend.

Qué se ha hecho. Primero la casa: historial de git limpiado y las cuatro claves revocadas y verificadas muertas; los puertos 8000 y 3000 retirados de internet, orígenes restringidos, WebSocket autenticado y el servidor endurecido con cortafuegos y `fail2ban`. Después la identidad: registro y acceso por usuario con JWT y aislamiento entre sesiones, donde se encontraron y cerraron **cuatro fugas** —y un quinto fallo que introdujo la propia corrección—, verificado once de once y desplegado en producción. Luego el módulo de IA: bus interno entre backend y modelo, disparo de la auditoría desde el panel, operación degradada cuando el modelo falla o no supera el umbral de confianza, y un **techo de gasto** que nace de una lección cara de la P1. Por último, el pulido con uso real: quince defectos en el Spider y dieciséis en el Repeater, la mayoría encontrados **usando** la herramienta, y la retirada del laboratorio vulnerable del producto entregable.

En qué estado se entrega. De los **veintiún requisitos numerados**: **quince cumplidos** —uno con una limitación documentada en pantalla pequeña—, **dos modificados** y **uno descartado**, los tres justificados en el apartado 4, **dos parciales** —el módulo de navegador, construido y en la red interna pero sin su prueba ejecutada, y el historial de contribuciones, partido por dos correos del mismo autor— y **uno pendiente declarado**: TLS. Suman veintiuno, y ninguna fila de la matriz queda sin estado. El código está **congelado y etiquetado** como `v1.0-practica3` y **desplegado en producción sobre esa misma etiqueta**: lo que se ve funcionando es lo que hay en el repositorio.

Cómo se puede comprobar. El anexo del apartado 13 reconstruye los **dieciocho casos** de prueba que cita la matriz con dos columnas —lo que daba el prototipo en septiembre y lo que da el código congelado — y dice **de dónde sale cada resultado**: seis reejecutados contra producción, seis demostrados por una

figura de esta memoria, cinco no reejecutados con su motivo escrito y uno con el procedimiento caducado. Lo que no se ha medido, no se afirma.

Lo que esta memoria no esconde. El uso de herramientas de IA en la propia práctica se declara en el apartado 12, con la distinción entre lo generado y lo verificado; las limitaciones conocidas están en el apartado 10; y las afirmaciones que hubo que rectificar sobre la marcha quedan declaradas en el cuerpo del documento, no corregidas en silencio.

3. Punto de partida

HookSuite nació en la Práctica 1 como prototipo funcional de una suite de auditoría web «tipo Burp Suite» operada desde el navegador y asistida por IA. Quedó desplegado en un servidor Hetzner (`www.hooksuite.de`) con Docker Compose y, desde la entrega de mayo de 2026, **nadie volvió a tocarlo**: el máster encadenó otras actividades y la Práctica 3 lo retoma tal como quedó.

Qué nos encontramos (verificación en vivo, 29-sep y 4-oct)

- La herramienta **seguía en pie** desde mayo, sin cambios: DNS → Hetzner, `nginx/1.31.0`, `uvicorn` y los 36 endpoints de la API respondiendo.
- Núcleo funcional sólido: Spider, Repeater, Intruder, Utilidades (hash/encoder/regex), sesiones por token UUID y WebSocket operativos.
- Dos hallazgos que **reorientan** la Práctica 3 (detalle en el apartado 7):
- La API `:8000` estaba **abierta a internet sin ninguna autenticación**, incluidos los endpoints que lanzan tráfico contra terceros.
- En el historial de git había claves de API reales (secretos expuestos).
- El módulo de IA (RF-08), que es lo que da sentido al «con IA» del producto, **nunca llegó a operar** salvo pruebas muy iniciales de la P1: panel construido pero inactivo.

Lo que la P1 nos enseñó a su costa: el saldo de la API

En las primeras pruebas del módulo de IA se cargaron 5 € de saldo en la API y se agotaron en muy poco tiempo. La causa está **reconstruida del historial de git** para esta memoria, no de lo que nadie recordara: el módulo ejecutaba una auditoría completa **en el arranque**, y el servicio está declarado con `restart: unless-stopped`, de modo que cuando el proceso terminaba Docker lo relanzaba y empezaba otra auditoría. Un bucle de auditorías mientras el contenedor estuviera levantado.

Se corrigió el 17 de mayo de 2026 con dos cambios: el módulo pasó a **esperar instrucciones** en lugar de auditar al arrancar —lo que además impide que el proceso termine, y con ello que Docker lo reinicie— y se dejó el análisis en modo simulado por defecto, como freno de mano.

Por qué figura en el punto de partida y no como anécdota: es el único incidente de la P1 con coste económico directo, y la P3 lo hereda a medias. El disparo automático está curado, pero el **tamaño** de una auditoría nunca se acotó (apartados 8 y 10). Es además una de las razones por las que el módulo de IA llegó a la P3 sin haber operado.

Numeración de requisitos

La P1 **no numeraba** los requisitos. El enunciado de la P3 (apartado 3) obliga a numerarlos ahora (RF-xx / RNF-xx) y a usar esa numeración en toda la práctica. La lista de abajo (apartado 4) está **derivada del informe de la P1** —de lo que el producto prometía y hace— y PENDIENTE DE VALIDAR POR EL GRUPO en la reunión antes de darla por firme.

Clasificación inicial de requisitos

Usando las cinco categorías del enunciado. El recuento es provisional, a validar por el grupo junto con la numeración.

Estado	Qué significa	Requisitos
Cumplido	Implementado en la P1 y funciona; en la P3 se verifica con prueba y evidencia	RF-01, RF-03, RF-04, RF-05, RF-06, RF-07, RF-11 · RNF-01, RNF-02, RNF-03, RNF-05, RNF-08 **(12)**

Parcial	Existe algo, pero incompleto o con fallos; hay que terminarlo y probarlo	RF-08 · RNF-04, RNF-07 **(3)**
Pendiente	Definido, pero sin implementar; se hace desde cero	RF-09, RF-10, RF-12 · RNF-09 **(4)**
A revisar	El feedback o la experiencia indican que estaba mal planteado; se reformula con justificación	RF-02 · RNF-06 **(2) en el punto de partida**; RF-04 se suma el 12-oct, durante la Fase 3 (§4)
Descartado	Se decide no implementarlo (debe ser la excepción y justificarse)	— ninguno **en el punto de partida**; RF-09 se descarta después, durante la Fase 3 (§4)

Sobre el feedback de la Práctica 1: no lo hubo

El enunciado pide reunir «la calificación y los comentarios recibidos del equipo docente» y construir esta práctica sobre ellos. **El grupo no recibió ni calificación ni comentarios de la Práctica 1**, así que ese material no existe y no se puede citar. Se dice aquí en vez de dejar el apartado a medias.

De dónde salen entonces las mejoras. De dos fuentes propias, las dos documentadas en esta memoria:

- **El plan de mejoras que la propia P1 dejó escrito.** En la reunión del 29 de septiembre el grupo eligió abordar sus cinco primeros puntos, y de ahí sale el alcance de esta práctica.
- **La revisión crítica del producto, hecha ejecutándolo.** La verificación en vivo del 29 de septiembre encontró los defectos que esta memoria documenta y que las fases siguientes corrigen uno a uno —entre ellos los dos más graves: la API respondía sin credenciales y el panel pedía una contraseña que no funcionaba—.

Y una precisión sobre la tabla de arriba. La categoría «A revisar» se define como «el feedback **o la experiencia** indican que estaba mal planteado». Sin feedback, en esta práctica fue **siempre la experiencia**: RF-02 y RNF-06 se reformularon por lo que se vio al usar el producto, no por lo que nadie nos señalara.

4. Requisitos

Lista completa y numerada. Numeración fijada en la P3 (la P1 no la tenía) a partir del informe de la P1; PENDIENTE DE VALIDAR POR EL GRUPO. El estado es el de partida (apartado 3); se actualiza a medida que la P3 avanza.

Requisitos funcionales (RF)

ID	Requisito	Estado (de partida, puesto al día durante la P3)
RF-01	Dashboard web accesible desde cualquier navegador, sin instalar nada en el cliente	Cumplido
RF-02	El servidor ejecuta las peticiones HTTP en nombre del auditor (proxy del lado servidor)	**Modificado** (§4)
RF-03	Spider: rastreo automático del objetivo (BFS, scope al dominio, extrae formularios), velocidad configurable	Cumplido
RF-04	Auditoría autenticada: sesión `httpx` persistente compartida por Spider/Repeater/Intruder que hereda las cookies del login	**Modificado** (12-oct, justificado abajo)
RF-05	Repeater: modificar y reenviar cualquier petición; respuesta en 4 vistas (Raw/Pretty/Preview/Headers)	Cumplido
RF-06	Intruder: fuzzing de parámetros marcados con `*`; 4 tipos (SQLi, Blind SQLi, XSS, genérico); resultados en tiempo real	Cumplido
RF-07	Utilidades: Encoder/Decoder (Base64/URL/HTML/JWT), Hash (MD5/SHA1/SHA256/SHA512), Regex Tester, Payload Generator	Cumplido

RF-08	**Análisis y clasificación de vulnerabilidades con IA (Claude) según OWASP** → panel Vulnerabilidades	Parcial
RF-09	Captura de tráfico HTTP en tiempo real (DevTools / CDP) → panel Red con código de colores	**Descartado** (§4)
RF-10	Ejecución de ataques con navegador real (Playwright)	**Parcial** (construido y en red; sin prueba ejecutada — §§7-8)
RF-11	Soporte multi-usuario simultáneo mediante tokens de sesión UUID	Cumplido
RF-12	Registro y login individual con JWT + aislamiento de historial/auditorías por usuario	Pendiente (mejora)

> **Matiz sobre el enunciado de RF-03, anotado el 11-oct.** El requisito habla de «velocidad configurable». El control que existe no regula velocidad: fija un **tope de páginas** a visitar. Un valor menor no rastrea más rápido, llega menos lejos. El enunciado se conserva tal como se escribió en la P1 —no se reescriben requisitos a posteriori—, pero la interfaz sí se corrigió para decir lo que de verdad hace, y el control pasó a llamarse «hasta N páginas».

Requisitos no funcionales (RNF)

ID	Requisito	Estado (de partida, puesto al día durante la P3)
RNF-01	Accesible desde navegador sin instalar software en el cliente	Cumplido
RNF-02	Actualización en tiempo real vía WebSocket (pings cada 30 s, eventos `{type,payload}`)	Cumplido
RNF-03	Desplegado en servidor Hetzner con contenedores Docker	Cumplido
RNF-04	Código en GitHub con historial de commits que refleje el trabajo del grupo	Parcial
RNF-05	Estabilidad ante carga: límite de concurrencia para no saturar producción	Cumplido
RNF-06	Operación degradada cuando la IA no responde o no supera el umbral de confianza	**Cumplido** (§4; demostrado el 10-oct)
RNF-07	**Seguridad del propio producto**: secretos fuera del repo, control de acceso, validación de entradas, dependencias sin CVEs	Parcial
RNF-08	Firewall dinámico por sesión (abre un puerto por sesión, lo cierra al logout o tras 4 h)	Cumplido (a verificar)
RNF-09	TLS/HTTPS en los endpoints públicos (Let's Encrypt)	Pendiente

Requisitos modificados o descartados (con justificación)

El enunciado pide que todo requisito modificado o descartado lleve su versión original, la nueva y el motivo.

RF-02 — proxy del lado servidor (modificado)

- **Versión original (P1):** interceptación del tráfico del navegador mediante archivo de autoconfiguración de proxy (PAC) + WebSockets.
- **Versión nueva:** el servidor ejecuta las peticiones directamente con `httpx`, sin interponerse en el navegador del auditor.
- **Motivo:** en la P1, el modelo PAC quedó expuesto y se saturó con tráfico de bots. El pivote a `httpx` directo elimina esa superficie y simplifica el flujo. Es el caso de libro de «requisito reformulado con justificación».
- **Cierre en la Fase 2 (6-oct): el PAC se retiró del código.** Hasta entonces el producto seguía sirviendo el fichero de autoconfiguración aunque el modelo estuviera abandonado desde la P1. Se comprobó que estaba muerto antes de tocarlo: su único consumidor en la interfaz era un componente **que ningún otro fichero importaba**, y el segundo (el lanzador de navegador de RF-09) no está integrado y además apunta a un puerto que la Fase 1 había cerrado, así que ya estaba roto por su cuenta. Se retiraron la ruta del backend, sus dos bloques en Nginx y el componente.
- **Ganancia de seguridad no buscada.** El PAC era la **única** ruta que tenía que quedar sin autenticar, porque un navegador no envía credenciales al pedirlo, y por eso llevaba una excepción expresa en el guardián de la Fase 2. Al jubilarlo, **toda** `/api` exige token sin excepciones: no queda ningún hueco que

mantener ni que explicar. Una excepción de seguridad que se puede eliminar vale más que una excepción bien documentada.

RF-04 — auditoría autenticada con sesión compartida (modificado el 12-oct)

- **Versión original (P1), tal como está escrito el requisito:** sesión `httpx` persistente **compartida por Spider, Repeater e Intruder**, que hereda las cookies del login del objetivo.
- **Versión nueva:** el Repeater deja de usar esa sesión compartida y envía una petición **independiente**, con exactamente las cabeceras que se ven en pantalla. Spider, Proxy e Intruder siguen compartiéndola.
- **Motivo.** El requisito describe un **mecanismo**, pero lo que persigue es poder auditar autenticado; y ese mecanismo rompía en el Repeater algo que vale más: **la reproducibilidad**. El cliente compartido hacía dos cosas invisibles —enviaba cookies que no estaban en la petición y se quedaba con las que devolvía el servidor—, así que **dos envíos idénticos daban resultados distintos** y lo que se veía en pantalla no era lo que salía por el cable. Se descubrió porque un POST **sin** `Cookie` en sus cabeceras recibía la respuesta de la última página que había visitado el Spider: si el servidor reconoce al auditor sin que él mande sesión, la sesión la pone el producto por detrás.
- **Por qué el requisito se sigue cumpliendo.** La sesión se hereda igual, pero **por un canal explícito**: desde el arreglo de esa misma mañana las cabeceras que el producto muestra son las realmente enviadas, de modo que la `Cookie` del objetivo viaja **en la petición**, visible y editable. Antes de ese arreglo este cambio habría roto la herencia de verdad; después de él, la traslada de un canal oculto a uno a la vista.
- **Lo que el cambio destapó, y es la lección de la jornada.** Al quitarle el cliente compartido, **todos los POST empezaron a devolver `419`**. La causa no era el cambio: los formularios que el Spider descubre viajaban con unas cabeceras incompletas, sin la cookie de sesión, y **el cliente compartido lo estaba tapando**. Un mecanismo «de más» sostenía un dato incompleto que nadie veía, y solo se descubrió al quitarlo. Se arregló el origen —los formularios descubiertos parten ahora de las cabeceras realmente enviadas al pedir la página— y el `419` desapareció. De aquí sale una regla de trabajo: **preguntar por qué algo está así antes de tocarlo** no es burocracia.
- **Estado en la matriz: Modificado**, la misma categoría que RF-02.
- **Corrección declarada de este documento (13-oct, por R9), y la hubo que buscar dos veces.** La matriz del apartado 9 daba a este requisito el estado **A revisar**, que es el nombre que la clasificación de partida usa para «estaba mal planteado y se reformula con justificación». En un entregable final esa palabra se lee como «no lo sabemos», así que **el estado final se escribe Modificado**: el requisito describía un **mecanismo**, el mecanismo cambió con justificación y el objetivo —auditar autenticado— se sigue cumpliendo.
- **Y al aplicarlo salió un defecto anterior, que se arregla en el mismo paso.** La columna de estado de las listas de este apartado se titulaba **Estado P1**, pero **no** contenía solo estados de la P1: `RF-09` figuraba ahí como **Descartado**, que es una decisión del 11 de octubre. Es decir, la columna **se actualiza** —lo dice su propia entradilla— y el título mentía. Se corrige el título y se ponen al día las dos celdas que nunca se actualizaron: RF-02, que la matriz ya daba por modificado, y RNF-06, que la matriz da por cumplido y demostrado desde el 10 de octubre. **La clasificación de partida (arriba) no se toca:** esa sí es histórica, lo dice expresamente, y reescribirla habría falsificado el punto del que salimos.

RNF-06 — operación degradada (a revisar en la P1; cumplido y demostrado en la P3)

- **Original:** enunciado como principio en la P1, sin demostrar.
- **Nuevo:** la operación degradada se demuestra haciendo que un fallo de la IA sea **distinguible** de un análisis correcto. El clasificador pasa a devolver tres estados donde hoy hay dos: *vulnerable* (con confianza suficiente), *analizado y limpio*, y *no analizado*, este último acompañado del motivo. Encaja con el criterio P3 «se comporta razonablemente ante errores», y se prueba a propósito: con una clave de API inválida el panel debe decir *no analizado*, no *limpio*.
- **Corrección de este documento (8-oct).** Este apartado prometía demostrar RNF-06 «con un umbral sobre el campo `confianza`». Ese umbral **ya existía** en el código desde la P1 —`CONFIDENCE_THRESHOLD = 60`, aplicado en los dos analizadores—, así que la demostración prometida no demostraba nada: se iba

a presentar como mejora de la P3 algo ya construido. Se deja escrito en vez de corregirlo en silencio, porque el error está en lo que este documento prometió, no en el producto.

- **Lo que de verdad faltaba, y no estaba en ningún pendiente.** Cuando la IA no puede analizar, el clasificador devuelve `None`, que es **exactamente** lo que devuelve cuando ha analizado y no hay nada: el panel presenta «sin vulnerabilidades» sin distinguir si eso es un resultado o una ignorancia. Y en el camino de fallo de la API es aún más brusco: el cliente lanza la excepción al agotar los reintentos y nadie la captura, de modo que **se interrumpe la fase de ataque completa** en lugar de continuar sin ese análisis. Lo uno no es operación degradada sino silencio, y lo otro no es degradación sino caída.
- **Aplicado y probado el 9-oct (commit `8ebea124`).** Los tres estados ya existen en el código: el clasificador devuelve `estado: "analizado"` cuando hay hallazgo, `None` cuando ha analizado y está limpio, y `estado: "no_analizado"` con su motivo cuando no ha podido analizar. Hizo falta añadir además el módulo `ia/esquemas.py`, que no existía: el cliente reescrito exige un esquema de respuesta y ninguna de las cuatro plantillas de consigna definía uno, de modo que el plan tal como estaba escrito habría fallado en la primera llamada. La prueba automática `ia/tests/test_tres_estados.py` recorre los tres estados en las cuatro vías de análisis con un cliente simulado —sin clave de API ni red, coste cero— y pasa *8 de 8*; sobre el código anterior daba *0 de 8*, y el error era precisamente el descrito arriba: la fase de análisis se caía entera. **Corrección declarada (9-oct, tarde).** Esta memoria afirmó durante unas horas que lo que impedía ver el estado degradado en el panel era un filtro del orquestador que comparaba la confianza contra `0.6` teniendo la escala en 0-100. Era falso: ese filtro ya estaba corregido desde las 13:03 de ese mismo día (commit `ea317969`, que importa el umbral del clasificador en lugar de duplicarlo en otra escala), cuatro horas antes de escribirse este párrafo. Se deja escrito en vez de corregirse en silencio, porque el modo en que se detectó es parte de lo aprendido. **El motivo real por el que RNF-06 todavía no se ve en el panel es más de fondo:** el estado degradado se calcula, se transporta y se publica dentro del módulo de IA, pero ningún consumidor lo lee —la marca `no_analizado` no aparece ni una sola vez en el frontend ni en el backend—. RNF-06 queda por tanto cumplido *dentro del módulo* y pendiente de *superficie*: lo que falta es el tramo que lleva ese estado hasta la interfaz, y se trata en el apartado 10.

RF-12 — registro y login individual con JWT (mejora, no requisito P1)

- No era requisito de la P1 (venía del road map de mejoras). En la P3 cuenta como **mejora**: suma, pero no compensa un requisito original sin cumplir. Se aborda por su valor de seguridad: es lo que cierra la fuga de datos entre usuarios documentada en los apartados 7 y 8.
- **Corrección de este documento (6-oct).** Este apartado se titulaba «login individual con JWT», sin la palabra **registro**. El requisito, tal como está escrito, pide las dos cosas: que cada persona **se cree su cuenta** y que entre con ella. Al redactar se estrechó el requisito a la mitad sin declararlo; se deja constancia porque un requisito recortado en silencio es peor que un requisito incumplido y explicado.
- **Cómo se resuelve el registro, y por qué no es abierto.** La persona se da de alta ella misma y elige su propia contraseña —nadie más la conoce, ni quien administra, que es el punto del requisito—, pero necesita un **código de invitación** que reparte el grupo. La razón es el propio producto: HookSuite lanza tráfico contra terceros, así que con altas anónimas cualquiera podría usar el Spider y el Intruder contra quien quisiera desde nuestra infraestructura. El código es obligatorio por configuración: si faltara, el servicio no arranca, en lugar de arrancar con el registro abierto sin que nadie lo note.
- **Lo que obliga a persistir, y lo que no.** Las **cuentas** se guardan en un volumen, porque el despliegue reconstruye los contenedores y unas cuentas que se evaporasen en cada despliegue harían inútil el registro. El **estado de sesión** sigue siendo volátil a propósito, por las razones del apartado 10: son cosas distintas y conviene no confundirlas.

RF-09 — captura de tráfico en tiempo real con DevTools/CDP (descartado)

- **Versión original (P1):** captura del tráfico HTTP en tiempo real mediante el protocolo de depuración de Chrome (CDP), volcada a un panel «Red» con código de colores por método y estado.
- **Versión nueva:** se descarta. El **11-oct** se retiró del producto la pestaña Red —entrada de navegación, ruta y página—, y la interfaz pasa de seis pestañas a cinco: Proxy, Repeater, Intruder, Utilidades y Vulnerabilidades.

- **Motivo, verificado contra el código antes de decidir.** RF-09 no era una pantalla a medio pulir, sino un vertical entero sin cerrar, con cuatro cosas rotas a la vez: el capturador se lanza **a mano** y busca el navegador en rutas absolutas de Windows (`C:\Program Files\...`) que no existen en los contenedores, que son Linux; sus hallazgos de consola se envían a un extremo **que no existe** en el backend y se pierden contra un 404; su lanzador apunta en una sola línea a una IP de producción fija, a un puerto que la Fase 1 cerró y a la ruta del fichero PAC que se jubiló al reformular RF-02; y la pestaña del panel era **pasiva** —no pedía nada al servidor, solo pintaba lo que llegara por WebSocket—, de modo que el usuario no podía distinguir «no hay tráfico» de «el capturador no está corriendo».
- **Por qué se descarta en vez de terminarse.** Cerrarlo exigía arreglar las cuatro cosas y, sobre todo, **probarlo**, a dos días de la congelación y con la auditoría con IA (RF-08) recién verificada de punta a punta. El enunciado pide que los descartes sean la excepción y vayan justificados; este es el único, y la alternativa honesta era entregarlo sin probar.
- **Qué se conserva, y por qué no se borra.** El capturador (`devtools/`) permanece en el repositorio: es trabajo hecho y queda como punto de partida en el apartado 10. Los extremos de red del backend (`backend/routes/network.py`) **también se conservan, y no por inercia**: los consume el módulo de IA y el de Playwright, así que retirarlos habría roto RF-08. Esa dependencia no era evidente desde la pestaña, y es la razón de que el descarte se limite a la interfaz.
- **Efecto sobre otros requisitos.** RF-01 (acceso desde el navegador sin instalar nada) se mantiene cumplido: la barra de navegación pasa de seis pestañas a cinco, y la figura del apartado 8 documenta el estado anterior, capturado **antes** de retirarlo porque el propio arreglo destruía la evidencia.

> **Descartados:** uno, **RF-09**, justificado arriba. El enunciado pide que sean la excepción: es el único de los doce requisitos funcionales que se descarta.

5. Arquitectura y decisiones técnicas

HookSuite se despliega como un conjunto de contenedores Docker coordinados por `docker-compose`, todos en una red interna `hooksuite-net`, con Nginx como única puerta de entrada. Arquitectura verificada fichero a fichero en la cocina local el 4-oct (`docker-compose.yml` + `infra/nginx.conf`).

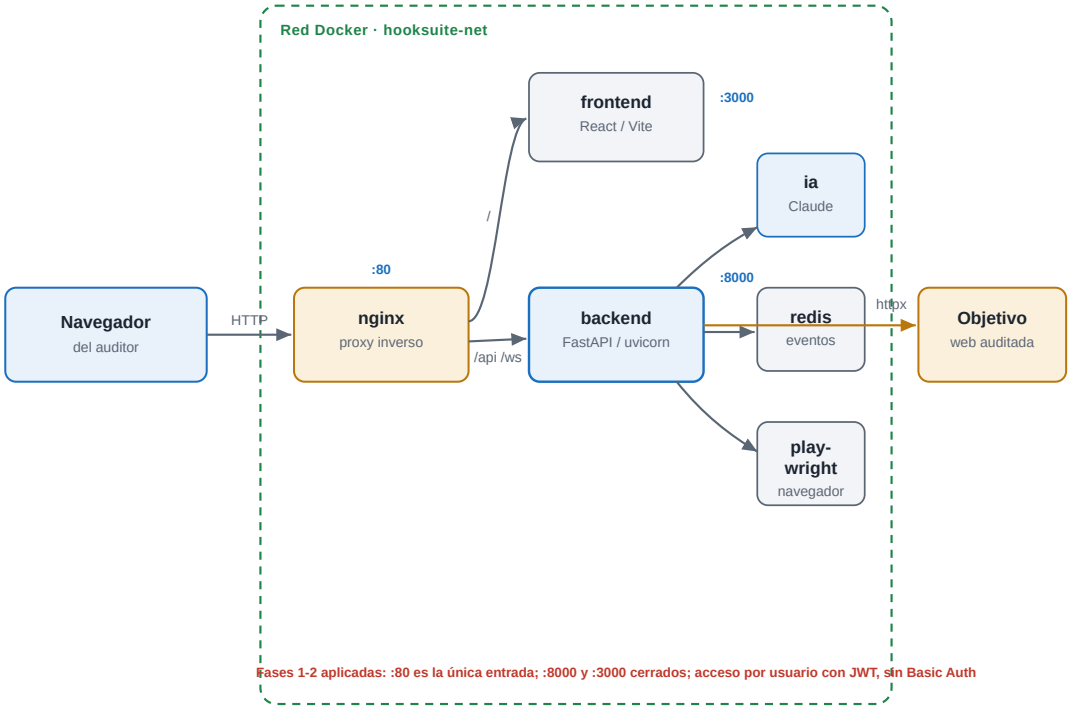


Figura 1: Arquitectura de contenedores de HookSuite: Nginx es la única entrada; el backend FastAPI orquesta Spider/ Repeater/Intruder/IA/Playwright y sale al objetivo auditado con `httpx`. Desde la Fase 2 la autenticación ya no está en Nginx sino en el backend, que exige token en toda `/api` y `/ws/` (apartado 7).

Componentes

Componente	Tecnología	Rol	Puerto (prod.)
nginx	`nginx:alpine`	Proxy inverso y único punto de entrada; enruta a frontend y backend (el control de acceso pasó al backend en la Fase 2)	`80→80`
frontend	React + Vite (build propio)	Dashboard web (SPA) servido como estático	`3000→80`
backend	FastAPI + uvicorn (build propio)	API de auditoría: proxy `httpx`, Spider, Repeater, Intruder, utilidades, vulnerabilidades y WebSocket	`8000→8000`
redis	`redis:7-alpine`	Bus de eventos y estado efímero entre servicios	interno
ia	Python + SDK Anthropic (build propio)	Clasificador de vulnerabilidades con Claude (RF-08)	interno
playwright	Python + Playwright (build propio)	Navegador real para ataques dirigidos (RF-10)	interno

Enrutado (Nginx)

Toda petición entra por Nginx, que reparte por ruta (`infra/nginx.conf`):

- `/` → **frontend**: sirve el panel de acceso propio del producto. Hasta la Fase 2 iba protegido con Basic Auth; ya no.
- `/api/` y `/ws/` → **backend** (API y WebSocket). Desde la Fase 2, **ambas exigen token**; sin él responden `401`.
- `/check/` y `/proxy.pac` → **retiradas en la Fase 2**, al jubilar el fichero de autoconfiguración de proxy (apartado 4, RF-02).

Desde la Fase 1, Nginx es además la **única** entrada: `backend` y `frontend` dejaron de publicar puerto propio, y la ruta hacia el laboratorio vulnerable se retiró del proxy (apartado 7).

El reparto de responsabilidades cambió con la Fase 2, y conviene decirlo porque es una decisión de arquitectura, no un detalle: antes Nginx era quien autenticaba (una contraseña única, igual para todos, que no distinguía personas) y el backend no pedía nada. Ahora Nginx solo reparte, y **quien autentica y autoriza es el backend**, que es el único que puede saber de quién es cada dato.

Dos entornos: la cocina y la caja

Toda la práctica se trabajó sobre **dos** instalaciones de HookSuite, y la separación entre ellas es una decisión de ingeniería, no una comodidad. Conviene explicarla antes de los apartados 6, 7 y 8, porque es el marco en el que ocurre todo lo que esos apartados cuentan.

Entorno	Para qué	Quién lo toca
La cocina (servidor doméstico propio)	Donde se reproduce cada defecto, se escribe el arreglo, se reconstruyen los contenedores y se verifica que funciona. Todas las pruebas de las Fases 1 y 2 se ejecutaron aquí.	Cualquier cambio, cuantas veces haga falta
La caja (servidor público contratado)	La instalación que usan las personas. Es la herramienta final.	**Solo recibe despliegues.** Ni un comando de prueba, ni un arreglo rápido, ni una reconstrucción

El orden de trabajo, siempre el mismo. Reproducir el fallo en la cocina → escribir el arreglo → reconstruir y verificar en la cocina → y solo entonces desplegar a la caja y volver a verificar allí. Ninguna de las dos verificaciones sustituye a la otra, y la memoria las mantiene **separadas a propósito**: en la matriz del apartado 9 hay requisitos marcados como «verificado en cocina, pendiente de producción», y el apartado 7 estuvo diciendo expresamente que no podía afirmar nada de la instalación pública hasta que el despliegue se hizo de verdad.

El despliegue descarta lo que encuentre en la caja. Se trae el código con `git fetch` y `git reset --hard`, nunca con `git pull`. La diferencia importa: `reset --hard` **borra** cualquier cambio local, así que si alguien editara un fichero en la caja, el siguiente despliegue se lo llevaría. Eso no es un efecto secundario que se tolere, es el comportamiento que se busca: convierte la caja en un destino y no en un sitio donde trabajar.

Y la regla nació de un fallo propio, que es la razón de que sea tan estricta. La interfaz que corría en producción se había reescrito directamente en la caja y **no estaba en ninguna rama del repositorio**:

existía solo allí, sin commitear, sin revisar y sin forma de reconstruirla si la máquina se perdía. Recuperarla fue una de las tareas de la Fase 0. La prohibición de editar en la caja no es prudencia abstracta: es la respuesta a haber estado a un fallo de disco de perder el frontend.

Por qué en esta herramienta pesa más que en otras. HookSuite lanza tráfico contra sistemas de terceros —rastrea, repite peticiones y prueba cargas—. Ensayar eso en la instalación que otras personas están usando no arriesga solo nuestros datos: arriesga dirigir tráfico de ataque desde la infraestructura pública del grupo sin querer. El laboratorio vulnerable con el que se probaba vivía, por eso, en la red interna de la cocina; el **13-oct se retiró del producto por completo** (apartado 7) y las pruebas se hacen contra una web propia del equipo, autorizada para ello.

Lo que esta separación NO garantiza, dicho sin adornos. La cocina no es idéntica a producción: comparte máquina con otros servicios y remapea puertos (ver la nota de reproducibilidad al final de este apartado). Por tanto «verificado en la cocina» es una afirmación más débil que «verificado en producción», y la memoria no las confunde en ningún sitio. El caso que lo demuestra se cuenta en el apartado 8: un despliegue que habría terminado sin errores **sin aplicar** la configuración del proxy, porque nombrar un servicio en un comando no prueba que el comando lo recree.

Decisiones técnicas

El bus interno entre el backend y el modulo de IA (RF-08)

El control de acceso de la Fase 2 tuvo una consecuencia que no se previo: **dejo al propio modulo de IA fuera**. Ese modulo preguntaba al backend por HTTP si habia trabajo pendiente, y esa ruta quedo protegida como todas las demas, de modo que desde el 6 de octubre recibia un rechazo por falta de credencial. El apartado 8 cuenta como se descubrio y por que nadie lo noto.

Reconectarlo admitia tres caminos, y la eleccion dice mas del producto que la implementacion:

1. **Darle una credencial de servicio** al modulo y que el control la aceptara. Es el camino corto, y abre una excepcion precisamente en la pieza cuyo valor es no tenerlas: el control se aplica por grupo de rutas para que una ruta nueva nazca protegida, y lo que hay que acordarse de poner es lo que se olvida.
2. **Dejar esa ruta sin control.** Peor: vive bajo el prefijo que el proxy publica a internet, asi que cualquiera podria encolar una auditoria **contra el objetivo que quisiera** desde nuestra infraestructura. Es el riesgo exacto que el codigo de invitacion del registro existe para evitar (apartado 7).
3. **Un bus interno**, que es lo que se hizo. El despliegue ya incluye un bus de eventos y el backend ya lo consumia para otro flujo. El panel pide la auditoria por la ruta normal —con su credencial, como cualquier otra peticion—, el backend publica la orden en el bus, y el modulo de IA la recoge de ahi. El salto interno no necesita autenticacion porque **la frontera la pone la red**: ese bus no esta publicado. Y no hay credencial nueva que custodiar, rotar ni documentar.

El detalle que ordena todo lo demas: el dueño viaja con la orden. Cada mensaje lleva el espacio de datos de quien pidio la auditoria, tomado del control de acceso en el momento de publicar. El modulo de IA construye su trabajo con ese dato y lo devuelve tal cual, de modo que los hallazgos aterrizan en la sesion de quien los pidio. Eso resuelve de raiz un problema que arrastrabamos: el modulo usaba un identificador de sesion **fijo**, heredado de antes del sistema de usuarios, que no era de nadie. No habia que arbitrar entre cuatro identificadores: habia que hacer que el dato viajara.

Y la contrapartida de un bus, que es la opuesta a la de un control de acceso. Un control de acceso falla cerrando; un bus falla **abriendo**: un mensaje que llega sin dueño podria repartirse «por si acaso», y eso es literalmente la fuga que cerro la Fase 2. Asi que un mensaje sin dueño **se descarta**, y hay pruebas de eso y de que dos auditores no ven lo del otro (apartado 8).

El Repeater se separa de la sesión compartida y de las redirecciones (RF-05)

Las cuatro herramientas que lanzan peticiones —Proxy, Spider, Repeater e Intruder— comparten una sola función de envío. Eso es bueno para el mantenimiento y malo para los matices: lo que conviene a un rastreador no conviene a un banco de pruebas manual. El 12-oct esa función gana **dos interruptores** en vez de duplicarse, y el Repeater es el único que los cambia:

- **Seguir redirecciones (activado por omisión; el Repeater lo desactiva).** Un Repeater que las sigue **nunca te enseña el `302`**: muestra la última parada y pierde por el camino la cabecera `Location` y cualquier `Set-Cookie` del salto, que en auditoría es justo lo que interesa. La herramienta de referencia muestra el `302` tal cual y deja que el auditor decida si lo sigue. El Spider las necesita activadas para rastrear, y el Proxy no se toca.
- **Cliente compartido (activado por omisión; el Repeater lo desactiva).** Separa el envío manual del cliente persistente que custodia las cookies del objetivo. La justificación completa está en RF-04 (apartado 4): sin esto, dos envíos idénticos no daban el mismo resultado.

Y un arreglo que beneficia a los cuatro a la vez. Las cabeceras que el producto muestra como «enviadas» se leen ahora **de la petición que la propia respuesta conserva**, no de un diccionario construido aparte. Antes, el Spider enseñaba un `User-Agent` sorteado por segunda vez, un `Host` que nadie había puesto y ninguna `Cookie`, mientras enviaba otra cosa. En una herramienta de auditoría eso no es un defecto de presentación: **la evidencia que se muestra tiene que ser la petición que salió**, porque es la que acaba copiada en un informe.

Decisión de alcance, declarada. La librería de red permite decidir las dos cosas **petición a petición**, así que el cliente persistente no se modifica y las otras tres herramientas conservan su comportamiento exacto. Se eligió eso antes que construir un segundo cliente para el Repeater, que habría duplicado la configuración de red —y con ella la desactivación de la verificación de certificados, que conviene tener en un solo sitio—.

Proxy del lado servidor: pivote PAC → httpx (RF-02)
La decisión arquitectónica de más peso heredada de la P1. En origen, HookSuite interceptaba el tráfico del navegador con un archivo de autoconfiguración (PAC) + WebSockets; ese modelo quedó expuesto y se saturó con tráfico de bots. Se pivotó a que **el servidor ejecute las peticiones directamente con `httpx`**: menos superficie y un flujo más simple, a cambio de perder la captura pasiva del tráfico del navegador —justo lo que retoman después DevTools (RF-09) y Playwright (RF-10)—. Versión original/nueva/justificación detalladas en el apartado 4.

Frontend: reescritura a Tailwind (cambio respecto a la P1)
La interfaz que corría en producción (`www.hooksuite.de`) se había reescrito al framework de estilos **Tailwind**, abandonando el sistema de variables CSS propio de la P1. Esa versión —la desplegada y en uso— es la que se toma como **base del frontend en la P3**. En la Fase 0 se reconcilió al repositorio (vivía solo en la caja, sin commitear); el detalle del proceso está en el apartado 8.

Puntos a corregir, detectados al montar la cocina
- **CORS abierto a cualquier origen** en el backend (FastAPI) → **corregido en la Fase 1**: lista cerrada de orígenes, configurable por entorno (apartado 7).
- **Verificación SSL desactivada** en el cliente `httpx`: justificable en una herramienta de auditoría (objetivos con certificados autofirmados), pero se documenta expresamente como decisión, no como descuido.
- **Acoplamiento Nginx ↔ DVWA**: `infra/nginx.conf` hacía del laboratorio un requisito de arranque del proxy (`host not found in upstream`) → **resuelto en la Fase 1** al retirar su ruta del proxy: el laboratorio queda solo en la red interna y Nginx ya no depende de él.
- **Playwright fuera de la red**: en el `docker-compose.yml` el servicio `playwright` no declaraba `networks: hooksuite-net`, así que quedaba aislado y no hablaba con el backend ni con el laboratorio, pese a tener configurados `BACKEND_URL=http://backend:8000` y `DVWA_URL=http://dvwa:80` (causa raíz de RF-10/RF-08 sin operar) → **corregido el 9-oct** con una línea en el compose, verificado con `docker compose config`, que ahora sitúa los siete servicios en `hooksuite-net`. **Corrección declarada (R9):** hasta ese arreglo la frase «todos en una red interna `hooksuite-net`» del apartado 5 era inexacta y el documento se contradecía a sí mismo; se deja dicho aquí en vez de reescribirlo en silencio.

> **Nota de reproducibilidad (cocina).** En el mediaserver los puertos `80/3000/8000` ya los ocupan otros servicios, así que la cocina los remapea a `8880/8830/8800` mediante un override local que no toca el compose versionado. Es solo del entorno de pruebas; la arquitectura de producción es la de la tabla.

⚠️ **HUECO** — responsable: **José María + Claude**. Añadir, si procede, un diagrama de secuencia del ciclo IA↔Playwright↔Backend cuando RF-08/RF-10 estén operativos.

6. Funcionalidades implementadas

Cada funcionalidad con capturas: qué hace, cómo se usa y qué requisitos cubre.

- Proxy interceptor, Repeater, Intruder, Utilidades (hash/encoder/regex), Vulnerabilidades, Red/DevTools, módulo IA.

La auditoría con IA, por fin arrancable desde el panel (RF-08)

Hasta el 10 de octubre el modulo de IA **no se podia usar**: existia, esperaba ordenes y no habia forma de darselas —ni un boton en ninguna pantalla—. Ahora la pantalla de vulnerabilidades lleva un campo de objetivo y un boton que lanza la auditoria. **El objetivo se hereda del rastreo en curso**, y si no hay ninguno el campo queda vacío y el botón se deshabilita solo: ofrecer un valor plausible y equivocado sería peor, porque auditar un sitio distinto del que se rastreó gasta llamadas para nada y sin un solo aviso.

> **Corrección declarada (13-oct, R9).** Este párrafo decía que «el objetivo por omisión es el laboratorio interno». Era falso **por partida doble**: el valor fijo se sustituyó por la herencia del rastreo el 10-oct, y el laboratorio salió del producto el 13-oct (apartado 7).

Dos detalles que no son adorno:

- **Encolar no es atender.** Si el modulo de IA no esta en marcha, el backend lo detecta —nadie escucha el bus— y la pantalla lo dice en rojo. Callarlo dejaria al operador esperando un resultado que no va a llegar, que es el mismo fallo silencioso que este documento persigue en otros tres sitios.
- **El estado degradado se ve.** Lo que la IA no pudo analizar aparece **arriba y en ambar**, separado de los hallazgos, con su motivo. Va separado a proposito: un fallo de analisis no es una vulnerabilidad, y mezclarlos obligaria a inventarle severidad y confianza a algo que precisamente no se pudo valorar. Y el mensaje de «no se han detectado vulnerabilidades» deja de aparecer cuando hay analisis incompletos: en su lugar dice que no hay hallazgos **pero que hay analisis sin completar**. Esa frase era, literalmente, el unico sitio donde el producto podia mentirle al operador sin que nada fallara.

Defectos corregidos en la Fase 1

Los **doce** defectos del prototipo que recoge la tabla, todos reproducidos antes de corregirlos y verificados después en producción. El detalle de cada uno (causa raíz, prueba y autoría) está en el diario del repositorio.

> **Corrección declarada (R9).** Hasta el 10-oct esta frase decía «diez defectos» y la tabla listaba doce. No fue un desfase posterior: el recuento y la tabla se escribieron en el mismo cambio del 5-oct, así que la cifra estuvo mal desde el principio y se publicó así. Se corrige aquí en vez de cambiarla en silencio. El número va ahora **atado a la tabla** —«los doce que recoge la tabla»— para que no pueda volver a divergir de ella.

Qué fallaba	Causa	Requisito
El Spider entraba en bucle y martilleaba el objetivo	No canonicalizaba las direcciones: el mismo recurso generaba variantes infinitas	RF-03
El Spider podía dirigirse a servicios internos del propio servidor	Faltaba filtrar destinos no legítimos	RNF-07
El botón de detener el Spider no existía en la interfaz	El servidor aceptaba la orden, pero la pantalla no ofrecía forma de darla	RF-03
Detener el Spider no detenía nada	El bucle de rastreo no consultaba la señal de parada	RF-03
Un mismo formulario aparecía decenas de veces	No se agrupaban: se emitía uno por página visitada	RF-03
Cancelar el Intruder devolvía un error del servidor	La operación invocaba un método inexistente	RF-06
Dos operaciones del proxy devolvían error del servidor	Un error de sintaxis construía un tipo de dato equivocado	RF-02

Importar un comando `curl` rompía las direcciones seguras	Una expresión regular mal escapada recortaba el esquema	RF-05
Los avisos de error no decían qué había fallado	Se mostraba el texto crudo de la librería de red	RF-05
Dos pantallas no recibían avisos en tiempo real	El servidor guardaba una sola conexión por sesión y la segunda desplazaba a la primera	RNF-04
El agente del cortafuegos era manipulable por cualquiera	Su canal se creaba con permisos para todo el sistema y no validaba lo recibido	RNF-08
El estado de sesión podía crecer sin límite	El recolector existía pero nunca se ejecutaba, y cualquier identificador nuevo creaba una sesión	RNF-07

Sistema de acceso por usuario (Fase 2)

Funcionalidad nueva, no un defecto corregido: cada persona se crea su cuenta, entra con ella y **solo ve sus propias auditorías**. Cubre RF-12 y es lo que cierra las cuatro fugas del apartado 7.

- **Registro con código de invitación.** La persona elige su contraseña y nadie más la conoce —ni quien administra—, pero necesita un código que reparte el grupo. No es abierto a propósito: HookSuite lanza tráfico contra terceros, así que con altas anónimas cualquiera podría usar el Spider o el Intruder contra quien quisiera desde nuestra infraestructura. El código es obligatorio por configuración: si falta, **el servicio no arranca**, en vez de arrancar con el registro abierto sin que nadie lo note.
- **Entrada con token firmado por el servidor.** Las contraseñas se guardan como hash, nunca en claro. El algoritmo de firma está fijado en el servidor y no se lee del propio token, porque aceptarlo del token permite presentar uno sin firma y que el servidor lo dé por bueno.
- **Cada usuario, su espacio.** El historial, las cookies capturadas y los hallazgos cuelgan de la cuenta de quien los generó. El identificador de ese espacio **sale siempre del token**: ni de la URL ni del cuerpo de la petición, para que el cliente no pueda elegir dónde escribe.
- **Cuentas persistentes.** Se guardan en un volumen propio, porque el despliegue reconstruye los contenedores y unas cuentas que se evaporasen en cada despliegue harían inútil el registro. El estado de sesión, en cambio, sigue siendo volátil a propósito (apartado 10): son cosas distintas.

Nota sobre el último: no era el defecto que teníamos anotado. El pendiente decía «las sesiones no se conservan al reiniciar»; al abrirlo resultó que el problema real era el contrario y más grave —podían crecer sin límite desde fuera—. La volatilidad se justifica en el apartado 10; lo que se corrigió fue el crecimiento.

El Repeater: un banco de trabajo de una sola petición (Fase 3, 12-oct)

El Repeater es la pantalla donde el auditor modifica una petición y la reenvía. Tras el pulido del 12-oct funciona así, y los criterios van explicados porque ninguno es evidente:

- **Una petición cada vez.** Mientras el Repeater tiene una cargada, el Spider **no deja enviarle otra**: su botón pasa a `repeater en uso`, en ámbar, y dice cómo liberarlo. No es una limitación técnica sino una decisión de diseño, justificada en el apartado 10.
- **Lo que se ve es lo que se manda.** Las cabeceras del editor son exactamente las que salen por el cable, y el envío no arrastra ninguna sesión por detrás. Reenviar dos veces la misma petición da dos veces el mismo resultado: esa es la prueba de aceptación de la pantalla.
- **Tres avisos que el producto antes se guardaba.** (1) **Sesión heredada:** si la petición lleva `Cookie`, `Authorization` o un token CSRF, un bloque ámbar lo dice y ofrece un botón que quita **solo** esas cabeceras, dejando intactos método, dirección y cuerpo. Probar la misma petición con y sin sesión es la operación que un auditor necesita, y antes había que borrarlas una a una. (2) **Redirección:** si la respuesta es `3xx`, el aviso dice a dónde apunta el `Location` y que el Repeater no la sigue. (3) **Sin respuesta:** un fallo de red ya no se disfraza de código HTTP `0`, ni enseña «0 ms» y «0,0 KB», que no son medidas sino la ausencia de ellas.
- **La dirección a la que se envió, siempre legible**, en su propia línea bajo la barra de estado. Con varias peticiones seguidas, saber de cuál es la respuesta que estás mirando no es un lujo.
- **El botón `preparar`, que es la pieza que faltaba.** La petición que el Spider manda al Repeater **no es una petición capturada**: es una plantilla leída del HTML de un formulario, con los campos sin valor rellenos con su propio nombre y el token de cuando se rastreó. **Nace vieja**, y al enviarla el servidor la rechaza o la rebota. `preparar` hace lo que hace un navegador antes de enviar un formulario: pide la

página, saca **todos** los campos ocultos —no solo el token, así vale para cualquier marco de trabajo—, sustituye en el cuerpo los que coincidan por nombre y actualiza la `Cookie` si el servidor la cambió. Solo toca campos que ya estaban: no inventa ni reordena.

- **Y la página preparada se queda**, en una pestaña propia junto a Raw, Pretty, Preview y Headers, con el mismo aislamiento que la vista previa. Antes, para volver a ver el formulario había que cambiar el método a GET, enviar y volver a POST. Pulsar `preparar` otra vez tras un envío fallido trae la página **con los valores que acabas de mandar** y los errores marcados: es la forma de corregir campo a campo sin salir de la pantalla. Mientras solo se ha preparado y no se ha enviado nada, las cuatro pestañas de respuesta no responden, y es correcto: no hay respuesta que enseñar.

El manual de uso gana su segunda sección. La del Repeater documenta sus **veintiséis** controles uno a uno —de qué tipo es cada uno, qué hace y bajo qué condición aparece— y el manual pasa a leerse por pastillas en vez de en una tirada, con las tres secciones sin escribir marcadas a la vista. La regla sigue siendo la del 11-oct: una sección se escribe cuando su pestaña está pulida y verificada, nunca antes.

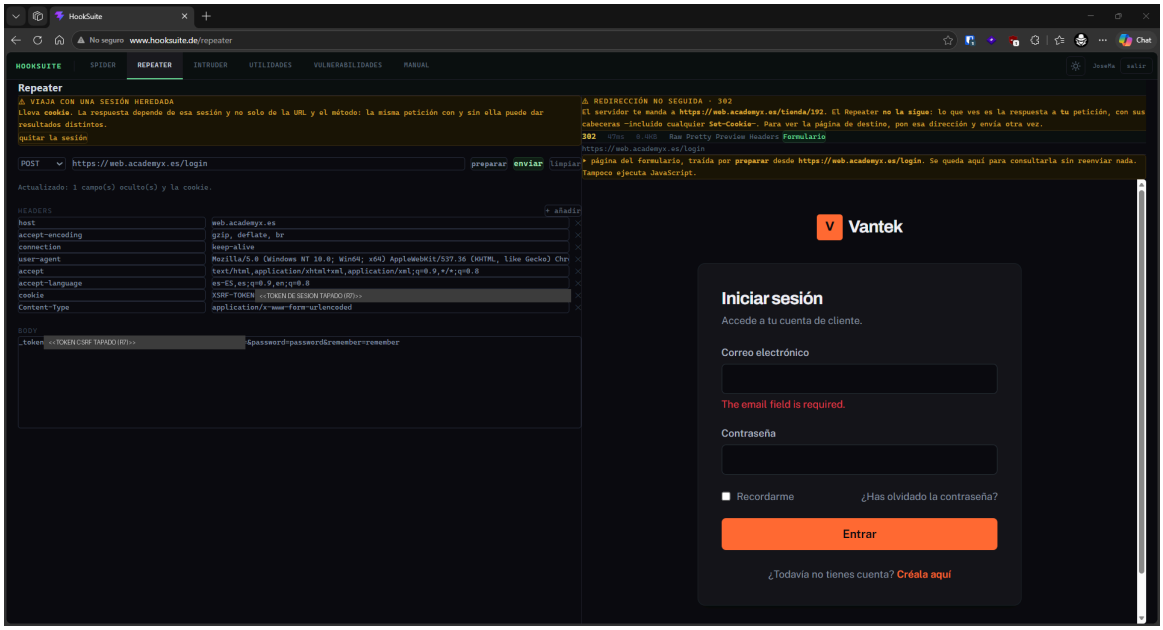


*Figura 2: RF-05 — el Repeater tras pulsar `preparar` sobre un formulario traído del Spider, con cuatro cosas a la vez: el aviso de que la petición **viaja con una sesión heredada** y el botón que la retira; el acuse de `preparar` —«1 campo(s) oculto(s) y la cookie» actualizados—; el aviso de **redirección no seguida** con el `302` y la dirección a la que manda el servidor; y la pestaña **Formulario**, que muestra la página pedida con sus estilos sin reenviar nada. Los valores de sesión van tapados (R7).*

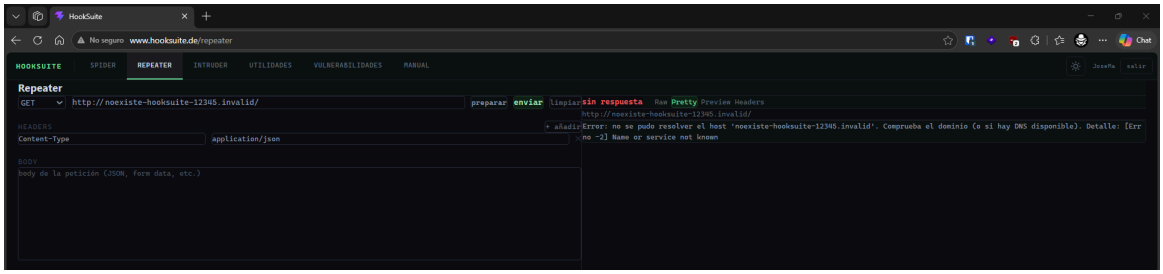


*Figura 3: RF-05 — un fallo de red deja de disfrazarse de respuesta. Donde iría el código HTTP aparece «sin respuesta» en rojo, y el tiempo y el tamaño **desaparecen** de la barra: 0 ms y 0,0 KB no son medidas, son la ausencia de ellas. Debajo, la dirección a la que se envió y el motivo exacto —el host no se pudo resolver—.*

Las funcionalidades, una a una: qué hace, cómo se usa y qué requisito cubre

Lo anterior cuenta **lo que cambió** en cada fase. Esta tabla cuenta **lo que hay**, pantalla por pantalla, tal como queda en la versión congelada: el panel tiene **seis pestañas** más la pantalla de acceso, y la columna de la derecha dice dónde se ve cada una sin repetir aquí figuras que ya están en otros apartados.

Pantalla	Qué hace y cómo se usa	Requisitos	Dónde se ve
Spider	Rastrea el objetivo siguiendo enlaces y formularios y levanta su mapa. Se escribe la dirección, se lanza, y la tabla se va llenando; se puede **detener** a medias, filtrar por método y mandar una petición al Repeater. Las cabeceras que muestra son **las que se enviaron de verdad**, no una plantilla	RF-03	figuras del apdo. 8 (antes y después del pulido)
Repeater	Banco de trabajo de **una** petición: se modifica y se reenvía **exactamente lo que se ve en pantalla**. Avisa si la petición lleva sesión heredada y ofrece quitarla; el botón `preparar` convierte la plantilla de un formulario en una petición que funciona; la respuesta se lee en Raw, Pretty, Preview, Headers y la página preparada	RF-04 · RF-05	cinco figuras en los apdos. 6 y 8
Intruder	Prueba muchos valores en un punto marcado de la petición y señala las respuestas que se salen del patrón. Se marca el campo, se elige diccionario, se lanza y se puede **cancelar** sin dejar el servidor trabajando	RF-06	**falta captura del producto congelado**
Utilidades	Cuatro herramientas que no salen a la red: **codificador** (base64, URL, HTML y **lectura de JWT**), **generador de huellas**, **probador de expresiones regulares** y **generador de payloads**. Se pega el texto y se ve el resultado al lado	RF-07	**falta captura del producto congelado**
Vulnerabilidades	Panel donde aterrizan los hallazgos con su severidad y su **confianza**, y desde donde se dispara la auditoría con IA. Si el modelo falla o no supera el umbral, el hallazgo aparece como **no analizado** en lugar de desaparecer	RF-08 · RNF-06	figura del apdo. 8 (operación degradada)
Manual	Manual de uso **dentro del propio producto**, control por control y navegable por pastillas. No lo pide ningún requisito: salió del pulido, al comprobar que explicar una pantalla destapa sus defectos	— (mejora)	figura del apdo. 8
Acceso (fuera de la barra)	Alta con código de invitación y entrada con token firmado por el servidor. Cada cuenta ve **solo sus propios datos**	RF-11 · RF-12	cuatro figuras de producción en el apdo. 7

Tres piezas trabajan sin pestaña propia, y conviene decirlo porque quien busque su pantalla no la va a encontrar: el **proxy del lado servidor** (RF-02) es el que ejecuta las peticiones por debajo de Spider, Repeater e Intruder; el **navegador real** (RF-10) lo dirige el módulo de IA, no una persona; y el **cortafuegos por sesión** (RNF-08) es un agente del sistema. La séptima pestaña que hubo, **Red** (RF-09), se retiró al descartar ese requisito —su figura se capturó **antes** de retirarla, que es lo que exige R6—.

Dos notas de honestidad sobre esta tabla.

- **La pestaña que se llama Spider vive en la ruta `/proxy`**. Es un resto del nombre que tenía antes del pivote de RF-02, y se deja dicho porque cualquiera que mire las rutas lo va a notar.
- **El manual interno cubre dos de sus cinco secciones**: Spider y Repeater escritas, e Intruder, Utilidades y Vulnerabilidades **marcadas como pendientes dentro del propio producto**. Es la misma regla que esta memoria aplica a sus huecos: quien usa la herramienta ve que falta, en vez de encontrarse una sección vacía.

Y una corrección de lo que esta memoria dio por roto. La verificación del 29 de septiembre anotó que «el codificador no soporta JWT aunque RF-07 lo promete». **Hoy sí lo soporta**: la lista de conversiones del codificador incluye la lectura de JWT, junto a base64, URL y HTML. RF-07 queda **cumplido** sin salvedades, y se dice aquí porque el defecto se declaró en su día y no se puede retirar en silencio (R9).

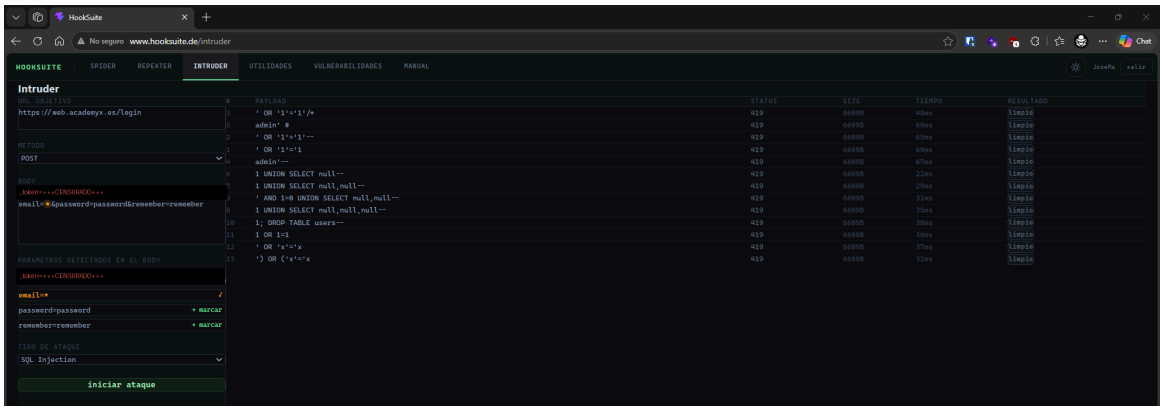


*Figura 4: RF-06 — el Intruder con un ataque de inyección SQL lanzado contra el formulario de acceso del objetivo propio. El diccionario está a la vista: **trece payloads** numerados, de `' OR '1'='1` a `1; DROP TABLE users--`, cada uno con su código, tamaño, tiempo y veredicto. A la izquierda, los **parámetros que el Intruder detecta solo** dentro del cuerpo de la petición, y cuál va marcado con el asterisco. Los trece responden `419` y el panel los marca **limpio**: el objetivo exige un testigo CSRF válido en cada envío y rechaza el reenvío, de modo que lo que la captura demuestra es el **mecanismo de fuzzing** —envío, medida y clasificación— y no un hallazgo. El testigo va tapado (R7).*

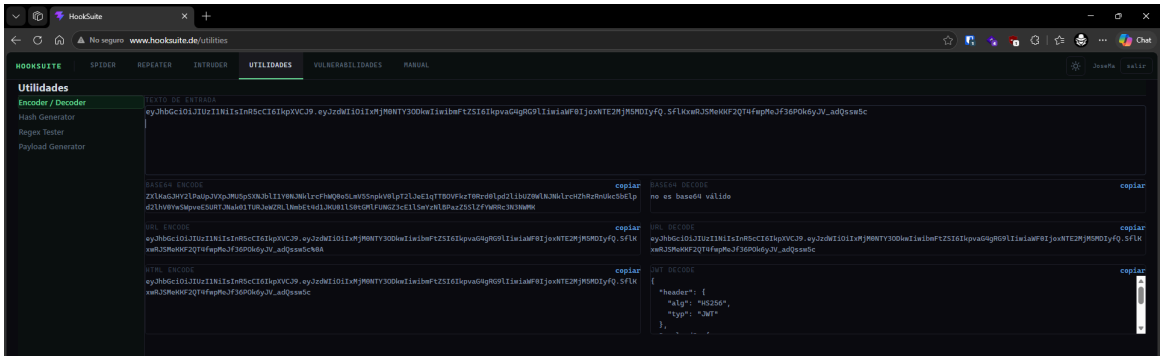


*Figura 5: RF-07 — el codificador con un JWT en la entrada y sus cuatro conversiones resueltas a la vez: base64, URL, HTML y la **lectura del JWT**, que descompone la cabecera (`HS256`) y la carga útil. Es justo la conversión que la verificación del 29 de septiembre dio por ausente, y la que cierra RF-07 sin salvedades. El testigo mostrado es el **ejemplo público de documentación** —la carga útil es el «John Doe» de manual—, no una credencial del producto.*

7. Seguridad del producto

Modelo de amenazas, secretos, autenticación, validación de entradas, dependencias y datos personales. (Base en `REQUISITOS.md` §3.) Contenido ya verificado en esta práctica:

Secretos: el incidente de las claves

En el historial de git había claves de API de Anthropic. Verificado el 4-oct: **las cuatro están muertas** (la API responde `401 / "API key is invalid."`). Aun así, el enunciado (§7) obliga a limpiarlas del historial, no basta con borrarlas en un commit nuevo.

- El 4-oct se **reescribió el historial** con `git-filter-repo`: se redactaron las claves por patrón y se eliminaron los `.env` rastreados y la basura (node_modules, venv de Windows), conservando el código, las 7 ramas y el `.env.example` (con la clave ya redactada).
- **Ejecutado en GitHub el 4-oct** (force-push de las 7 ramas, con copia de seguridad previa). Verificado sobre un clon nuevo: **0 claves en todo el historial** de las ramas. El repositorio público queda sin secretos.
- Salvedad: GitHub conserva un tiempo los refs internos de los **pull requests** antiguos apuntando a los commits viejos; los purga por su cuenta. Como las claves están muertas, no supone riesgo.

LIMPIEZA EJECUTADA Y VERIFICADA

Exposición de la caja: cómo estaba

Estado de partida, verificado en vivo el 29-sep y el 4-oct (evidencia fotográfica en el apartado 8):

- La API `:8000` **no tenía autenticación**: lo declaraba su propia `openapi.json` (40 operaciones sin seguridad) y `/health` respondía sin credenciales.

- El Basic Auth del `:80` se **evitaba** sirviendo el frontend directamente por `:3000`.
- El servidor **no tenía cortafuegos** (política de aceptación por defecto) ni protección contra fuerza bruta, y acumulaba unos **127.000 intentos de acceso en 7 días**.
- DVWA **no** estaba expuesto a internet (solo red interna) — corrige un supuesto previo nuestro.

Endurecimiento aplicado en la Fase 1 (5-oct)

Cierre de la exposición

La API y el frontend dejaron de publicar puerto propio: **la única entrada es el proxy**. No bastaba con retirarlos del `docker-compose.yml`, porque el frontend llevaba la dirección del servidor incrustada en tiempo de compilación y hablaba directamente con la API; hubo que **reconstruirlo para que use el mismo origen** y que sus llamadas (incluido el WebSocket) pasen por el proxy. Comprobado desde fuera tras el despliegue: los puertos directos **rechazan la conexión**, el proxy responde, y el paquete del frontend ya no contiene ninguna referencia a la dirección antigua.

Orígenes permitidos (CORS)

El backend aceptaba **cualquier** origen con credenciales, combinación además inválida por especificación: en la práctica, el servidor **reflejaba el origen de quien preguntara**, de modo que cualquier web podía hacerle peticiones autenticadas. Se sustituyó por una **lista cerrada** configurable por entorno. Comprobado: un origen arbitrario ya no recibe permiso; uno legítimo sí.

El laboratorio vulnerable: fuera del producto

En la Fase 1 se le retiró del proxy la ruta que lo publicaba, de modo que quedó accesible solo desde la red interna. El **13-oct**, al congelar, se retiró del producto **entero**: no está en el despliegue ni en la caja — ni el servicio, ni el contenedor, ni la imagen—.

Por qué, y es una decisión del 1-oct que tardó doce días en aplicarse. Las pruebas de este proyecto se hacen contra una web propia del equipo, autorizada para ello, y no contra el laboratorio clásico; mantenerlo en el despliegue significaba entregar un producto que arrastra un sistema **deliberadamente vulnerable** que ya nadie usaba. Se retiró de la composición de contenedores, del entorno del módulo de navegador, y del único punto del código que todavía caía a él cuando llegaba una orden sin objetivo: ahora esa orden **se descarta y se dice**, en lugar de auditar un host que ya no existe.

Lo que no se tocó, y se declara. El módulo de navegador conserva una función de acceso a ese laboratorio, y los datos de ejemplo del frontend lo siguen nombrando. Son RF-10 —parcial— y código que no se ejecuta en el producto entregado; reescribirlos el día de la congelación añadía riesgo sin ganar nada. Queda anotado en el apartado 10.

Endurecimiento del servidor

- **Cortafuegos** activo, con política de denegación por defecto y únicamente el acceso remoto y el proxy permitidos.
- **Bloqueo automático de fuerza bruta** contra el acceso remoto: en su primer minuto ya registraba 148 intentos fallidos y había bloqueado dos direcciones.
- **Acceso remoto solo por clave**: se desactivó la autenticación por contraseña. La prueba más limpia es el propio mensaje del servidor, que pasó de ofrecer «clave o contraseña» a admitir **solo clave**.
- **Limpieza de reglas abandonadas**: siete reglas heredadas de la P1 —una de ellas con la dirección pública de un integrante y otra con un valor de ejemplo— se retiraron tras comprobar que ningún servicio escuchaba en esos puertos.
- **Agente de firewall duplicado**: se encontró un segundo proceso del agente, lanzado a mano en mayo y fuera del gestor de servicios, con permisos de administración del cortafuegos. Se terminó; queda uno solo, gestionado por el sistema.
- **Sistema operativo**: se estrenó el núcleo ya instalado pero sin aplicar desde mayo. Antes de reiniciar se corrigió una condición de carrera en el arranque que habría dejado el agente del cortafuegos en bucle (ver apartado 8).

Dos matices honestos, para no atribuirnos de más

- **El acceso de administrador por contraseña ya estaba bloqueado** por la configuración por defecto del sistema: los 127.000 intentos registrados nunca tuvieron ninguna posibilidad. Lo que se cerró fue la autenticación por contraseña **a nivel general**. Se hizo igual porque deja de depender de un valor por defecto que una actualización podría cambiar, y porque cierra la puerta a cualquier usuario futuro.
- **El cortafuegos del sistema no filtra los puertos que publica Docker**, cuyas reglas se evalúan antes. Quien cerró de verdad la API y el frontend fue el despliegue, al dejar de publicarlos. El cortafuegos protege el resto del servidor, no esos puertos.

EXPOSICIÓN CERRADA Y SERVIDOR ENDURECIDO

Autenticación y aislamiento: de cuatro fugas a cero (Fase 2)

Hasta la Fase 2, la API y el WebSocket **no tenían autenticación** por detrás del proxy: el único control era el Basic Auth de la entrada, que además no distinguía personas. Eso no era una carencia, sino **cuatro fallos distintos**, los cuatro medidos en la cocina el 6-oct antes de tocar una línea de código. La tabla enfrenta cada uno con lo que lo cierra; la evidencia de los dos lados está en el apartado 8.

Cómo estaba (medido el 6-oct)	Cómo queda (Fase 2)
La API entera respondía sin credencial. La asimetría lo delataba: `GET /health` devolvía `401` por el Basic Auth de la entrada, pero `GET /api/session/new` devolvía `200` y entregaba un token. El proxy protegía unas rutas y dejaba `/api` abierta.	Un guardián aplicado **por router**, no ruta por ruta, para que una ruta nueva nazca protegida. Toda `/api` y `/ws/` exigen token: sin él, `401` con cabecera `WWW-Authenticate`. El Basic Auth de la entrada se retiró, porque la credencial la pide ahora quien debe.
El token de sesión no era una credencial, era un nombre. El backend **creaba** sesión para cualquier cadena inventada, así que no había frontera que violar: bastaba escribir un token cualquiera en la URL.	El token lo **firma el servidor** y el espacio de datos se deduce del token, nunca de la URL ni del cuerpo. El navegador dejó de inventarse su identificador. La ruta que repartía identificadores sin valor de credencial se retiró.
La cookie de sesión de la web auditada se compartía entre todos. Se guardaba en una tabla global indexada solo por dominio, sin dueño; un segundo usuario leía la cookie capturada por el primero, y se obtenía además sin token.	Las cookies viven dentro de la sesión de cada usuario y la tabla global desapareció. Verificado: el segundo usuario ya no obtiene la cookie del primero.
Los hallazgos se difundían a todos los paneles. Una vulnerabilidad publicada sin token aparecía en la sesión de dos auditores distintos: en una herramienta de auditoría, eso es una fuga de datos del cliente de otro.	Los dos endpoints que admitían peticiones sin token escriben ahora solo en el espacio de quien llama. Verificado: el segundo usuario no ve el hallazgo del primero.

Un quinto fallo, que introdujo la propia corrección

Se documenta porque el enunciado pide las pruebas que fallaron, y porque la lección vale más que el acierto. El guardián comprobaba que el espacio de datos nombrado **en la ruta** fuera el tuyo, pero **cinco rutas reciben ese nombre en el cuerpo de la petición**, y ahí no miraba nadie: un usuario autenticado podía escribir en el espacio de otro con solo ponerlo en el cuerpo. Es el mismo error que este documento le reprocha al código anterior —dejar que el cliente elija dónde escribe— un nivel más abajo.

La peor de las cinco fue el Repeater, y se escapó del primer inventario por buscarla con el nombre de modelo equivocado. Ahí el daño no era ensuciar el historial ajeno: el Repeater mantiene **un cliente HTTP persistente por usuario** que acumula las cookies del objetivo auditado —el mecanismo que da servicio a RF-04—, de modo que nombrar a otro en el cuerpo reutilizaba **su sesión ya autenticada contra la web auditada**.

Se descartó la corrección evidente, que era validar ese campo contra el token: funciona, pero deja el dato en manos del cliente y obliga a acordarse en cada modelo nuevo. Las rutas toman el espacio del token y **descartan** lo que venga en el cuerpo: lo que no se lee no se puede falsear. Comprobado como ataque real (apartado 8).

Desplegado y verificado en producción (8-oct)

Hasta esta fecha, este apartado decía expresamente que no podía afirmar nada de la instalación pública: todo estaba **verificado en la cocina**, y la caja seguía sirviendo la Fase 1 con las cuatro fugas abiertas. Se dejó escrito así a propósito, porque la tentación de contar una mejora en presente antes de que llegue

a producción es justo como una memoria acaba describiendo un producto que no existe. El 7 de octubre se comprobó además **en vivo y en la propia instalación pública** que las fugas no eran teóricas: la API respondía sin credencial a cualquiera.

El despliegue se hizo el **8 de octubre a las 06:52 UTC**, en cuatro pasos con punto de retorno previo: se etiquetaron las imágenes en servicio y se copió la configuración, se mergeó el cambio a la rama principal, se trajo el código a la caja con `git reset --hard` y se generaron los secretos, y se reconstruyeron los contenedores. La verificación inmediata, con el sistema ya en marcha:

Qué se midió	Resultado
Entrada del panel (`/`)	`200` — carga **sin pedir Basic Auth**: la contraseña compartida está retirada
API (`/api/auth/yo`)	`401` — **exige token**; la fuga nº 1 de la columna izquierda, cerrada
Reinicios del backend	`0` — prueba que los secretos llegaron: el código se niega a arrancar sin ellos
Contenedores recreados	backend, frontend y nginx; los otros cuatro, intactos a propósito

El dato de los **cero reinicios** no es decorativo: el acceso por usuario se diseñó para **no arrancar** si faltan el secreto de firma o el código de invitación, en lugar de arrancar sin autenticación pareciendo correcto. Que el contenedor esté sirviendo sin un solo reinicio es la prueba de que esa comprobación se superó.

El flujo completo del acceso, capturado en la instalación pública el mismo día del despliegue:

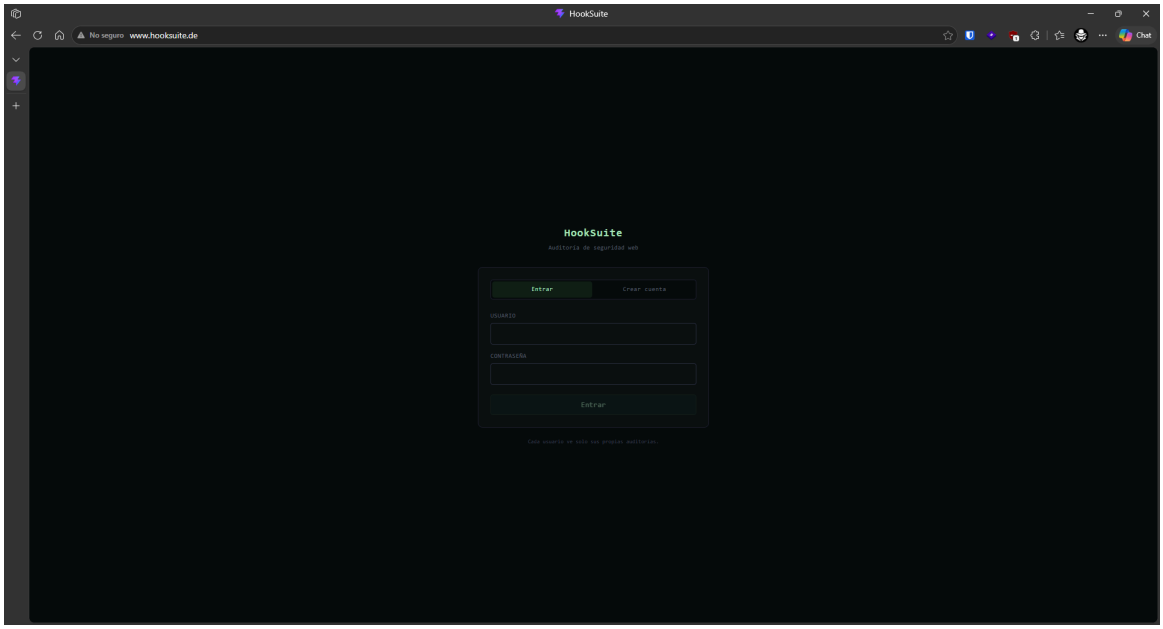


*Figura 6: RF-12 — la entrada de `www.hooksuite.de` carga **sin** el cuadro de Basic Auth que antes pedía una contraseña compartida por todo el grupo. La puerta la guarda ahora el acceso por usuario.*

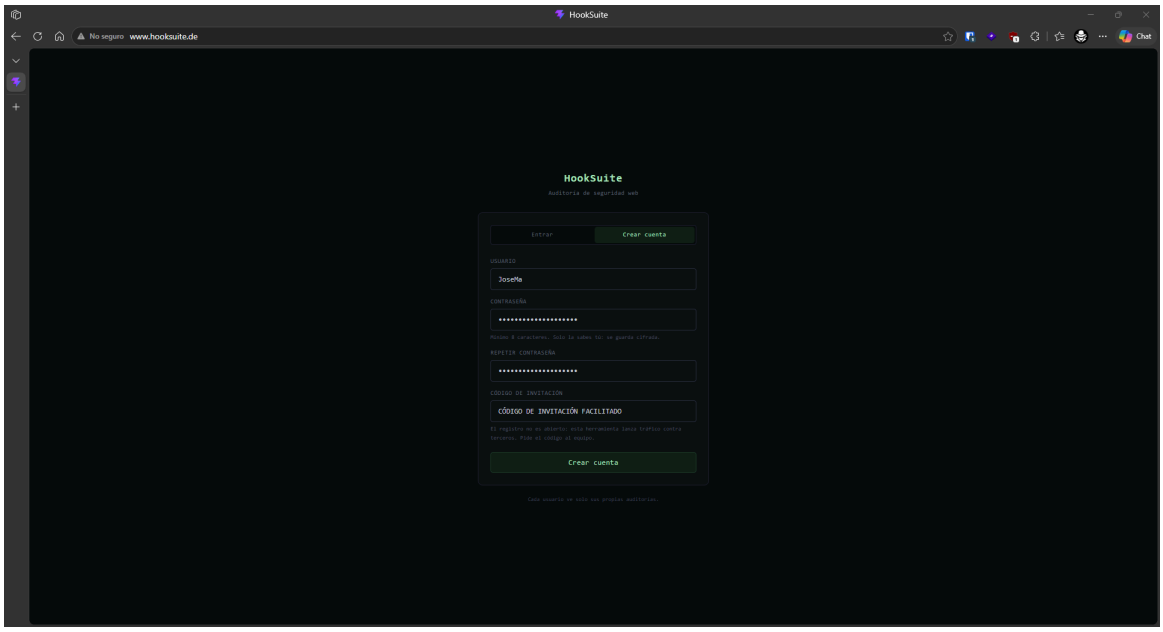


*Figura 7: RF-12 — el alta exige un **código de invitación**. El registro no es abierto a propósito: la herramienta lanza tráfico contra terceros, y con altas anónimas cualquiera atacaría a quien quisiera desde nuestra infraestructura. El campo de la imagen muestra el texto de ayuda, no el código.*

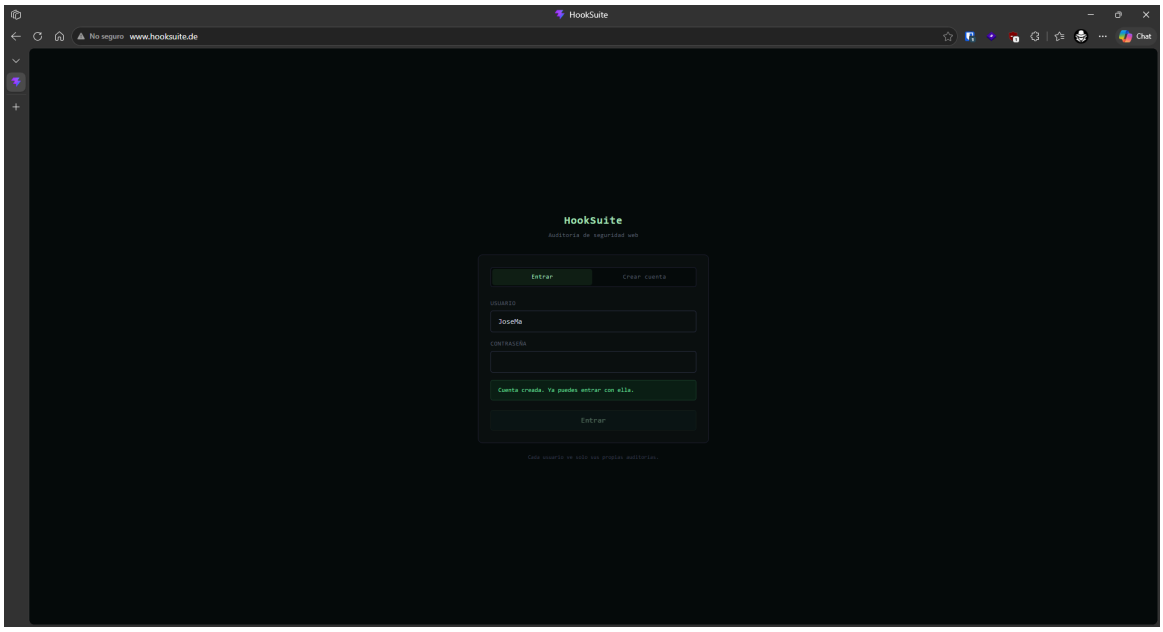


Figura 8: RF-12 — confirmación del alta del primer usuario de producción, creado con el código de invitación. La instalación arrancó deliberadamente con cero usuarios.

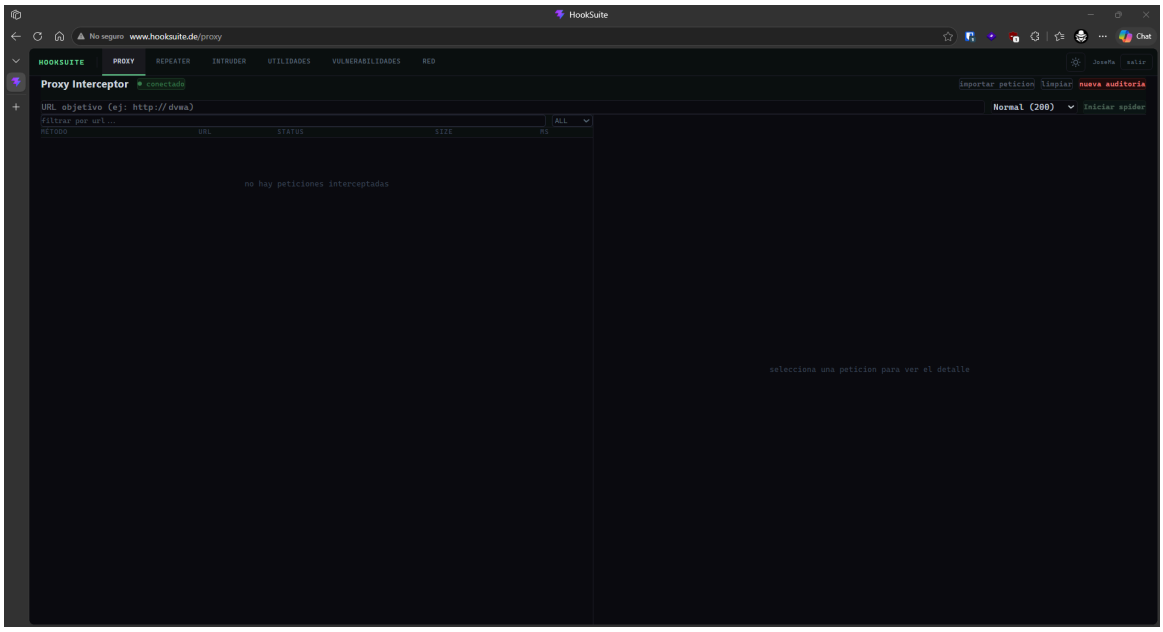


*Figura 9: RF-12 y RNF-07 — sesión iniciada: el usuario figura en la cabecera y el interceptor aparece como **conectado**, lo que prueba que el WebSocket viaja también autenticado y no solo la API.*

Lo que sigue sin poder afirmarse, y no se disfraza. El dominio publica también una dirección IPv6, y por ese camino la comprobación devolvió `404` en lugar del panel. No está confirmado qué ve un visitante real que llegue por IPv6, porque desde el equipo de pruebas no hay salida por esa vía. Mientras no se compruebe desde una red con IPv6, la disponibilidad solo está demostrada por IPv4.

CUATRO FUGAS CERRADAS Y VERIFICADAS EN PRODUCCIÓN (8-oct, 06:52 UTC) · PENDIENTE: COMPROBAR LA LLEGADA POR IPv6

Dos efectos del pulido del Repeater sobre la seguridad del producto (12-oct)

- **Cerrado: el último residuo del modelo anterior al login.** La petición de envío del Repeater llevaba en su cuerpo el identificador del espacio de datos, y el esquema del servidor seguía declarándolo. No se leía —el espacio sale siempre del token firmado, nunca del cuerpo— pero **sugería que el cliente puede elegir dónde escribe**, que es exactamente el hueco de autorización que cerró la Fase 2: con el nombre de otro en el cuerpo se reutilizaban **sus** cookies del login auditado. Se retiró del frontend el 12-oct a mediodía y del esquema del servidor esa misma noche. Un campo muerto que insinúa una capacidad que no existe es deuda de seguridad, no ruido.
- **Nuevo, y es una contrapartida asumida: el panel muestra ahora la cookie de sesión del objetivo.** Al corregir que las cabeceras mostradas sean las realmente enviadas (apartado 5), la `Cookie` del sitio auditado pasa a estar a la vista, como en la herramienta de referencia. Es lo correcto —la evidencia tiene que ser la petición que salió— y tiene un coste de método que conviene declarar: **cualquier captura de ese panel publica credenciales de un tercero**. Esas capturas van censuradas antes de entrar en esta memoria o en el artefacto (R7); ya ocurrió dos veces el mismo día. Lo que esto arrastra en el navegador del auditor —el borrador guarda esas cabeceras— se trata como limitación abierta en el apartado 10.

Lo que sigue sin cubrir

- **Sin TLS** (RNF-09): el acceso viaja en claro, así que el token de sesión es interceptable por quien esté en el camino. Pendiente de configurar en el proxy.
- **Credencial de servicio para los módulos internos** (Fase 3). **Reescrito el 13-oct por R9: lo que decía aquí se había quedado atrás dos veces.** Afirmaba que ninguno de los dos módulos llegaba a las rutas protegidas —el clasificador porque RF-08 «todavía no tiene disparador», el agente de navegador porque ni siquiera alcanzaba el servidor— y que el `401` aparecería **en cuanto** RF-08 tuviera disparador. Las dos premisas caducaron: el disparador existe desde el **10-oct** y el fallo de red del agente se corrigió el **9-oct** y llegó a producción el **13-oct**.

Lo que de verdad ocurre hoy, y son dos cosas distintas. El **clasificador de IA ya no necesita credencial**: se le movió al bus interno precisamente para eso (apartado 5), recibe las órdenes por ahí y sus hallazgos vuelven por el mismo camino. Que RF-08 funcione de punta a punta —con los hallazgos en el panel, figura del apartado 8— es la prueba: si su camino de vuelta diera `401`, el panel estaría vacío. Para él esto no es un pendiente, está **resuelto por diseño**.

El agente de navegador sí sigue sin credencial, y ese es el hueco real. Sondea por HTTP la ruta de instrucciones y publica en la de resultados, las dos bajo el prefijo protegido y **sin cabecera de autorización**, así que recibe `401`. Hasta hoy el síntoma estaba tapado por el fallo de red —no llegaba ni a intentarlo—; desde el despliegue de hoy ya está en la red interna, de modo que el `401` es lo único que le separa de operar. Queda como trabajo pendiente de RF-10, y la salida no es repartir una credencial más: es pasarlo **al mismo bus que el clasificador**, por las tres razones del apartado 5.

> **Corrección declarada de este documento (R9).** Una versión anterior de este apartado justificaba lo anterior diciendo que ninguno de los dos módulos estaba en marcha. Era falso: valía para el entorno de pruebas, donde no se levantan, y se escribió sin comprobar la instalación de producción. La conclusión se sostiene, pero por un motivo distinto del que se dio, y la diferencia importa: uno de los dos está vivo y conectado al servidor.

- **Modelo STRIDE, validación de entradas, dependencias y datos personales**, abajo.

Modelo de amenazas (STRIDE)

El modelo se aplica a los seis componentes del apartado 5 y a la frontera que movió la Fase 2: hasta entonces autenticaba el proxy con una contraseña compartida; hoy autentica el backend, por persona.

Una premisa que conviene decir antes de la tabla, porque un lector la echaría en falta: HookSuite es una herramienta **ofensiva**, y su función es enviar tráfico arbitrario a un objetivo que elige quien la usa. Por eso el control no puede ser «no permitir destinos arbitrarios» —eso sería desactivar el producto— sino **que solo entren personas autenticadas y que cada una responda de lo suyo**. Lo que en otra aplicación sería una vulnerabilidad (hacer peticiones a donde diga el usuario) aquí es el requisito RF-02.

Amenaza	Dónde aplica en HookSuite	Qué lo contiene hoy	Qué queda abierto
S — Suplantación	Entrar como otro auditor; hablar con la API sin ser nadie	Entrada única por el proxy; `/api/` y `/ws/` exigen token firmado por el servidor, con el algoritmo **fijado en el servidor** y no leído del token; alta con código de invitación; contraseñas en hash	**Sin TLS, el token es interceptable en el camino** (RNF-09). El agente de navegador sondea sin credencial propia (arriba)
T — Manipulación	Tocar datos de otra cuenta; manipular lo que recibe el agente de cortafuegos	El espacio de datos **sale siempre del token**, nunca de la ruta ni del cuerpo —es la lección del quinto fallo—; modelos declarados que rechazan la petición mal formada antes de la lógica; el canal del agente dejó de crearse con permisos para todo el sistema y valida lo que recibe	En claro y sin TLS, manipulable en tránsito
R — Repudio	Saber quién lanzó qué contra un objetivo	Cada petición del historial, cookie capturada y hallazgo cuelga de la cuenta que lo generó; el repositorio identifica autoría por confirmación	**No hay registro de auditoría propio**: los rastros son los del contenedor, que no se conservan ni se firman. Se declara, no se resuelve
I — Divulgación	Ver auditorías ajenas; filtrar secretos o cookies del objetivo	Cuatro fugas de aislamiento cerradas y una quinta introducida y corregida; las cookies del objetivo cuelgan de su dueño y no de una tabla global; claves fuera del historial; orígenes restringidos; el paquete del frontend ya no lleva la dirección directa de la API	Sin TLS todo viaja en claro. El borrador del Repeater guarda la cookie del objetivo en el navegador (apartado 10)

D — Denegación de servicio	Agotar el servidor, el saldo de la API o martillear al objetivo	Límite de peticiones simultáneas (RNF-05); **techo de llamadas** a la IA, configurable y con degradación visible en el panel; el recolector de sesiones, que existía y no se ejecutaba, ahora corre; en el host, cortafuegos y bloqueo de fuerza bruta	La vía de huella digital no compara contra el techo; el Spider no tiene límite de ritmo propio contra el objetivo
E — Elevación de privilegio	Que un usuario del producto mande sobre el cortafuegos del host	El canal del agente pasó a tener permisos acotados y a validar; se retiró un segundo agente lanzado a mano en mayo, fuera del gestor de servicios y con permisos de administración; acceso remoto solo por clave	El agente de cortafuegos es, **por diseño**, un componente con privilegio sobre las reglas del host: su superficie es ese canal, y no desaparece

Validación de entradas

- **Modelos declarados en el borde, no comprobaciones dispersas.** El backend declara su entrada con modelos en seis ficheros —el de esquemas comunes y las rutas de alta, red, navegador, rastreo y vulnerabilidades—, así que una petición mal formada se rechaza **antes** de llegar a la lógica.
- **Todas las rutas protegidas menos la puerta.** En `main.py` se registran nueve grupos de rutas y **todos** llevan la dependencia de protección salvo el de alta y entrada, que es por donde se consigue el token.
- **El identificador del espacio de datos no se acepta del cliente.** Se descartó la corrección evidente —validar el campo contra el token— porque deja el dato en manos de quien llama; las rutas lo toman del token y punto. Es lo que cierra la familia entera del quinto fallo, no solo el caso encontrado.
- **Canonicalización en el Spider:** decodificar entidades, normalizar y deduplicar la cadena de consulta. Sin eso, el rastreo generaba direcciones infinitas y martilleaba al objetivo (la trampa de araña del 1-oct).
- **El importador de comandos `curl`** corrigió una expresión regular que recortaba el esquema y convertía direcciones seguras en inseguras.
- **En el módulo de IA, la validación es doble:** el esquema de respuesta se endureció tras **tres rechazos seguidos** de la API —validar contra una prueba propia no es validar— y el techo de llamadas corta por arriba, de modo que una entrada que dispare una auditoría enorme no se convierte en gasto ilimitado.
- **Lo que no se valida, y es deliberado:** la dirección del objetivo. Es la función del producto; el control está en el acceso, no en el destino.

Dependencias

Medido el **13-oct sobre el código congelado**, con las herramientas puestas para ello (`npm audit` sobre el fichero de bloqueo y `pip-audit` sobre los requisitos declarados). Se dan los números que dan las herramientas, sin redondear hacia abajo.

- **Frontend:** 241 dependencias resueltas (84 de producción, 113 de desarrollo, 45 opcionales) y **12 avisos** —nueve altos, uno medio, dos bajos—. Restringido al árbol de producción, **ocho**: siete altos y uno bajo. Directas: el cliente HTTP del panel (alto), el enrutador (bajo, por su paquete interno) y la herramienta de construcción (alto, que entra en el árbol de producción a través del complemento de estilos).
- **Backend: 78 avisos en 12 paquetes** del árbol resuelto. Los que más pesan, y el primero importa más que los otros: **la librería que firma los tokens de sesión acumula trece avisos** en la versión fijada —es la pieza central de la Fase 2—, la de criptografía siete, la de formularios multiparte siete y el armazón ASGI bajo FastAPI otros siete.
- **El proxy de la P1 sigue pesando, y se comprobó antes de decirlo.** `mitmproxy` aporta tres avisos propios y arrastra cinco paquetes más (servidor web, compresión, serialización, protocolo binario y TLS). La tentación era declararlo código muerto y proponer quitarlo: **no lo es.** `main.py` y la ruta de alta importan el gestor que lo lanza como subproceso con su complemento, aunque RF-02 se implemente hoy con el cliente HTTP nuevo y las rutas del fichero de autoconfiguración se retirasen en la Fase 2. **Las dos arquitecturas conviven**, y esa convivencia es lo que mantiene viva esta superficie.

- **Política de fijado, desigual:** el backend fija sus doce versiones; el módulo de navegador tres de cuatro y el de herramientas de depuración las tres; **el módulo de IA solo una de cuatro**, así que su construcción no es reproducible al día de hoy. El frontend declara rangos, pero el fichero de bloqueo fija el árbol.
- **Qué no se ha hecho, y por qué: no se ha actualizado ninguna dependencia hoy.** Es el día de la congelación, el vídeo se graba sobre esta etiqueta y subir la librería de tokens o el cliente HTTP cambia el código que se entrega. Queda declarado aquí y en el apartado 10.
- **Honestidad de método:** `pip-audit` resuelve el árbol completo, así que esos 78 incluyen transitivas, y **no se ha medido cuáles son alcanzables** desde el camino de ejecución del producto. Decir «78 avisos» no es decir «78 agujeros explotables»; es decir cuántos hay que mirar.
- **Por qué los paquetes se nombran aquí por su papel y no por nombre y versión.** El repositorio es público y el producto está desplegado **sin TLS**: publicar la lista exacta de paquete, versión y número de avisos es publicar un mapa de por dónde entrar. Las versiones están en los ficheros de requisitos para quien tenga acceso al repositorio, y cualquiera puede reproducir la medida con las dos órdenes citadas arriba. Es la misma regla que se aplicó a las claves: se cuenta el incidente, no se publica el valor (R7). **Única excepción:** el proxy de la P1, que esta memoria ya nombra en los apartados 4 y 5 al explicar el pivote de RF-02; ocultarlo aquí no taparía nada.

Tratamiento de datos personales

- **Qué guarda el producto, y de quién.** De cada auditor: su cuenta —nombre y contraseña **en hash**, nunca en claro— en un volumen propio, y colgando de ella su historial de peticiones, las cookies que captura del objetivo y sus hallazgos. Desde la Fase 2 nada de eso se comparte entre cuentas.
- **De terceros, nada por iniciativa propia.** El producto no recolecta datos personales por su cuenta: lo que entra es lo que el auditor captura del objetivo que decide auditar. Las pruebas de esta práctica se hacen contra una **web propia del equipo y autorizada**, de modo que no hay datos de terceros en juego.
- **La cookie del objetivo es el dato más sensible que pasa por aquí**, porque equivale a una sesión abierta en la web auditada. Vive en el espacio de su dueño, y el editor avisa cuando una petición la lleva. Pero el **borrador** del Repeater la escribe en el almacenamiento del navegador: está declarado en el apartado 10 y no se tocó el día de la congelación.
- **Rastros a retirar antes de la entrega:** los usuarios de prueba creados en la cocina, y una regla heredada de la P1 que contenía la dirección pública de un integrante —esa ya se retiró en la Fase 1—.
- **Conservación y borrado:** no hay caducidad ni borrado automático de cuentas ni de historial; el recolector solo caduca el estado efímero de sesión. Se declara como lo que es: una decisión no tomada, no una medida.
- **Y el que lo atraviesa todo:** sin TLS, todo lo anterior viaja en claro (RNF-09). Es la limitación que más pesa en este apartado.

8. Pruebas y evidencias

Plan de pruebas, casos ejecutados, resultados y las pruebas que fallaron con su explicación. El detalle caso a caso (PR-01…PR-20) se mantiene en la matriz del apartado 9 y en el diario del repo.

Dónde se tomó cada evidencia, porque cambia lo que vale. Salvo cuando se dice lo contrario, **toda** la verificación de las Fases 1 y 2 se ejecutó en la cocina —el entorno de pruebas descrito en el apartado 5—, y se repitió después contra la instalación pública tras cada despliegue. Las dos se registran por separado y con su fecha: una prueba en la cocina dice que el arreglo funciona; solo la de producción dice que **está puesto**. Esa distinción no es formalismo: más abajo se cuenta un despliegue que habría pasado por correcto sin aplicar la configuración que se le pedía.

Evidencia de exposición (RNF-07 / PR-16), capturada el 4-oct

La prueba PR-16 —llamar a la API sin credenciales— y la verificación del frontend confirman que, antes de endurecer, el producto quedaba accesible sin autenticación. Capturas obtenidas antes de cualquier arreglo (R6: la evidencia de un fallo se captura antes de corregirlo, porque el propio arreglo la destruye):

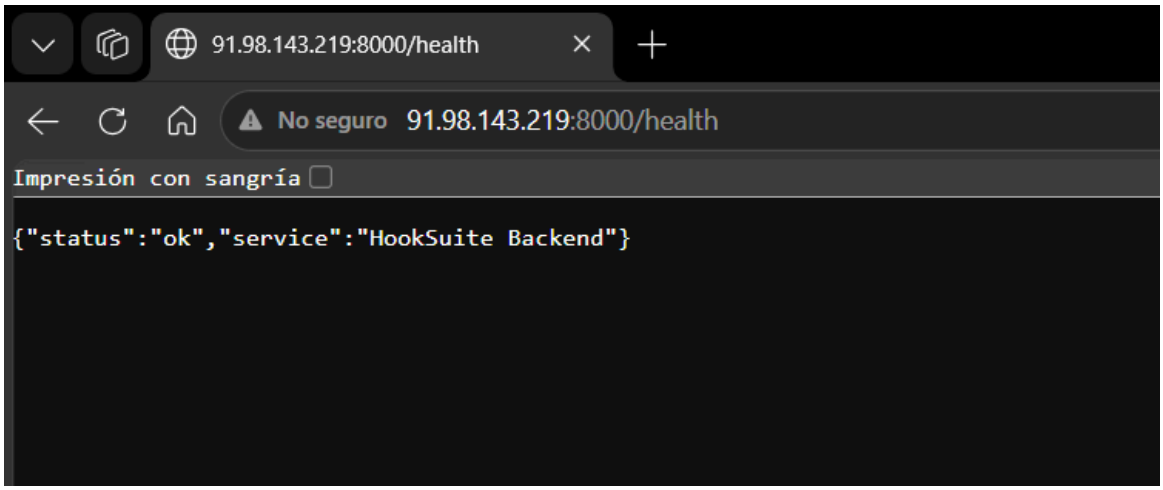


Figura 10: RNF-07 / PR-16 — `GET /health` de la API responde `200` sin credenciales ni cabecera `WWW-Authenticate`: la API no exige autenticación.

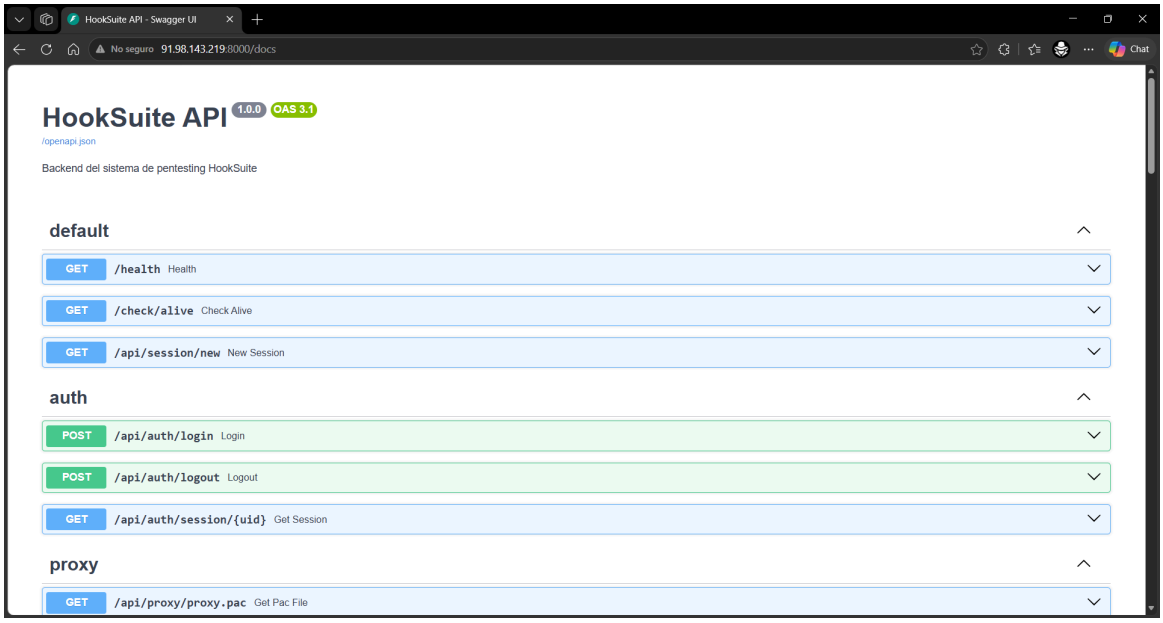


Figura 11: RNF-07 / PR-16 — la documentación Swagger (`/docs`) queda abierta a cualquiera, exponiendo las 40 operaciones de la API, incluidas las que lanzan tráfico contra terceros.

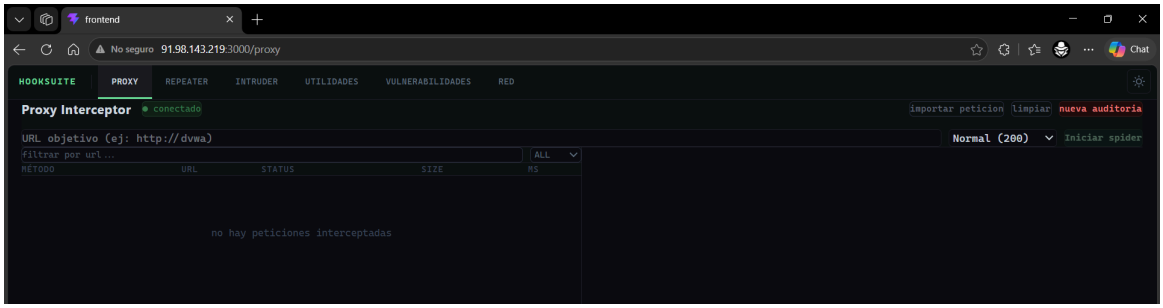


Figura 12: RNF-07 — el frontend servido directamente por `:3000` carga `200` sin Basic Auth, evitando por completo el login de Nginx del `:80`.

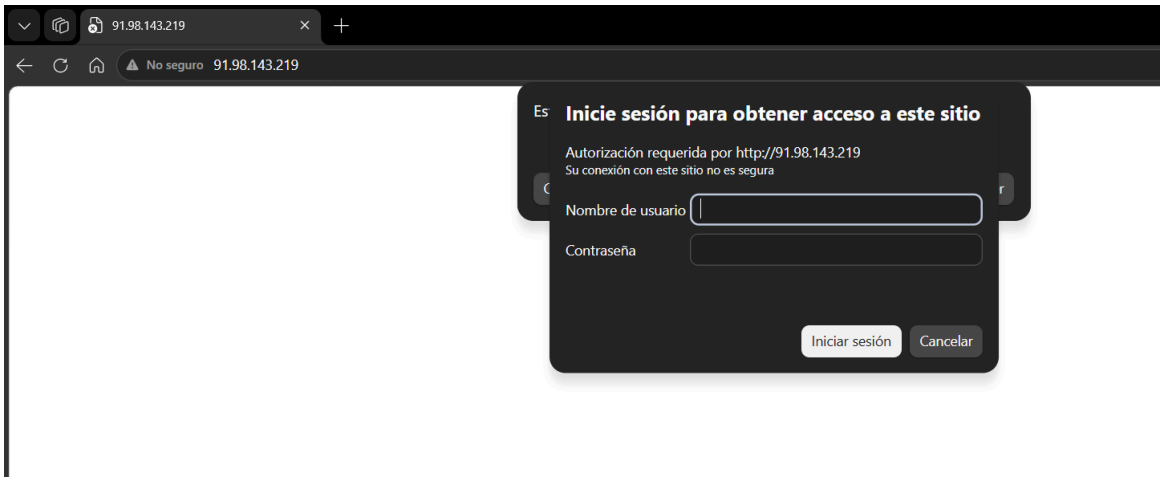


Figura 13: RNF-07 — por contraste, la entrada «correcta» (`:80` tras Nginx) sí pide Basic Auth: el control existe, pero se puede rodear por el puerto directo.

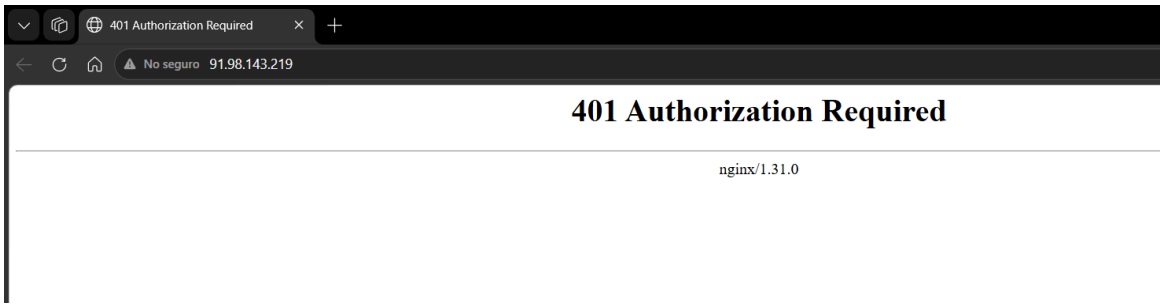


Figura 14: RNF-07 — la credencial documentada en el informe de la P1 es rechazada con `401` por `nginx/1.31.0`: la credencial publicada ya no sirve, lo que refuerza el cambio de modelo de acceso en la Fase 2.

Defectos corregidos: antes y después (RF-03)

Dos pares de capturas tomadas sobre el producto desplegado, antes y después de corregir. El objetivo de las pruebas es una web propia del equipo, autorizada para ello.

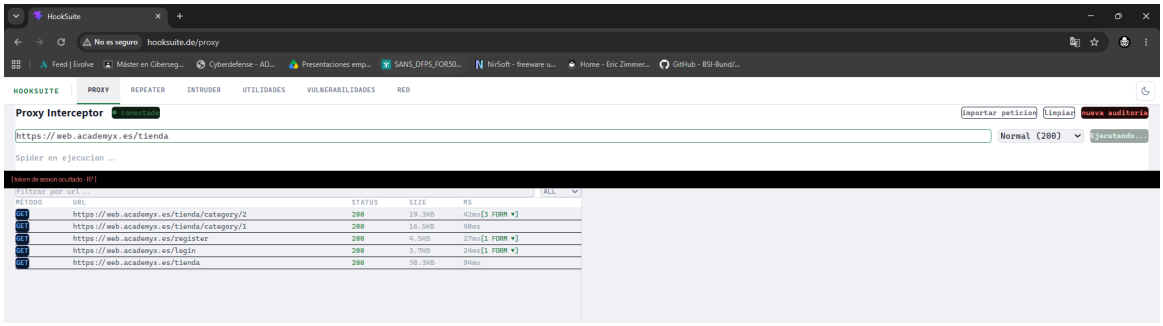


Figura 15: RF-03 — ANTES: el Spider está rastreando («Spider en ejecucion…») y el único botón del panel aparece bloqueado como «Ejecutando…». No existe ninguna forma de detener el rastreo desde la interfaz.

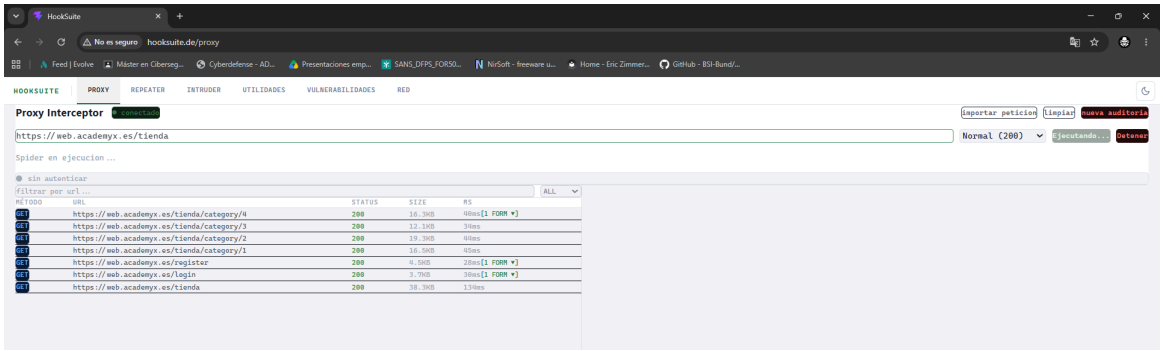


Figura 16: RF-03 — DESPUÉS: en la misma situación aparece el botón «Detener» junto al de ejecución. Obsérvese además el contador «1 FORM» en cada petición, frente a las decenas de la captura siguiente.

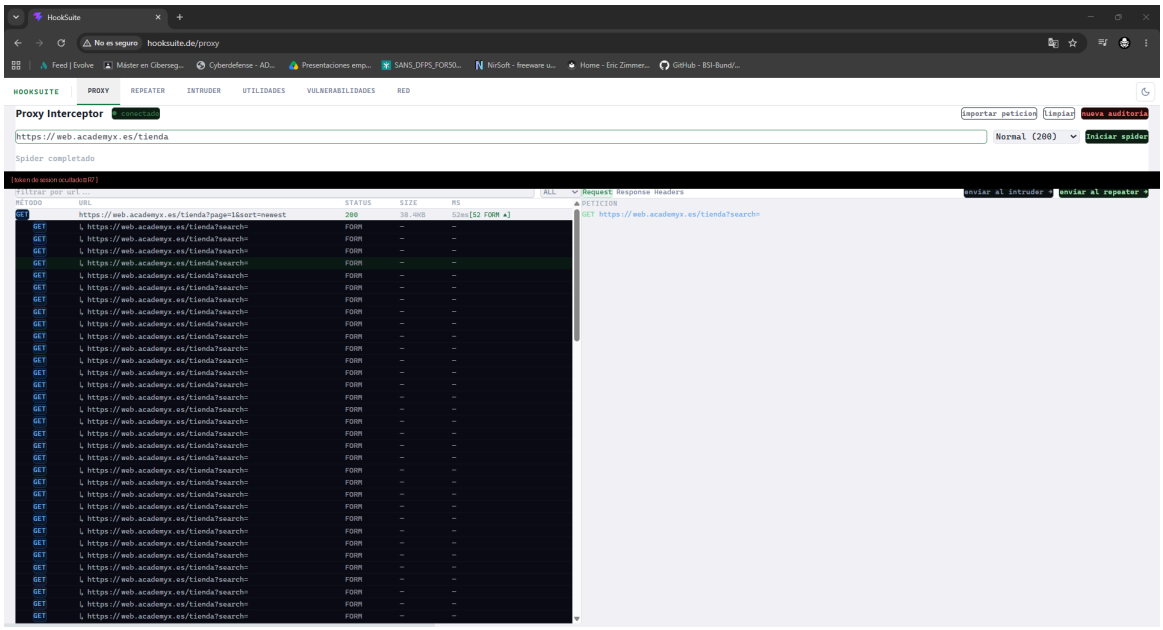


*Figura 17: RF-03 — ANTES: una sola petición (`/tienda?page=1&sort=newest`) arrastra **52 entradas de formulario idénticas**, que inundan el panel de resultados.*

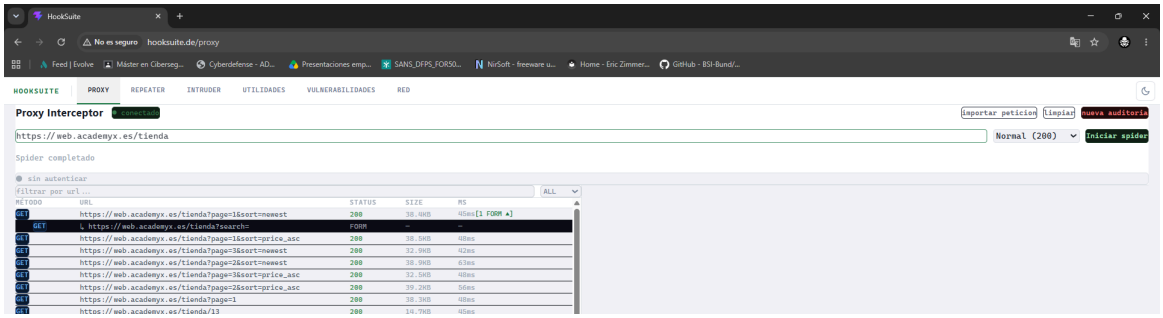


*Figura 18: RF-03 — DESPUÉS: la misma petición de la misma página queda en **una sola entrada**. Solo cambia el contador: 52 → 1.*

> **Nota de tratamiento de las capturas (R7).** Las dos del estado anterior mostraban un identificador de sesión de la web de pruebas en la franja superior; se publican con esa franja tapada, y los originales no salen del almacén interno. Las del estado posterior no lo contienen porque la sesión no estaba autenticada.

El pulido del Spider (11-oct): quince defectos, y catorce los encontró el uso

Cerrada la Fase 2, el 11-oct no se añadió funcionalidad: la jornada se dedicó a **pulir** lo que ya existía, pestaña por pestaña, empezando por la primera. De esa pasada salen quince defectos corregidos en el Spider. Lo que más dice de ellos no es el número, sino **cómo aparecieron**: catorce usando la herramienta y uno leyendo el código. Cada fila está atada a su entrada del diario de desarrollo, que es la fuente.

#	Qué pasaba	Cómo apareció	Diario
1	Pulsar «Detener» no paraba hasta terminar todos los formularios de la página en curso	usando	11:17
2	Tres textos de la pantalla describían algo distinto de lo que el código hacía	usando	11:17
3	El indicador de sesión decía «autenticado» sin que nadie se hubiera autenticado	usando	11:49
4	El mismo indicador fallaba en los dos sentidos: ni se encendía con sesión ni se apagaba sin ella	usando	12:45
5	El campo de URL se bloqueaba al iniciar, y no decía por qué ni cómo liberarlo	usando	12:51
6	Los tres botones de borrado hacían cosas distintas y nada permitía saber cuál usar	usando	13:01
7	La pestaña se llamaba «Proxy» y no es un proxy, sino un rastreador	usando	14:19

8	Los formularios colgaban de páginas ajenas: la pertenencia se adivinaba por prefijo de URL	usando	14:19
9	La columna de estado mezclaba códigos HTTP con la palabra FORM, que no es un código	usando	14:27
10	Al terminar, el rastreo no decía cómo había terminado	usando	17:52
11	El objetivo fijado se perdía al refrescar la página, y permitía lanzar otro encima	usando	17:52
12	El recuento del mensaje final no cuadraba con las filas del panel	usando	18:12
13	Las primeras filas capturadas no se alcanzaban con el scroll	usando	18:15
14	La etiqueta del desplegable de formularios insinuaba más de lo que había	usando	18:15
15	Los filtros escondían los formularios; filtrar por POST vaciaba la tabla entera	leyendo el código	19:22

El patrón que se repite, y es el hallazgo de la jornada. En once de los quince, el backend **ya sabía** la respuesta y no llegaba a la pantalla: de qué página salía cada formulario, cómo había terminado el rastreo, cuál era el objetivo, cuántas páginas habían fallado. No eran fallos de cálculo sino de **traslado**: funciones correctas que la interfaz no dejaba ver. Un producto puede estar entero por dentro y resultar inservible por fuera, y las pruebas que miran el backend no detectan nada de esto.

El decimoquinto se encontró de otra manera, y merece su propio comentario. Apareció al escribir el manual de uso: para documentar el filtro hubo que leer cómo funcionaba, y leyéndolo se vio que filtraba las filas **antes** de agruparlas. Como cada formulario cuelga de su página, una página que no pasaba el filtro se llevaba por delante a sus formularios. Filtrar por POST —que es justo lo que se hace para ver formularios— vaciaba la tabla.

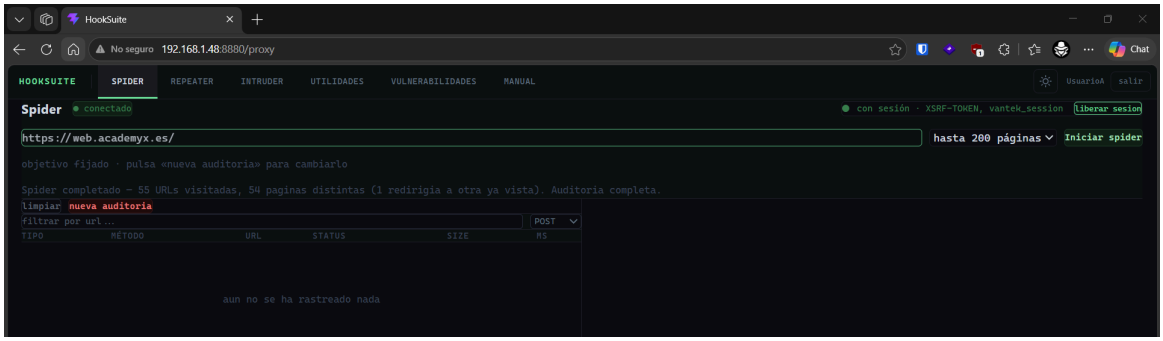


Figura 19: RF-03 — ANTES: con el panel lleno de un rastreo y el filtro de método en POST, la tabla se queda vacía. Los formularios son POST, pero sus páginas son GET: al caer la página, el formulario se quedaba sin grupo donde colgar y desaparecía.

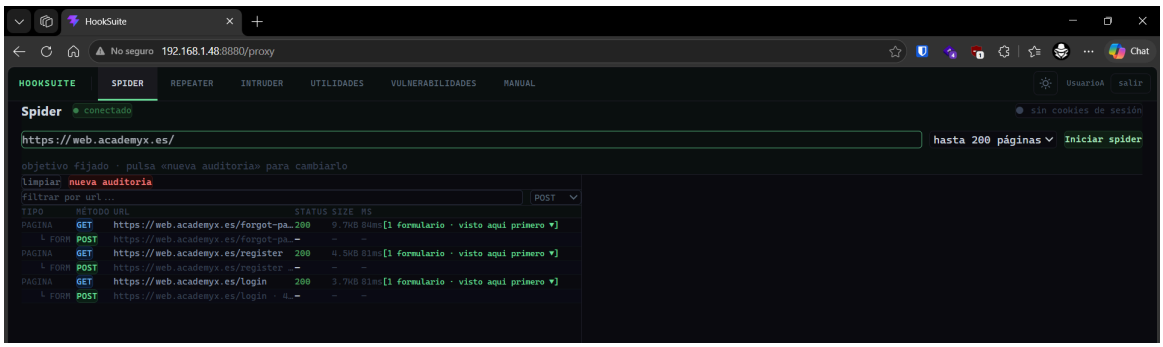


Figura 20: RF-03 — DESPUÉS: el mismo filtro muestra tres páginas con su formulario POST ya desplegado debajo. Una página entra ahora si coincide ella o si coincide alguno de sus formularios, y si entra solo como contenedor aparece abierta.

Dos de los quince los introdujimos nosotros al arreglar otros. Están contados en la tabla y se detallan en el apartado 12: un arreglo que hizo **desaparecer** formularios que antes solo sobraban, y un cambio de maquetación que recortó las últimas filas. Se señalan aquí porque el recuento honesto incluye los defectos propios, no solo los heredados.

El pulido del Repeater (12-oct): dieciséis defectos, y una función retirada el mismo día

El 12-oct repitió el método del día anterior sobre la segunda pestaña: no se añadió alcance, se pulió lo que había. El resultado se parece al del Spider en el número y se diferencia en algo que conviene contar entero, porque es lo más instructivo de la jornada: **una función se añadió por la mañana y se retiró por la tarde**.

Nº	Qué pasaba	Cómo apareció	Diario
1	Un fallo de red se mostraba como una respuesta más, con el código HTTP `0` y medidas de «0 ms» y «0,0 KB»	leyendo el código	10:50
2	El panel no decía a qué dirección se había enviado: con varias peticiones seguidas no se sabía de cuál era la respuesta	usando	10:50
3	El cuerpo de la petición de envío mandaba el identificador de espacio de datos, residuo del modelo anterior al login	leyendo el código	10:50
4	`Limpiar` no era definitivo: al volver a la pestaña reaparecían la petición y sus cabeceras heredadas	usando	12:37
5	El Spider guardaba y enseñaba como enviadas unas cabeceras **fabricadas aparte**: otro `User-Agent`, un `Host` que nadie puso y ninguna `Cookie`	tirando de un hilo	12:50
6	El arreglo anterior seguía sin mostrar la cookie de sesión, porque la pone el cliente de red por debajo y no pasaba por ese diccionario	captura de producto	13:35
7	Los cinco rótulos del panel de detalle no se distinguían del contenido: se leían las cabeceras de respuesta como si fueran de petición	usando	13:30
8	Las respuestas se cruzaban: se veía una dirección con el cuerpo de otra petición, y no era reproducible a voluntad	usando	16:49
9	No había forma de probar la misma petición **sin** sesión: el único botón vaciaba la petición entera	usando	16:49
10	El Repeater seguía las redirecciones siempre, así que **nunca** se veía el `302` ni su `Location`	comparando con la herramienta de referencia	18:03
11	Dos envíos idénticos daban resultados distintos: el cliente compartido ponía cookies que no estaban en la petición	usando	18:37
12	Los formularios descubiertos por el Spider viajaban sin sesión, y todos los POST devolvían `419`	usando	18:45
13	La petición que el Spider manda al Repeater nace vieja: es una plantilla con el token del rastreo, no una petición capturada	usando	20:24
14	`Limpiar` no borraba la página preparada, que se quedaba a la vista	usando	20:43
15	Dos avisos que ya no podían aparecer nunca: quedaba el código que los lee, no lo que los provoca	escribiendo el manual	21:03 y 21:24
16	El manual del Spider afirmaba de las cabeceras lo que el código no cumplía, y no documentaba un botón nuevo	escribiendo el manual	21:13

La función que no llegó a la noche, y por qué se cuenta en vez de omitirse. A las 10:08 el Repeater ganó un **historial** de peticiones: el contador de la cabecera, que hasta entonces no se podía abrir, pasó a ser el botón que despliega la lista, con borrado por fila y supervivencia al refresco. A las 17:28 se retiró entero. El motivo es que **todo el comportamiento errático de la jornada giraba a su alrededor** —respuestas cruzadas, entradas con la petición y la respuesta mal emparejadas, el panel mostrando la respuesta de otra— y tres arreglos sucesivos no lo dejaron estable, con la congelación al día siguiente. Un Repeater que mezcla respuestas es peor para un informe que un Repeater sin historial. Lo que ocupó su lugar es la decisión de diseño del apartado 10: **una petición cada vez**, con el estado «ocupado» visible desde la otra pantalla.

Y la causa de fondo, que apareció cuatro horas después de retirarlo. El `302` invisible explicaba la confusión entera: al enviar un formulario que rebota, el producto mostraba el resultado del GET posterior sin decir que había habido un salto, de modo que la respuesta parecía ser la del POST. Si el `302` se hubiera visto desde el principio, el historial probablemente no habría necesitado retirarse. Se deja escrito porque ese es el orden real de los hechos: **primero se retiró el síntoma y después se encontró la causa**.

El patrón de los dieciséis, comparado con el del Spider. En el Spider, once de quince eran fallos de **traslado**: el servidor sabía la respuesta y la pantalla no la enseñaba. Aquí el patrón es otro y más incómodo: **siete de los dieciséis son fallos de fidelidad** —el producto mostraba algo distinto de lo que hacía—. Cabeceras que no eran las enviadas, un `302` que no se veía, dos envíos iguales con distinto resultado, un fallo de red disfrazado de respuesta, respuestas cruzadas y avisos que ya no podían salir. En una herramienta de auditoría ese es el defecto más caro, porque **lo que la herramienta muestra es la evidencia que acaba en el informe**: un auditor que copie esas cabeceras documenta algo que no ocurrió.

El proyecto tenía un verificador configurado y sin usar. La regresión más tonta de la jornada —el botón de enviar dejó de hacer nada porque un borrado por rango de líneas se llevó la declaración de una variable— la habría cazado el analizador estático del propio repositorio, que **nunca se había ejecutado**. Se probó a mordida (un fichero con una variable inexistente devuelve el error esperado) y desde entonces va **por delante** de cada verificación: hoy no queda ninguna variable no declarada en el frontend, con control positivo para que ese cero signifique algo. Lo que destapa de paso va al apartado 10: el frontend acumula veintitrés errores y dos avisos de otras reglas. **El proyecto no pasa su propio analizador**, y conviene decirlo.

Lo que ninguna de las verificaciones del asistente cazaba, dicho con nombre. Cada cambio de la jornada se verificó contando cadenas de texto dentro del paquete construido. Ese método confirma que un texto **está**; no dice que se **lea** bien ni que la lógica funcione. Tres veces el mismo día dio por bueno algo roto: escapes que se mostraban en crudo, una dirección presente pero ilegible por falta de sitio, y la regresión del botón de enviar. La construcción de la imagen no comprueba variables no declaradas, el analizador no detecta errores de ejecución, y contar textos no detecta ninguno de los dos. **Lo único que cazó esos tres fue abrir la pantalla**, y de ahí que las capturas de producto las tome una persona (R6). El detalle de los ocho defectos que introdujo el asistente está en el apartado 12.

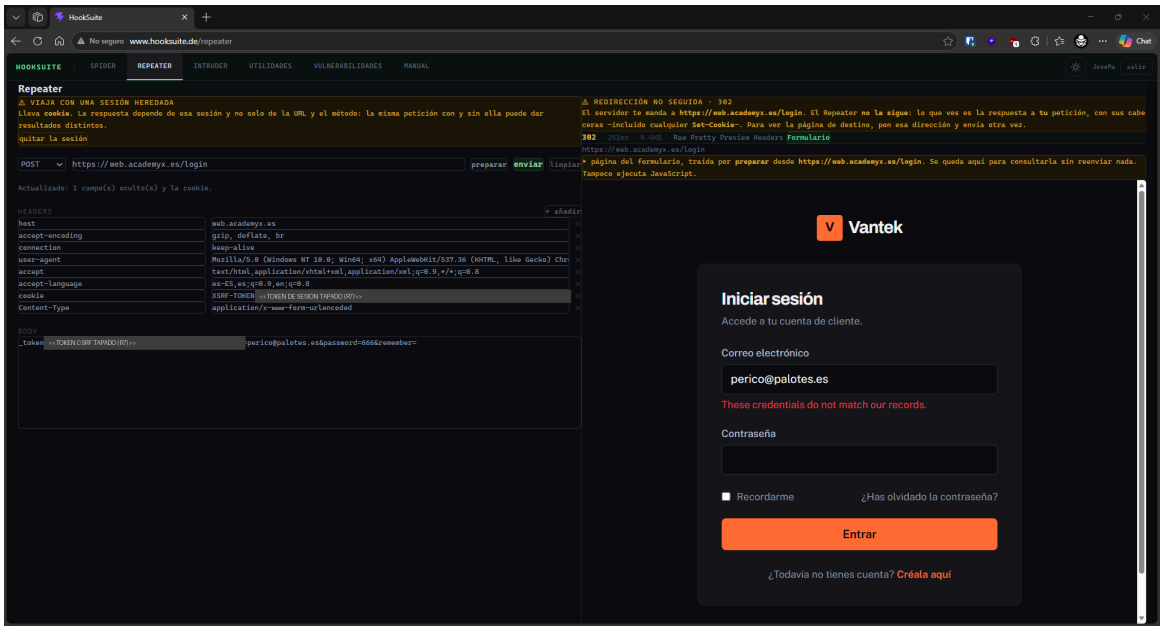


Figura 21: RF-05 / RF-03 — el final de los `419`. La misma clase de petición que antes era rechazada llega ahora a la lógica de la aplicación: el servidor responde `302` y el formulario contesta que las credenciales no coinciden. Es decir, lo que fallaba no era el Repeater sino unas cabeceras incompletas en los formularios que descubre el Spider. Valores de sesión tapados (R7).

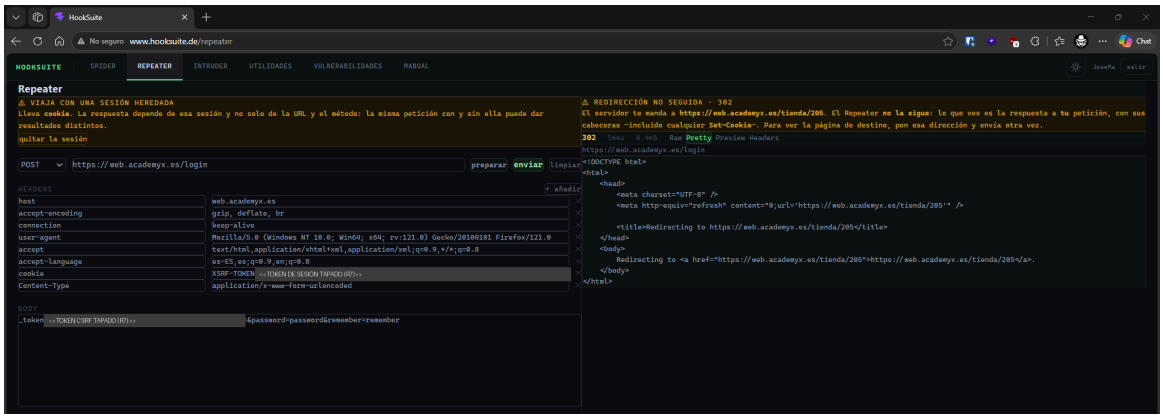


*Figura 22: RF-04 — prueba de aceptación del Repeater con cliente propio, **primer envío**: `302`, 0,4 KB y destino `/tienda/205`.*

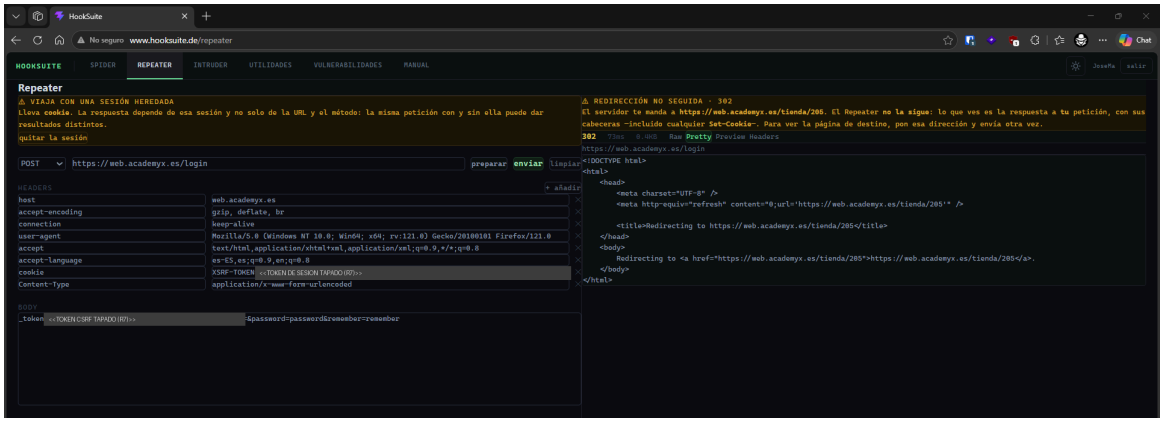


*Figura 23: RF-04 — **segundo envío de la misma petición, sin tocar nada**: idénticos el estado, el tamaño, el destino y el cuerpo de la respuesta. Lo único que cambia es el tiempo (56 ms → 73 ms), y ese cambio es justamente lo que demuestra que son dos envíos distintos y no la misma figura repetida. Con el cliente compartido anterior, dos envíos iguales podían no coincidir.*

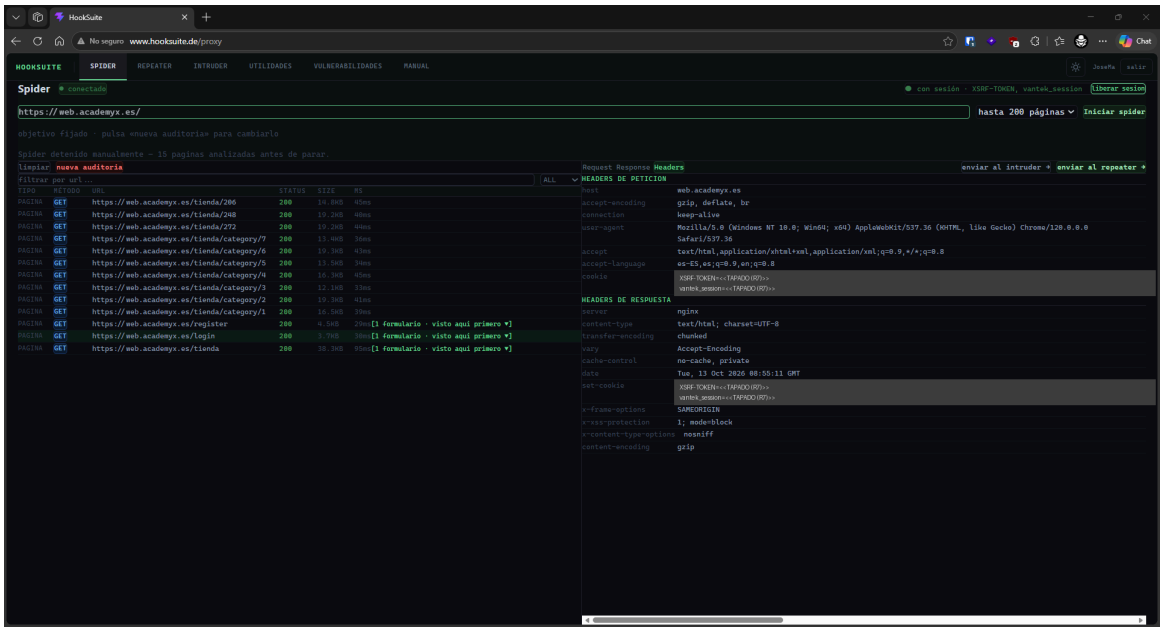


*Figura 24: RF-03 — el panel de detalle después del pulido, con los dos defectos de fidelidad corregidos: los rótulos de sección se distinguen del contenido —antes compartían el mismo gris, y eso llevó a leer las cabeceras de respuesta como si fueran de petición— y las cabeceras que se muestran son **las realmente enviadas**, con la `Cookie` de sesión incluida. Los cuatro valores de sesión van tapados (R7).*

> **Nota de tratamiento de estas figuras (R7).** Todas se tomaron en producción, sobre el código congelado, contra una web propia del equipo autorizada para ello. Desde el 12-oct el panel de cabeceras

muestra la cookie de sesión del objetivo —es lo correcto, porque es lo que se envía—, así que cada figura se publica con esos valores tapados y los originales no salen del almacén interno.

Condición de carrera detectada antes de reiniciar el servidor

El agente del cortafuegos y el servicio de contenedores no tenían orden de arranque entre sí. Si el segundo ganaba la carrera, **creaba una carpeta donde debía ir el canal de comunicación del agente** —comportamiento normal al montar un fichero que no existe— y el agente quedaba en un bucle de reinicios. El servidor llevaba sin reiniciarse desde mayo, así que nunca se había puesto a prueba. Se corrigió fijando el orden de arranque antes de reiniciar; tras el reinicio, el contador de reinicios del agente marcaba **cero**: la carrera no llegó a producirse.

Otras evidencias ya en mano

- **Incidente de las claves y limpieza del historial:** ensayo de reescritura verificado antes/después (3 → 0 claves), detallado en el apartado 7.
- **Salidas de terminal** (health sin login, `openapi.json` sin seguridad, cierre de puertos, mensajes de error) guardadas como texto en el repo, no como foto (R7: así ninguna captura publica por error un secreto).

Evidencia de las fugas de aislamiento (RF-12), capturada el 6-oct

Cuatro pruebas lanzadas contra la cocina a través del proxy, como las haría un cliente, **antes de tocar una sola línea de código de la Fase 2**: el propio arreglo borra esta evidencia, y el apartado pide las pruebas que fallaron con su explicación. Los valores de cookie empleados son inventados a propósito, de modo que la captura no contiene ningún identificador real (R7).

Las cuatro confirmaron el fallo: la API responde `200` sin credencial mientras `/health` responde `401`; un token inventado obtiene sesión propia; un segundo usuario lee la cookie de sesión capturada por el primero —y también se obtiene sin token—; y una vulnerabilidad publicada sin token aparece en la sesión de dos auditores distintos.

La captura íntegra, con la causa en fichero y línea de cada una, está en `evidencias/fugas-aislamiento-06oct.md`. Una corrección respecto a lo que teníamos anotado: la difusión no ocurre solo en las dos llamadas que emiten a todas las sesiones, sino también en los dos endpoints que admiten peticiones sin token, que además **persisten** el dato en la sesión de todos los usuarios. Son cuatro puntos de difusión, no dos.

El aislamiento, verificado después de cerrarlo (RF-12)

Con el sistema de acceso puesto, se repitieron las mismas pruebas contra la cocina. **Once comprobaciones, once en verde**: alta de dos usuarios distintos; entrada de ambos; contraseña incorrecta rechazada con `401` y no con un error del servidor; la identidad del portador del token resuelta correctamente; la API respondiendo con token; `403` al intentar tocar el espacio de otro con token propio —es decir, autenticación **y** autorización, que son cosas distintas—; el segundo usuario sin ver el hallazgo del primero; y el segundo usuario sin ver la cookie de sesión del primero, que era la tercera fuga.

Se probó además **como ataque**, no solo como comprobación: un usuario autenticado intentó dirigir el Spider al espacio de otro nombrándolo en el cuerpo de la petición, que era el quinto fallo del apartado 7. El espacio de la víctima quedó vacío y el rastreo fue a parar al del atacante.

Ni el código de invitación ni las contraseñas aparecen en ninguna salida: el guion de pruebas los mantiene en variables (R7).

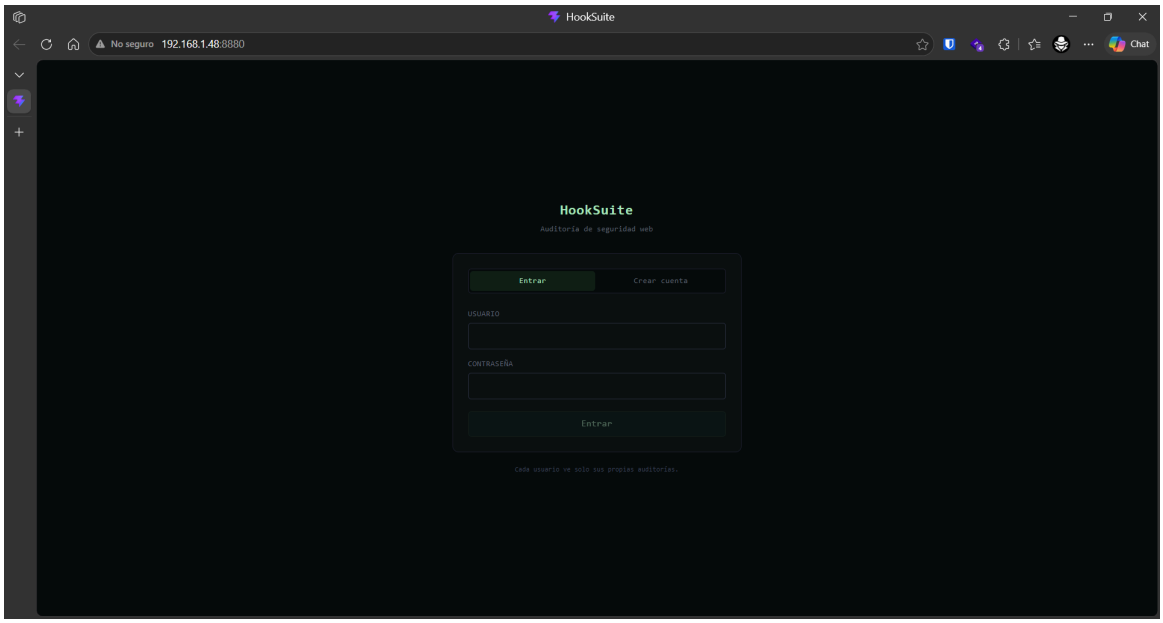


Figura 25: RF-12 — la entrada del producto tras retirar el Basic Auth de Nginx: un panel de acceso propio, con pestañas «Entrar» y «Crear cuenta». Antes, la raíz devolvía `401` pidiendo una credencial que ningún integrante del grupo conocía.

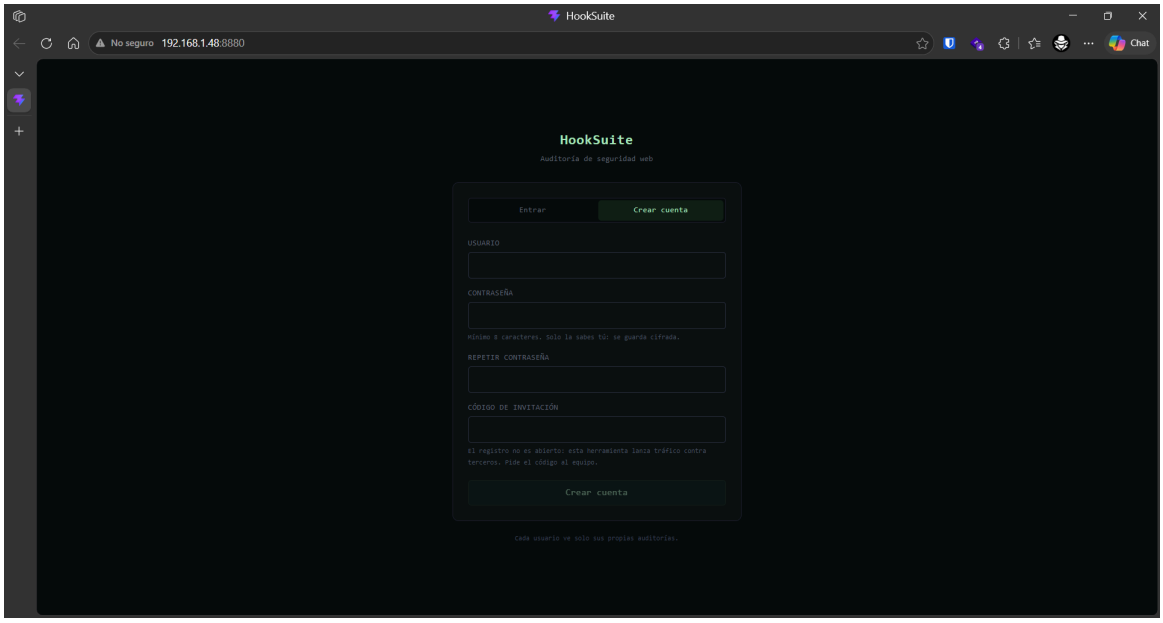


*Figura 26: RF-12 — el alta exige **código de invitación**. El registro no es abierto a propósito: la herramienta lanza tráfico contra terceros, así que con altas anónimas cualquiera podría atacar desde la infraestructura del grupo. La contraseña la elige el usuario y solo la conoce él.*

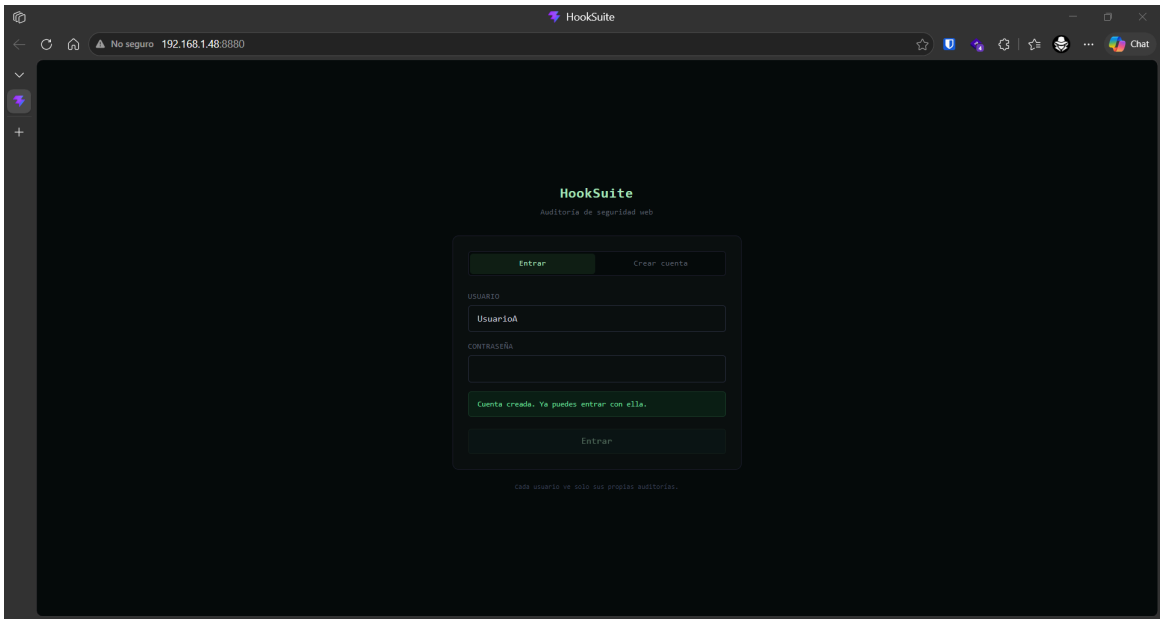


Figura 27: RF-12 — alta completada: la interfaz confirma la creación de la cuenta y devuelve al formulario de entrada. Cubre la mitad «registro» del requisito, que el enunciado pide junto al login.

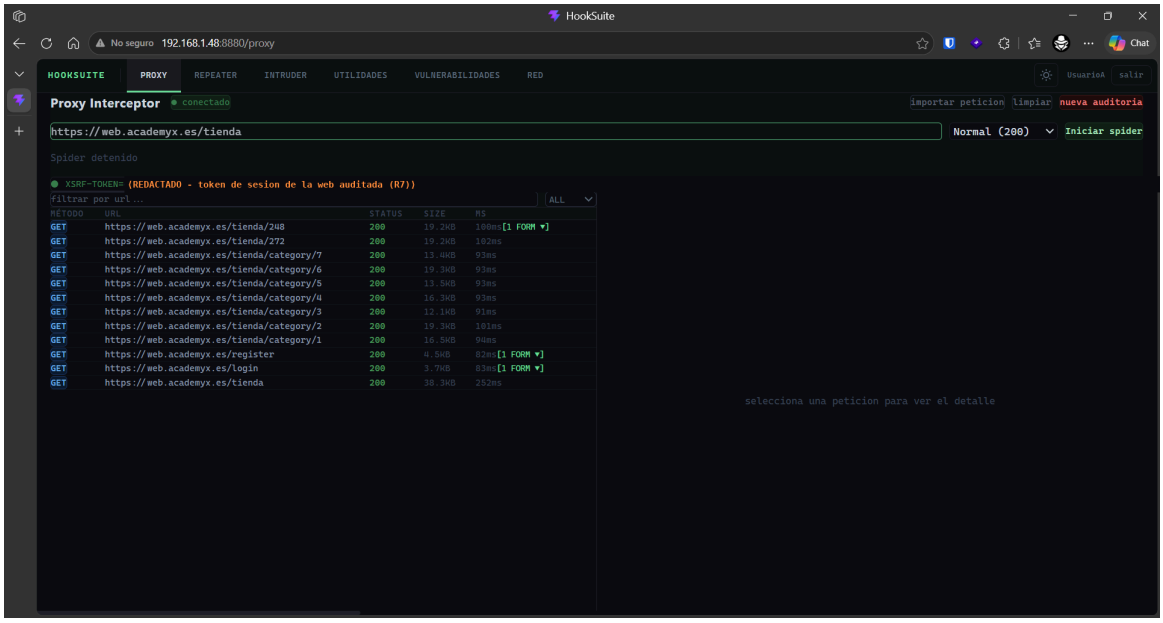


Figura 28: RF-12 — ANTES de la Fase 2 este panel era común a todos. Ahora el usuario A, autenticado, ve las trece peticiones que su propio Spider capturó del objetivo autorizado. Compárese con la captura siguiente, tomada al mismo tiempo.

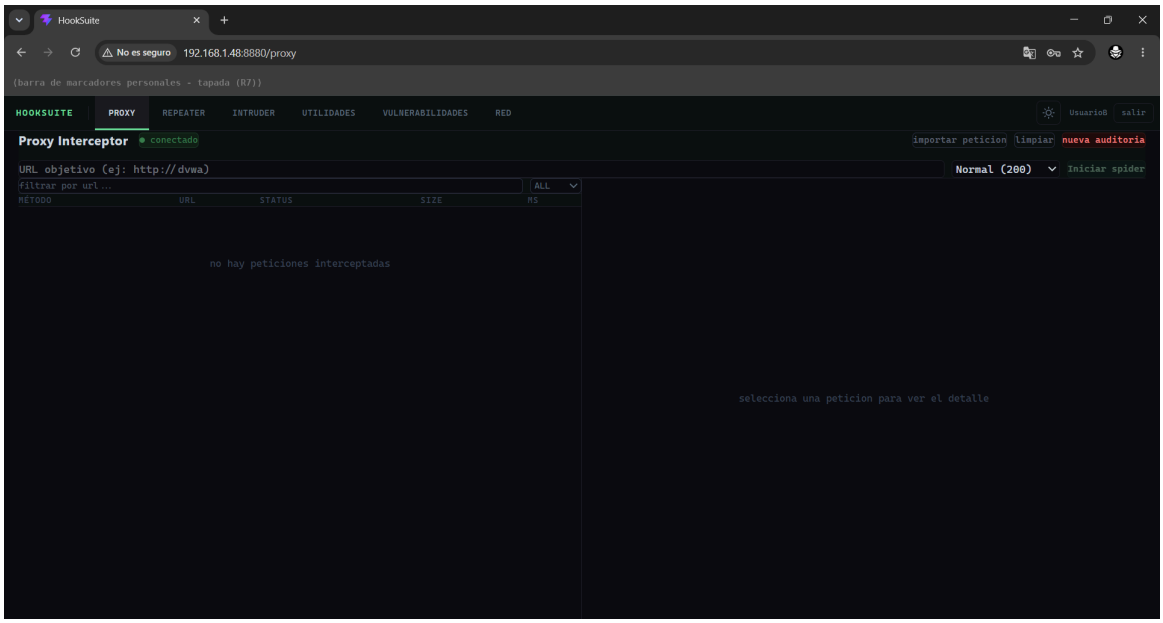


*Figura 29: RF-12 — DESPUÉS, el contraste: el usuario B, autenticado a la vez y en otro navegador, ve su panel **vacío**. Antes de la Fase 2 habría visto exactamente lo mismo que el usuario A. Este par es la prueba del aislamiento por usuario.*

> **Nota de tratamiento de las capturas (R7).** La del usuario A mostraba el valor íntegro de la cabecera `XSRF-TOKEN`, que es el identificador de sesión capturado **de la web auditada**: justo el dato que este requisito existe para no compartir. Se publica con ese valor tapado y la etiqueta a la vista, para que se vea qué se redactó; el original no sale del almacén interno. La del usuario B lleva tapada la barra de marcadores del navegador, ajena al proyecto.

El despliegue habría terminado «en verde» sin aplicar la configuración del proxy

La comprobación previa al despliegue de la Fase 2 —hecha **antes** de tocar la instalación pública— encontró un defecto no en el producto, sino **en el propio plan de despliegue**, escrito dos días antes. Se cuenta porque el enunciado pide las pruebas que fallaron, y porque el modo de fallo es de los que no avisan.

El plan mandaba reconstruir «solo el frontend, el backend y el proxy». Pero el proxy **no se construye**: usa una imagen descargada, y su bloque del fichero de composición **no cambiaba ni un byte** entre la rama publicada y la que se iba a desplegar. Compose lo veía idéntico a lo que ya estaba corriendo y, por tanto, **no lo habría recreado**. Y como el proxy lee su configuración una sola vez al arrancar —y el contenedor en producción llevaba vivo desde dos días antes—, el despliegue habría acabado **sin errores y sin aplicar nada**: el panel de acceso nuevo habría seguido detrás de la contraseña compartida que la Fase 2 venía a jubilar, y las dos rutas retiradas habrían seguido publicadas.

Lo que convierte esto en una lección y no en una anécdota: era el mismo fallo que ya había ocurrido dos días antes, con el mismo servicio y la misma causa. Y la razón de que se repitiera es instructiva: **nombrar un servicio en un comando da la impresión de cubrirlo**. El plan lo nombraba, así que nadie volvió a mirar si el comando lo tocaba de verdad.

Y el 10 de octubre apareció el caso espejo, que completa la lección. Al revés que entonces, se recrearon el frontend, la API y el modulo de IA **y no el proxy** —porque su configuracion no habia cambiado—. Resultado: **502 en todo**. Los contenedores recreados reciben **direcciones nuevas**, y el proxy resuelve los nombres de sus destinos **una sola vez, al arrancar**: seguia llamando a las direcciones de los contenedores que acababan de desaparecer. Su registro lo decia con todas las letras, nombrando direcciones que ya no existian.

Las dos mitades tienen la misma raiz: **el estado del proxy se fija al arrancar**, tanto la configuracion que lee como las direcciones que resuelve. De ahi sale una regla operativa que no estaba escrita en ningun sitio y ahora si: **cuando se recrea cualquiera de los servicios a los que el proxy apunta, se recrea tambien el proxy.** No es una precaucion: es que no hacerlo lo deja hablando con direcciones muertas.

Arreglo: forzar explícitamente la recreación de ese servicio. Cuesta segundos y no construye nada. **Se descartó** recargar la configuración en caliente —que también habría funcionado, pero depende de un detalle del sistema de ficheros en lugar de ser incondicional— y también tocar su bloque del fichero de composición solo para que Compose lo viera cambiado, que es resolver el síntoma ensuciando la fuente.

Un modulo que parecia sano y no podia hablar con nadie (RF-08)

La prueba mas barata de esta practica fue una peticion con `curl`, y descubrio un defecto que llevaba cuatro dias en produccion sin que nadie lo notara: **el modulo de IA no podia comunicarse con el backend desde la Fase 2**.

Ese modulo preguntaba al backend cada cinco segundos si habia trabajo. Al poner el control de acceso, esa ruta quedo protegida junto con las demas —correctamente—, y el modulo no presenta credencial ninguna, de modo que recibia un rechazo. Hay ademas un segundo muro detras del primero: incluso con una credencial valida, el control comprueba que el espacio de datos nombrado en la ruta sea el tuyo, y el modulo usaba un identificador fijo que no es de ningun usuario.

Lo grave no es el defecto, es por que no se vio. El codigo comprobaba si la respuesta era un exito; un rechazo por credencial **no provoca ningun error**, simplemente no es un exito, asi que el bucle volvia a dormir cinco segundos y lo intentaba otra vez. Y la comprobacion de arranque —que esperaba al backend hasta treinta veces— **descartaba su propio resultado** y continuaba igual, escribiendo en el registro que estaba esperando instrucciones. Un operador que mirara ese registro habria visto un modulo en marcha y a la espera. Estaba en marcha, y no podia recibir nada.

Es el mismo patron que RNF-06 vino a cerrar dentro del clasificador, un nivel mas arriba: **un fallo que no se distingue de la normalidad**. Que apareciera dos veces en el mismo modulo, por caminos distintos, sugiere que no es un descuido aislado sino una forma de escribir —tratar el camino de error como un caso que «ya se vera»— y es la razon por la que ahora el modulo falla ruidosamente: comprueba el bus al arrancar y se detiene si no esta.

Como se midio, y por que importa el metodo. Tres peticiones sin credencial contra la instalacion de pruebas: la consulta de trabajo pendiente, la comprobacion de arranque y el envio de una orden. Las tres rechazadas. La evidencia se guardo **como texto** y no como captura, de modo que es comparable con la de despues. Y el alcance del problema —que el camino de vuelta de los resultados estaba igual de roto— se establecio **sin abrir el fichero del orquestador**, que esta en la lista de material que este equipo no vuelca a la conversacion con la herramienta de IA (apartado 12): basto contar sus referencias al backend y mirar su esqueleto de funciones.

Evidencia completa en `docs/evidencias/canal-ia-backend-muerto-10oct.md`.

Las dos contrapartidas del bus, probadas antes de confiar en el

Reconectar el modulo por un bus interno (apartado 5) cambia la forma del riesgo: un control de acceso falla cerrando, un bus falla **abriendo**. Asi que lo que se prueba no es solo que el reparto funcione, sino que **no se puede repartir mal**. Dieciocho casos en dos conjuntos, con dobles para el bus, el gestor de sesiones y el orquestador: **ni red, ni credencial de modelo, ni contenedores, cero coste**.

Los dos que justifican el esfuerzo:

- **Un hallazgo sin dueño se descarta**, no se guarda ni se emite. Sin esa regla, un mensaje mal formado reabriria exactamente la fuga que cerro la Fase 2 —un hallazgo publicado sin credencial apareciendo en el panel de dos auditores distintos—.
- **Dos auditores no ven lo del otro**, comprobado repartiendo a dos espacios y verificando que cada uno recibe solo lo suyo.

Y tres mas que cubren el fallo silencioso en el otro extremo: si la auditoria se interrumpe, **se publica el motivo** en lugar de no publicar nada; si nadie escucha el bus, publicar **lo dice** en vez de dar el trabajo por encolado; y si una orden llega sin dueño, **no se audita**, porque no habria a quien devolverle el resultado.

Reproducible con `python3 backend/tests/test_bus_reparto.py` y `python3 ia/tests/test_bus_publicacion.py`.

El recorrido completo del disparador, verificado de punta a punta (RF-08)

Durante cuatro días el disparador estuvo **construido y sin poder demostrarse**: el botón existía, el bus estaba cableado, y aun así una pulsación desde el panel no movía ni una línea del módulo de IA. El apartado llegó a listar tres hipótesis sin poder decidir entre ellas.

El 10-oct se cerró, y la forma de cerrarlo es parte del resultado: en lugar de seguir estimando sobre ventanas de tiempo aproximadas, **se anotó la hora exacta de la pulsación** y se midió contra ella. A partir de las 13:58:10 el servidor registra **una** petición al disparador, con respuesta correcta, y el registro del módulo de IA pasa de 192 a 376 líneas. Panel → autenticación → bus → módulo suscrito: el recorrido entero, por primera vez.

Lo que no se da por explicado. Cincuenta y ocho minutos antes, la misma pulsación no había movido nada, y entre una y otra **no se tocó el producto** —ningún fichero ni contenedor cambió—. La explicación cómoda sería una sesión caducada, pero no encaja: el sondeo de instrucciones sigue devolviendo **no autorizado** ahora, con sesión válida y el disparador funcionando. Queda abierto y declarado, porque un apartado de pruebas que explica lo que no sabe explicar vale menos que uno que lo admite.

El techo de gasto, ejecutado: una auditoría entera por 0 € (RF-08)

Este caso estuvo declarado durante semanas como **preparado y no ejecutado**, y se deja constancia de que ya no lo está. Una auditoría del módulo de IA no tiene límite de consultas al modelo: el recorrido anida páginas × tipos de prueba × cargas, y cada carga es una consulta. Con el disparador funcionando, cada pulsación arranca ese recorrido completo.

El caso estaba diseñado para costar **cero**: fijando el techo en 0 por variable de entorno, ninguna consulta debe salir y la auditoría debe quedar marcada como truncada. **Ejecutado el 10-oct desde el panel, con sesión real.** La auditoría recorrió cuarenta y ocho elementos, no salió **ni una sola** consulta al modelo —coste 0 €— y los cuarenta y ocho quedaron marcados como no analizados, cada uno con el motivo del techo. El techo no se limita a cortar: **el corte se ve**, que era la condición que se le puso al diseñarlo.

La mitad que falta del caso, y se declara: el techo en 1, donde debe pasar una sola consulta y el resto quedar como «no analizado». Esa sí necesita clave de API válida, porque la consulta que pasa tiene que salir de verdad.

> ⚠️ **HUECO** — responsable: **José María + Claude**. Ejecutar la segunda mitad del caso del techo —techo en 1— en cuanto haya clave de API válida: debe pasar una sola consulta y el resto quedar como «no analizado» con el motivo del techo. La primera mitad (techo 0) ya está ejecutada y capturada. Coste previsto: una consulta.

AISLAMIENTO POR USUARIO VERIFICADO EN LA COCINA (11/11) · PENDIENTE DE VERIFICAR EN PRODUCCIÓN

EVIDENCIA DE EXPOSICIÓN Y DE LA FASE 1 COMPLETA

> ⚠️ **HUECO** — responsable: **José María + Claude**. Plan de pruebas formal y capturas de PRODUCTO de los bugs (500 de `check/alive`, trampa del Spider) reproducidos en la cocina. Las de pantalla las saca José María (R6), tras la Fase 3, cuando el login nuevo cambie las pantallas. Las pruebas las hacen José María y Claude (R2).

Operación degradada (RNF-06), probada sin gastar saldo

La prueba que demuestra RNF-06 no necesita ni clave de API ni conexión: sustituye el cliente del modelo por uno simulado y comprueba qué devuelve el clasificador en cada situación. Ocho casos: los tres estados sobre la vía del proxy, el estado degradado en las cuatro vías, el camino correcto de la identificación de servidor, y que se le entrega un esquema de respuesta al cliente.

El valor de la prueba está en haberla visto fallar. Ejecutada contra el código anterior devuelve *0 de 8*, y el motivo es revelador: los cuatro métodos llamaban a `.get()` sobre el objeto que el cliente reescrito devuelve ahora, de modo que la fase de análisis no es que fallara en silencio, es que se interrumpía entera. Contra el código corregido devuelve *8 de 8*. Una prueba que pasa siempre no prueba nada; esta distingue las dos versiones.

Por qué una prueba y no una revisión visual. El fichero del clasificador está en la lista de material que este equipo no vuelca a la conversación con la herramienta de IA (apartado 12), así que se editó sin leerlo, mediante una receta de pasos verificables. En ese modo de trabajo la prueba no es un complemento de la revisión: es lo que la sustituye, y es más estricta, porque comprueba el comportamiento en vez de la impresión que deja el código al leerlo.

Reproducible con `python3 ia/tests/test_tres_estados.py`. Evidencia completa, con las dos salidas, en `docs/evidencias/rnf06-tres-estados-antes-y-despues-09oct.md`.

RNF-06 visto en el panel, que es donde tenía que verse

Una prueba unitaria demuestra que el clasificador **devuelve** el estado degradado; no demuestra que alguien lo **enseñe**. Entre las dos cosas hubo un hueco real: durante un día el estado existió en el módulo de IA y no lo consumía nadie al otro lado, de modo que RNF-06 estaba cumplido y era invisible. El 10-oct se cerró el hueco y se comprobó en la interfaz, con una auditoría lanzada desde el panel y sesión real.

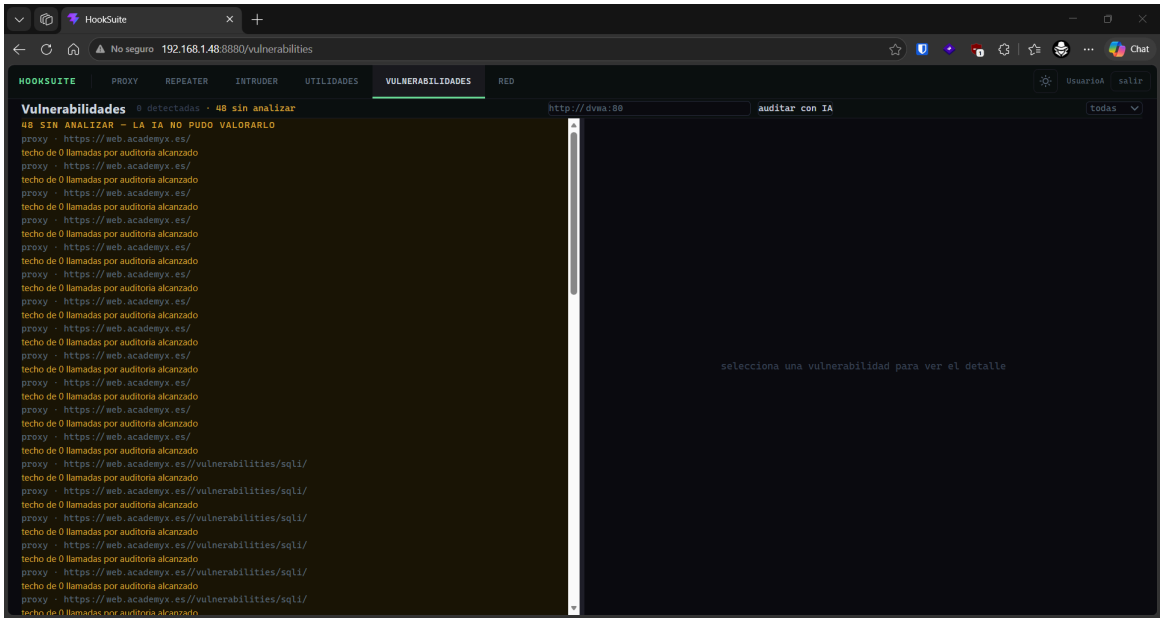


*Figura 30: RNF-06 / RF-08 — el panel tras una auditoría truncada por el techo de gasto. El encabezado anuncia **48 sin analizar — la IA no pudo valorarlo** y cada entrada lleva su motivo, `techo de 0 llamadas por auditoria alcanzado`: el recorte se ve en lugar de pasar por «sin hallazgos». El recuento `0 detectadas · 48 sin analizar` distingue las dos cosas. Los cuarenta y ocho elementos provienen de un rastreo previo sobre **web.academyx.es**, web propia del equipo levantada para practicar; el objetivo que figura en la barra, `http://dvwa:80`, era el **valor fijo** que el panel traía entonces: se sustituyó el 10-oct por la herencia del objetivo rastreado, y el laboratorio salió del producto el 13-oct (apartados 6 y 7). La figura se conserva porque lo que documenta es el recorte de gasto, no el objetivo.*

Lo que la figura prueba, y que ninguna prueba unitaria alcanzaba: que el mensaje de lista vacía **dejó de mentir**. Antes, una auditoría que no había podido analizar nada y otra que había analizado y no encontrado nada producían la misma pantalla. Ahora el aviso va aparte de los hallazgos —en su propia tira, por encima— porque mezclarlo obligaría a inventarle una severidad a algo que precisamente **no se pudo valorar**.

Lo que el modelo puede devolver, y lo que el código daba por hecho (RF-08)

Las cuatro vías de análisis del módulo de IA piden al modelo una respuesta con un formato fijo, y ese formato se le impone mediante un **esquema** que viaja con la consulta: la API garantiza que la respuesta se ajusta a él. El esquema, entonces, no es documentación: **es el control**. Y un campo declarado con tipo abierto no es un campo permisivo, es un campo sin control.

La prueba comprueba que el esquema obliga a las dos cosas que las instrucciones piden: que la bandera de resultado sea un booleano y que la confianza venga en escala de 0 a 100. Contra el código anterior devuelve *5 de 11*, y los fallos no eran cosméticos:

CORRECCIÓN DECLARADA (10-OCT): EL RANGO DE LA CONFIANZA YA NO LO IMPONE EL ESQUEMA

Lo de arriba dejó de ser cierto a medias el 10-oct, y se corrige aquí en lugar de reescribirlo en silencio (R9). La API **no admite** `minimum` ni `maximum` en salida estructurada, de modo que el rango 0-100 **tuvo que salir del esquema**: para ese campo, el esquema ya no es el control. Lo que sigue en pie es el resto —las tres banderas sí las impone— y, sobre todo, el matiz que ya constaba: la rendija de la escala 0-1 **nunca la cerró el rango**, porque 0,85 siempre estuvo dentro de 0-100. El control que se pierde es el que cazaba un 150 absurdo; el peligroso, el del descarte silencioso, vive en el clasificador y sigue intacto (sección siguiente). Queda como **limitación conocida**: un valor fuera de escala ya no lo para nadie.

1. De los tres campos de bandera —uno por vía, con **tres nombres distintos para el mismo papel**—, solo el primero estaba declarado como booleano. Los otros dos admitían cadena de texto, y el clasificador evalúa los tres igual, dentro de un `if`. En el lenguaje en que está escrito, **cualquier cadena no vacía es verdadera, incluida la palabra que significa «no»**: una respuesta negativa del modelo se habría registrado como vulnerabilidad confirmada.
2. La confianza se declaraba como número **sin rango**, así que nada rechazaba un valor fuera de la escala.
3. El esquema de la vía de identificación de servidor declaraba **cero campos**, mientras sus instrucciones piden nueve, uno de ellos una lista de objetos.

La causa era una sola, y estaba fuera del repositorio del producto. Los esquemas no se escriben a mano: los genera una herramienta de la práctica que deduce los campos a partir de las lecturas que el propio clasificador hace sobre la respuesta. Esa herramienta llevaba una lista, escrita a mano, de los campos «que el código interpreta», y se redactó mirando **una sola** de las cuatro vías; su propio comentario lo delata, porque habla de la bandera en singular. El esquema vacío tiene la misma raíz por el camino opuesto: la vía de identificación de servidor devuelve la respuesta del modelo **sin inspeccionarla**, de modo que no hay ninguna lectura de la que deducir nada. Un esquema no se puede derivar de un código que no mira los datos.

El arreglo fue por tanto en el generador —no en el fichero generado, que lleva en su cabecera la advertencia de que se regenera— y después se regeneró. La prueba pasa *11 de 11*, y la de RNF-06 sigue en *8 de 8*: el cambio no rompió nada.

Reproducible con `python3 ia/tests/test_esquemas_bandera.py ia/esquemas.py`. Evidencia con las dos salidas, antes y después, en `docs/evidencias/esquemas-bandera-antes-10oct.md`.

Tres rechazos seguidos de la API: validar contra tu propia prueba no es validar (RF-08)

La primera auditoría **real** —clave válida, objetivo propio, techo de gasto puesto— no devolvió ni un hallazgo. Las llamadas que salieron se rechazaron antes de procesarse, y hubo que lanzarla **tres veces** para llegar al fondo, porque cada rechazo tenía un motivo distinto:

1. Los objetos del esquema iban **abiertos**, y la salida estructurada no lo admite.
2. La confianza declaraba un rango con mínimo y máximo, y tampoco lo admite para números.
3. Veintidós campos declaraban su tipo como una **lista de tipos posibles** que incluía «objeto»; para la API eso ya cuenta como objeto, y exige la declaración explícita que esos campos no llevaban.

Coste: cero. Un rechazo por petición mal formada no consume saldo, y el techo acotaba el daño de todas formas. La factura fue de tiempo, no de dinero.

Lo que este episodio enseña, y es lo que justifica contarlo. La prueba de esquemas daba *11 de 11* **y las seguía dando con el defecto dentro**. Comprobaba lo que **nosotros** le pedíamos al esquema —tipos de las banderas, rango de la confianza— y nunca lo que la API exige para aceptarlo. Ninguna prueba local podía cazarlo, porque el requisito incumplido **no estaba escrito en ninguna parte nuestra**: vivía en el servicio del otro lado. Hizo falta que un esquema saliera de casa **por primera vez** para descubrirlo. Dicho corto: **un esquema validado contra tu propia prueba no está validado; está de acuerdo contigo.**

Y un error de método, que es el que explica las dos vueltas. Con el primer rechazo se arregló únicamente lo que nombraba el mensaje, en vez de revisar el esquema entero contra esa misma clase de límite. El segundo rechazo estaba ya ahí, esperando, a la vista de cualquiera que hubiera mirado. La

regla que queda escrita: **cuando un servicio externo rechaza una construcción, se audita la construcción completa contra ese tipo de restricción, no se parchea la línea del error.**

Lo que se hizo para que no haya una tercera vez. La prueba nueva (`ia/tests/test_esquemas_api.py`) ya no comprueba solo los dos defectos conocidos: recorre los esquemas en profundidad y rechaza **cualquier** clave que no conste como admitida, de modo que un límite que aún no nos haya mordido salte aquí y no en una auditoría. Se verificó distinguiendo las dos versiones —*1 de 5* contra los esquemas defectuosos, *5 de 5* contra los corregidos—, y lleva una comprobación de que sabe contar: si el recorrido no encontrara ni un objeto, falla, para que un cero no se confunda con un aprobado.

La corrección de fondo, que es de método y no de código. Las tres restricciones eran averiguables **preguntando**: basta mandar el esquema al servicio y ver si lo acepta. En su lugar se dedujo tres veces a partir del mensaje de error, y cada deducción costó un ciclo completo de arreglar, reconstruir la imagen y relanzar la auditoría. De ahí sale la herramienta `ia/tests/validar_esquemas_contra_api.py`, que somete cada esquema al servicio real y responde si pasa —los cuatro pasan—. Queda fuera de la batería automática porque consume llamadas, y se ejecuta cuando el esquema cambia. **La regla general: cuando el contrato lo pone otro, se le pregunta a él; la prueba propia solo puede vigilar lo que ya sabemos.**

Dónde estaba el defecto, por segunda vez. En el generador de esquemas, que es la única pieza del proyecto que **no está bajo control de versiones**. La primera vez fue la lista de campos escrita a mano que se describe más arriba. Dos defectos distintos, el mismo sitio, y la misma causa de fondo: lo que no tiene historial es donde nadie mira.

Un fallo silencioso no es un fallo menor: la escala de la confianza (RF-08 / RNF-06)

El filtro por confianza compara el valor que devuelve el modelo contra un umbral. Las instrucciones piden ese valor en escala de 0 a 100, pero **ninguna de las cuatro pide que sea un número entero** —se comprobó leyendo las cuatro—, y ahí había una rendija que el esquema **no puede** cerrar: si el modelo devolviera la escala de 0 a 1, un valor de 0,85 queriendo decir «85 %» está **dentro** del rango permitido, de modo que pasaría el control y después quedaría por debajo del umbral. El hallazgo **desaparecería sin dejar rastro**, que es exactamente la situación que RNF-06 existe para evitar. El caso peor es el valor 1: en escala de 0 a 1 significa certeza total, y leído como «1 %» descartaría un hallazgo seguro.

Dónde se arregló, y por qué no en el esquema. Imponer «entero» en el esquema habría exigido algo que las instrucciones no piden, y habría rechazado un valor decimal legítimo dentro de 0 a 100. El arreglo va en el clasificador: la franja entre 0 y 1 se trata como **sospecha de escala** y se encamina al estado degradado de RNF-06, con su motivo. Así el problema **se ve** en el informe en lugar de corregirse a la fuerza o desaparecer.

La prueba distingue las dos versiones, que es la única forma de que valga: contra el código corregido devuelve *19 de 19*; contra el anterior, *6 de 19*. Y los seis que pasan en la versión antigua son precisamente los casos legítimos, lo que demuestra que la guarda no introduce falsos positivos. El caso que lo prueba todo es el que comprueba que un hallazgo con confianza 0,85 **no** se devuelve como «sin vulnerabilidad»: falla contra el código anterior, es decir, el descarte silencioso queda **demostrado** y no argumentado.

Reproducible con `python3 ia/tests/test_escala_confianza.py`. Evidencia con las dos salidas en `docs/evidencias/escala-confianza-guardia-10oct.md`.

La interfaz antes de descartar RF-09, capturada antes de retirarla

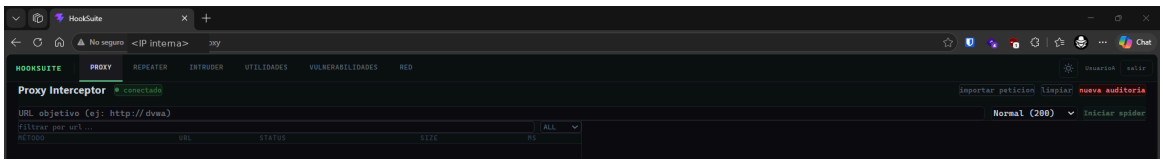


*Figura 31: Barra de navegación de HookSuite con las **seis** pestañas, antes de retirar RF-09; «RED» es la última. Capturada en la cocina el 11-oct, antes de aplicar el cambio. La dirección interna de la barra del navegador va censurada. Requisitos: RF-09 (descartado) y RF-01 (interfaz).*

Esta figura existe por una regla que el grupo se dio al empezar la Fase 3: **las evidencias de un defecto se capturan antes de arreglarlo, porque el arreglo las destruye**. Retirar la pestaña hacía imposible volver a fotografiarla, de modo que se pidió la captura antes de tocar el código y el despliegue se mantuvo a la espera hasta tenerla.

Hay una segunda decisión deliberada, y conviene explicarla porque va contra la intuición: de las tres evidencias previstas se tomó **una sola** imagen. Las otras dos documentaban el defecto técnico —el extremo de consola que responde 404— y ese se guardó **como texto** en lugar de como fotografía. Una salida de terminal en texto es buscable, comparable entre versiones y no corre el riesgo de publicar de refilón una credencial que estuviera en pantalla; una captura no cumple ninguna de las tres cosas. La imagen se reserva para lo único que no puede contarse de otro modo: cómo se veía la interfaz.

9. Matriz de trazabilidad

La pieza con la que se corrige la práctica: requisito por requisito, dónde está implementado, qué prueba lo verifica y dónde se ve. Las rutas están comprobadas en el código (cocina local, 4-oct). La columna de evidencia enlaza la figura de la memoria y el **minuto exacto del vídeo**; se rellena al grabar.

Req.	Descripción	Estado	Implementación (ruta)	Prueba	Evidencia
RF-01	Cumplido (seis pestañas: se retira Red y se añade Manual el 11-oct)	Cumplido	`frontend/src/App.jsx` + `Layout.jsx` + `pages/manual/ManualPage.jsx`	PR-01	fig. apdo. 10. vídeo —: —
RF-02	Modificado (2 errores de servidor corregidos en F1)	Modificado	`backend/services/proxy_service.py`	PR-19	justif. apdo. 4. apdo. 6
RF-03	Cumplido (4 defectos corregidos en F1; quince más en el pulido del 11-oct; el 12-oct las cabeceras que muestra pasan a ser las realmente enviadas)	Cumplido	`backend/services/spider_service.py`, `frontend/.../ProxyPage.jsx`, `.../RequestsTable.jsx`	PR-03	figs. apdo. 8. vídeo —: —
RF-04	**Modificado** (12-oct: el Repeater envía con cliente propio y la sesión se hereda por cabecera explícita)	Modificado	`backend/services/session_service.py`, `backend/services/proxy_service.py`, `backend/routes/repeater.py`	PR-04	justif. apdo. 4. figs. apdo. 8
RF-05	Cumplido (parser y avisos de error corregidos en F1; el importador de peticiones se traslada aquí desde el Spider el 11-oct y se **retira** el mismo día; dieciséis defectos corregidos en el pulido del 12-oct, con un historial añadido y retirado esa misma jornada)	Cumplido	`backend/routes/repeater.py`, `backend/services/proxy_service.py`, `frontend/src/pages/repeater/`	PR-05	figs. apdos. 6 y 8. vídeo —: —
RF-06	Cumplido (cancelación corregida en F1)	Cumplido	`backend/services/intruder_service.py`	PR-06	apdo. 6.

			vídeo —: —
RF-07	Cumplido Utilidades (incl. JWT)	PR-07 `frontend/src/pages/utilities/`	fig. —. vídeo —: —
RF-08	Cumplido (recorrido verificado de punta a punta el 10-oct; techo de gasto ejecutado) Vulnerabilidades	PR-08 `backend/routes/playwright.py`, `backend/services/redis_consumer.py`, `frontend/.../VulnerabilitiesPage.jsx`	figs. apdo. 8. vídeo —: —
RF-09	**Descartado** (11-oct; justificado en §4) Red	`devtools/` conservado. `backend/routes/network.py` se mantiene (lo consumen IA y Playwright)	fig. apdo. 8. vídeo —: —
RF-10	**Parcial** (construido y en la red interna desde la Fase 3; PR-10 sin ejecutar y el sondeo de instrucciones devuelve `401` con sesión válida — §§7 y 8)	PR-10 `playwright/` + `backend/routes/playwright.py`	fig. —. vídeo —: —
RF-11	Cumplido (reimplementado en F2) Multi-usuario	PR-11 `backend/services/guardia.py` + `services/auth_service.py`	apdo. 7. figs. apdo. 8
RF-12	Cumplido en F2 (verificado en cocina 11/11; pendiente de producción) Registro y login por usuario + aislamiento	PR-12 `backend/services/auth_service.py`, `guardia.py`, `usuarios_store.py`, `frontend/.../LoginPage.jsx`	apdo. 7. figs. apdo. 8
RNF-01	Cumplido, con limitación documentada en móvil (apdo. 10) Consola desde navegador sin instalar nada en el cliente	PR-01 `frontend/` servido por Nginx	apdo. 10
RNF-02	Cumplido (autenticado en F2: el token viaja en el primer mensaje, no en la URL) WebSocket en tiempo real	PR-17 `backend/main.py` (`/ws/`)	apdo. 7. vídeo —: —

RNF-03	Despliegue en Docker en Hetzner	PR-13 · `docker-compose.yml` + `infra/nginx.conf`	fig. — . vídeo —: —
RNF-04	Parcial: el reparto es real, pero dos correos del mismo autor parten el grafo de contribuciones	repositorio del producto	apdo. 11
RNF-05	Cumplido	PR-20 · `backend/services/intruder_service.py` (`asyncio.Semaphore`)	apdo. 8
RNF-06	Operación degradada cuando la IA falla o no supera el umbral	**Operación degradada y visto en el panel** (10-oct) · `ia/tests/test_tres_estados.py` (8/8), `ia/tests/test_escala_confianza.py` (19/19) · `ia/analyzers/vulnerability_classifier.py`, `backend/services/redis_consumer.py`, `frontend/.../VulnerabilitiesPage.jsx`	fig. apdo. 8
RNF-07	Cumplido en F1 (exposición cerrada y servidor endurecido) y F2 (autenticación y aislamiento, verificados en producción)	PR-14, PR-16 · ver apartado 7	apdo. 7-8
RNF-08	Operativo y endurecido en F1 (permisos del canal y validación de entradas)	PR-15 · `firewall_agent.py`	apdo. 7-8
RNF-09	Pendiente	PR-18 (por configurar en Nginx)	apdo. 8

RUTAS VERIFICADAS EN CÓDIGO · PRUEBAS Y MINUTOS DE VÍDEO POR COMPLETAR

> **Corrección declarada de este documento (7-oct), por R9.** La fila de RF-11 citaba como implementación la ruta `/api/session/new`. Esa ruta **se retiró en la Fase 2**: repartía identificadores que no eran credenciales, y la interfaz no la llamaba desde hacía tiempo. Citar como prueba de un requisito una ruta que ya no existe es el tipo de error que una memoria arrastra hasta la defensa, así que se deja dicho en lugar de corregirlo en silencio. Quien cumple hoy RF-11 es el sistema de acceso por usuario.

> **Segunda corrección declarada (10-oct), por R9.** Esta matriz **no tenía fila** para RNF-01, RNF-04 ni RNF-06, pese a estar los tres declarados en el apartado 4. El caso de RNF-06 es el que más duele: se le dedican varias secciones del apartado 8 y aun así no se podía trazar desde aquí, que es precisamente la pieza con la que se corrige la práctica. Las tres filas se añaden ahora. El defecto no es de redacción sino de método —la matriz se rellenó requisito a requisito desde la lista de RF y los RNF se fueron añadiendo según aparecían en el trabajo—, y explica por qué faltaban justo los tres que ninguna fase tuvo como protagonista.

Tercera corrección declarada (10-oct), por R9, y es la más seria de las tres. Aquí decía que «el estado de cada prueba (PR-xx) y su resultado detallado están en el apartado 8», y **no es cierto**: de los **dieciocho** códigos de prueba que cita esta columna, solo **cuatro** —PR-01, PR-08, PR-16 y PR-20— aparecen en algún otro punto de esta memoria. Los catorce restantes son una **referencia a ninguna**

parte: quien corrige lee «PR-05» en la fila de RF-05 y no tiene dónde ir a ver qué se probó ni con qué resultado.

> **Cuarta corrección declarada (13-oct), y le toca a las cifras de la anterior.** Este mismo párrafo decía «diecinueve códigos» y «solo tres». Contados uno a uno al construir el anexo salen **dieciocho** y **cuatro** —el que faltaba por contar era PR-08—. Son errores de recuento propios, no del producto, y se corrigen aquí en lugar de cambiarse en silencio.

Y la afirmación que había que corregir antes de copiar nada al anexo (13-oct, R9). Aquí se decía que «el plan existe y está completo, los diecinueve casos, con procedimiento, resultado esperado y resultado observado». **No es así, y era la afirmación más engañosa de este apartado.** El plan de trabajo del equipo tiene **doce** casos —PR-01, 05, 06, 07, 08, 11, 13, 14, 16, 17, 18 y 19—, de modo que **seis de los códigos que esta columna cita (PR-03, PR-04, PR-10, PR-12, PR-15 y PR-20) no tienen caso escrito en ninguna parte**. No es que el lector no tenga el plan: es que de un tercio de lo citado no hay plan que tener.

Y los doce que sí existen llevan resultados del 29 de septiembre, medidos contra el prototipo **antes** de las Fases 1 y 2, así que varios son hoy literalmente falsos: PR-01 figura como fallo por un Basic Auth que la Fase 2 retiró; PR-16 dice que «toda la API responde sin credenciales», que es justo lo que la Fase 2 cerró; y PR-19 da un error de servidor que la Fase 1 corrigió. Copiarlos tal cual habría metido tres afirmaciones falsas en el entregable.

Qué se hace con eso, y por qué el anexo no es un volcado. Esos resultados del 29-sep **no son basura**: son la foto del punto de partida, y el enunciado pide expresamente las pruebas que fallaron con su explicación. El anexo del apartado 13 no copia el plan, lo reconstruye con **dos columnas** —qué daba el 29-sep y qué da hoy— y declara, caso por caso, de dónde sale el resultado de hoy: de una reejecución, de una figura de esta memoria, o de que no se ha podido reejecutar y por qué.

> **Hecho el 13-oct.** El plan está ahora en el **anexo del apartado 13**, reconstruido caso por caso y no copiado: con el resultado del 29-sep, el de hoy y de dónde sale cada uno. Los tres desfases que habrían entrado como afirmaciones falsas —PR-01, PR-16 y PR-19— quedan corregidos con medida nueva. La numeración RF/RNF es la fijada en el apartado 4 y PENDIENTE DE VALIDAR POR EL GRUPO.

⚠️ **HUECO** — responsable: **José María + Claude**. Al revisar el vídeo grabado, anotar el minuto exacto donde se demuestra cada requisito y el número de figura de su captura (R6: el nombre de la captura va atado al ID del requisito, p. ej. `RF-08-panel-ia.png`, para que esta columna se rellene sola).

10. Limitaciones y trabajo futuro

Qué no funciona, qué funciona con condiciones y qué se haría con más tiempo. Este apartado recibe material hasta el último día.

El estado de sesión es volátil, por diseño

El estado de una sesión de auditoría —tráfico capturado, resultados del Intruder, vulnerabilidades— vive **en memoria del backend**: al reiniciarlo, se pierde. Es una **decisión consciente**, no un olvido, y se sostiene en tres razones:

1. **El patrón de acceso del backend no lo soporta sin reescribirlo.** Una treintena de puntos obtienen el diccionario de la sesión y lo mutan en memoria (`session["requests"].append(...)`). Persistir de verdad obligaría a escritura-a-través en cada mutación, es decir, a tocar todo el backend a pocos días de la entrega.
2. **El modelo de sesión cambia en la Fase 2.** El login por usuario ata las sesiones a identidades; lo que se persistiera con el modelo anterior habría que rehacerlo.
3. **Una sesión de auditoría es un espacio de trabajo efímero.** El operador lanza un rastreo, examina los resultados y exporta lo que importa; no espera que la herramienta recuerde una sesión de hace tres días.

Lo que **sí** se hizo, porque era el riesgo real detrás de «todo en memoria»: **poner topes**. El recolector de sesiones existía en el código pero nunca se invocaba, y cualquier llamada a la API con un identificador nuevo creaba una sesión más —con la API sin autenticación, eso permitía agotar la memoria del servidor desde fuera—. Ahora hay un tope de sesiones y de peticiones por sesión (configurables por entorno), el recolector se ejecuta de verdad y, al desalojar, **prefiere las sesiones que no tienen conexión en tiempo real abierta**, para no interrumpir a quien está trabajando. Verificado con prueba de abuso: 120 identificadores inventados dejan el número de sesiones en el tope, y la sesión con conexión viva sobrevive.

Trabajo futuro. La persistencia es viable sin cambiar la arquitectura: el despliegue ya incluye Redis y el backend ya lo usa para otro flujo. El momento correcto es **después** del modelo de usuarios, para persistir sesiones ya asociadas a su propietario.

El coste de una auditoría con IA no está acotado

El módulo de IA no limita cuántas veces consulta al modelo durante una auditoría: el recorrido anida páginas × tipos de prueba × cargas, y cada carga es una consulta. Hoy es inocuo, porque el módulo no se puede disparar desde la interfaz; deja de serlo en el momento en que exista el disparador, que es trabajo en curso. El antecedente está en el apartado 3: en la P1, un disparo involuntario de este mismo recorrido agotó el saldo de la API.

Trabajo futuro, diseñado y no aplicado. Un techo de consultas por auditoría, configurable por entorno y situado en el **cliente** del modelo —por donde pasan las cuatro vías de análisis, de modo que ninguna se escapa—, que al agotarse devuelva la vía degradada de RNF-06 para que la auditoría quede marcada como truncada en vez de parecer completa. Un techo silencioso sería peor que no tenerlo: dejaría el informe diciendo «sin hallazgos» sobre una auditoría a medias.

El valor por defecto se fijará **con el dato** de la primera auditoría real —el resultado llevará el número de consultas consumidas— y no a ojo, que es lo que permite defenderlo.

Lo que queda de la confusión de escala, una vez cerrada

Rectificación declarada. Una versión anterior de este apartado, escrita el mismo día unas horas antes, daba esta limitación por **abierta** y decía que cerrarla dependía de un dato que «no consta». El dato se obtuvo después —ninguna de las cuatro instrucciones pide que la confianza sea un número entero— y con él la limitación se cerró, aunque no por donde este apartado anticipaba: el arreglo está en el clasificador, no en el esquema, y se cuenta en el apartado 8. Se rectifica aquí en lugar de reescribirlo en silencio.

Lo que **sí** queda, y es consciente:

1. **Un valor legítimo de 1 se marca como sospechoso.** La guarda no puede distinguir «1 %» de «certeza total en escala de 0 a 1», porque son el mismo número. Se eligió marcarlo: con el umbral en 60 ese valor tampoco se habría reportado, de modo que el coste es un aviso de más y la ganancia es que el caso peor —descartar un hallazgo seguro— deja de ser posible. **Preferir el falso positivo visible al falso negativo silencioso** es la misma elección que ya tomó RNF-06.
2. **La vía de identificación de servidor no lleva guarda**, porque no compara contra el umbral: entrega la respuesta del modelo sin filtrar, así que no hay descarte silencioso que cerrar.
3. **No se cambiaron las instrucciones del modelo para pedir un entero**, que era la alternativa más limpia en el plano conceptual —instrucciones y esquema de acuerdo en un contrato más estricto—. Se descartó por el momento: son cuatro ficheros del material que no se vuelca a la conversación, se editarían sin verlos, y son prosa y no una línea de código, a tres días de la congelación. Queda como trabajo futuro de coste bajo.

El manual de uso va por dos pestañas escritas y tres por escribir

El 11-oct se añadió una sexta pestaña, **Manual**, con el modo de uso de cada herramienta. Ataca de raíz el patrón descrito en el apartado 8 —funciones correctas que no se ven—, y por eso no es documentación accesoria sino parte del producto.

Están escritas **dos secciones de cinco**: la del Spider, con los veinticinco controles de esa pantalla detallados uno a uno —de qué tipo es cada elemento (botón, campo de texto, desplegable, aviso,

indicador, pestaña, columna), qué hace y **bajo qué condición aparece**, porque buena parte no está en pantalla desde el principio—, y la del Repeater, con veintiséis, añadida el 12-oct en cuanto esa pestaña quedó pulida. Desde entonces el manual se lee **por pastillas** y no en una tirada, y las tres secciones sin escribir se marcan a la vista del usuario, que era la decisión del 11-oct: si falta, se ve que falta.

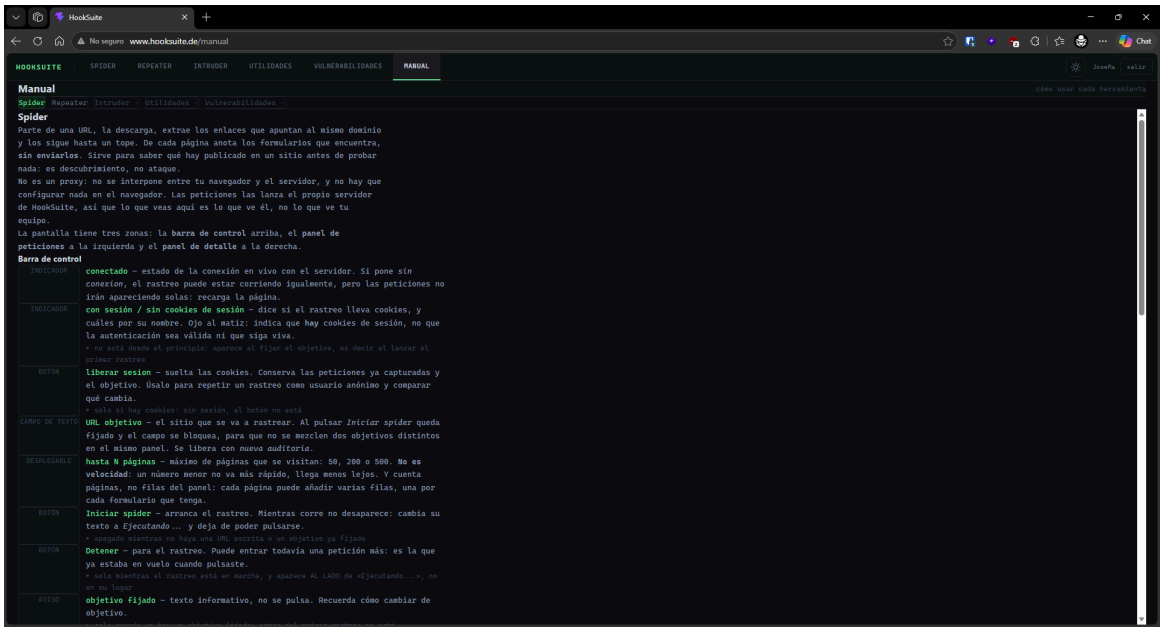


Figura 32: RF-01 — la pestaña Manual: cinco pastillas, la activa resaltada y las tres secciones sin escribir marcadas con un punto, de modo que el usuario ve que el manual está incompleto. Debajo, el modo de documentar: cada control con su tipo (indicador, botón, campo de texto, desplegable, aviso), qué hace y bajo qué condición aparece.

La regla que se fijó es deliberada: **una sección se escribe cuando su pestaña se ha pulido y verificado**, nunca antes. Durante el pulido del Spider, lo que suponíamos de la pantalla y lo que el código hacía no coincidieron quince veces; un manual redactado sobre suposiciones induce a error, que es peor que no tenerlo. La propia redacción lo ha confirmado tres veces: destapó el defecto de los filtros (apdo. 8); un error del manual mismo, que afirmaba que el botón «Iniciar spider» se sustituía por «Detener» cuando en realidad cambia a «Ejecutando…» y «Detener» aparece al lado; y, al escribir la sección del Repeater, veinte líneas de **código muerto** —un aviso que ya no podía aparecer nunca, porque lo provocaba el historial retirado esa misma tarde—. Documentar obliga a mirar el código con los ojos de quien lo usa, y eso encuentra cosas que el uso no encuentra.

Queda pendiente decidir si el manual lleva capturas. Se montó el soporte y se descartó: las capturas de evidencia disponibles son recortes pensados para demostrar un arreglo, no para enseñar a usar, y las nuevas hay que tomarlas con ese propósito.

Dos cosas del Spider que se dejan dichas, no hechas

- **Cuántas veces aparece un formulario.** Un formulario se cataloga una sola vez, bajo la primera página donde se encontró. Es lo correcto para no inflar el recuento, pero se pierde un dato útil para el auditor: en cuántas páginas está presente. La etiqueta actual dice «vistos aquí primero», que avisa del comportamiento sin dar el número.
- **La etiqueta del desplegable.** Se corrigió para que no insinuara más de lo que hay, y quedó anotada para revisarla en la fase de diseño.

El Repeater trabaja con una petición cada vez, y es una decisión

El Repeater no tiene historial: mantiene **una** petición cargada y, mientras la tenga, el Spider no permite enviarle otra —su botón pasa a `repeater en uso` y explica cómo liberarlo—. Se cuenta aquí porque la ausencia de historial es lo primero que echará en falta quien conozca la herramienta de referencia, y porque el camino hasta aquí explica la decisión mejor que la decisión misma: **el historial existió durante nueve horas del 12-oct y se retiró** (apartado 8).

Por qué se sostiene como diseño y no como carencia. El desorden que motivó la retirada venía de tener varias peticiones conviviendo sin que nada dijera cuál era la buena: un estado **implícito**. Ahora hay una sola, el estado ocupado se ve **desde la otra pantalla** y se libera con un gesto deliberado. Para una herramienta cuya salida acaba en un informe, que no se pueda confundir qué respuesta pertenece a qué petición vale más que poder recuperar la de hace diez minutos.

Trabajo futuro, con el camino ya identificado: una lista propia en la sesión del servidor, separada de la que comparten el Proxy y el Spider. La vía que se descartó para esta entrega era añadir un campo de origen al registro **compartido** por esas dos herramientas y filtrar en el extremo del servidor: toca código común a tres pantallas y obliga a recrear el servidor, que borra las sesiones en memoria. Es trabajo para después de la entrega.

El borrador del Repeater guarda las credenciales del objetivo en el navegador

Limitación conocida y declarada, detectada el 13-oct al destilar la jornada anterior. El editor del Repeater guarda lo que estás escribiendo —método, dirección, cabeceras y cuerpo— en el almacenamiento de sesión del navegador, para que no se pierda al cambiar de pestaña o refrescar. Ese guardado **no filtra las cabeceras de sesión**, así que la `Cookie` del sitio auditado —credenciales de un tercero— queda escrita en el navegador del auditor.

Cómo llegó a estar así, porque el recorrido es la parte aprovechable. El 12-oct a las 12:37 se añadió exactamente esa protección, y se añadió donde hacía falta entonces: en la persistencia del **historial**. A las 17:28, al retirar el historial, la función de filtrado se fue con él. Lo que sigue escribiendo en el navegador es el **borrador**, que nunca pasó por ese filtro. Dicho de otro modo: **la protección no se eliminó por error, desapareció con la función a la que estaba pegada**. Conviene retener el patrón —una medida de seguridad acoplada a la función que la motivó se pierde cuando esa función se retira— porque no lo detecta ninguna prueba: el producto sigue funcionando igual.

Y una afirmación de este proyecto que queda rectificada (R9). El diario del 12-oct, a las 13:35, dice que «el filtro que impide persistir credenciales en el almacenamiento del navegador sigue en pie». Dejó de ser cierto cuatro horas más tarde, el mismo día. Se declara aquí en vez de corregirse en silencio.

Riesgo real, acotado sin minimizarlo. El almacenamiento de sesión es legible por cualquier código que se ejecute en la página, de modo que aprovecharlo exige antes una ejecución de código en el propio panel; la única superficie de ese tipo que el producto pinta —la vista previa de la respuesta del objetivo— está aislada desde el 11-oct, y el almacenamiento se vacía al cerrar la pestaña. No es una puerta abierta: es un dato que no debería estar ahí.

Por qué no se arregla ahora, y qué costaría. La corrección son tres líneas: reutilizar al guardar la misma lista de cabeceras de sesión que el editor ya usa para avisar de que la petición viaja autenticada. Se decide no tocar el producto el día de la congelación. La contrapartida del arreglo también está medida: tras un refresco la petición volvería sin sesión y habría que pulsar `preparar` otra vez, que es la misma contrapartida que el grupo ya aceptó el 12-oct para el historial.

Y dos detalles del mismo borrador, menores pero medidos, para que no se lean como un olvido. El borrador se **relee del almacenamiento en cada repintado** de la pantalla, aunque solo haga falta la primera vez —se vuelve a interpretar el mismo texto para descartarlo acto seguido—. Y `limpiar` borra la clave del almacenamiento, pero el guardado automático se dispara inmediatamente después, al vaciarse los campos, y **la vuelve a escribir con la petición en blanco**: no queda nada del trabajo anterior, pero la clave reaparece. Ninguno de los dos afecta al uso; se dejan escritos porque se encontraron leyendo el código y van al mismo sitio que su arreglo cuando se retome.

Lo que falta para igualar al Repeater de referencia

Tres diferencias conocidas y no cerradas:

- **No hay botón para seguir una redirección.** El Repeater muestra el `3xx` y no lo sigue, que es el comportamiento correcto; la herramienta de referencia ofrece además seguirla con un clic. Es trabajo de interfaz y de un extremo nuevo en el servidor, y quedó fuera por la fecha.
- **El origen de las peticiones, que es la diferencia de fondo y no está en el Repeater.** En la herramienta de referencia el formulario llega **capturado del navegador**, con los datos que el auditor

escribió y un token recién emitido: una petición que ya funcionaba. Aquí llega una **plantilla** leída del HTML por el Spider, que nunca se envió. El botón `preparar` cierra la distancia a mano —pide la página y refresca los campos ocultos y la cookie— pero no la elimina: eliminarla exige capturar el tráfico real del navegador, que es justamente lo que se descartó con RF-09.

- **Las cabeceras repetidas se muestran unidas por coma**, no como dos filas, porque el panel de detalle espera un objeto. Se ven —antes se perdía una— pero no se editan por separado.

El frontend no pasa su propio analizador estático

El repositorio tiene configurado un analizador estático para el frontend y, hasta el 12-oct, **nunca se había ejecutado** (apartado 8). Puesto a funcionar, arroja **veintitrés errores y dos avisos** que no son del trabajo de estos días: casi todos de reglas del marco de interfaz —efectos que escriben estado, funciones con efectos secundarios donde se esperan puras— y concentrados en la pantalla de vulnerabilidades. Lo que sí quedó cerrado es la regla que caza el fallo que costó la regresión del botón de enviar: no queda ninguna variable no declarada en todo el frontend.

Por qué no se arreglan ahora: son avisos de calidad sobre pantallas que funcionan y ya están capturadas, y tocarlas el día de la congelación arriesga el entregable sin mejorar el producto. Queda como **trabajo futuro con el dato ya medido**, que es lo que permite planificarlo en vez de prometerlo. Y una limitación del método, dicha entera: el analizador solo puede ejecutarse dentro de la etapa de construcción de la imagen, porque el proyecto no instala las dependencias del frontend en el servidor.

Restos del laboratorio retirado

El laboratorio vulnerable salió del producto el 13-oct (apartado 7), pero **su nombre sigue en dos sitios que no se ejecutan**: la función de acceso del módulo de navegador (RF-10, parcial) y los datos de ejemplo del frontend, que **sí viajan en el paquete** que se sirve al navegador aunque solo se usen cuando no hay servidor detrás. No es un riesgo —los valores son inventados— pero el entregable menciona un componente que ya no incluye. Es limpieza de diez minutos que no se hizo el día de la congelación a propósito: tocar código que funciona, el último día, para arreglar un nombre, es justo el cambio que no compensa.

La herramienta que genera los esquemas no está versionada

El generador de esquemas vive en la carpeta de trabajo de la práctica, **fuera del repositorio del producto**: no tiene historial, no pasó por revisión y su autoría no queda registrada, al contrario que todo el código. Es incoherente con la norma del equipo de que todo entra por petición de incorporación, y no es una incoherencia inocua: **el único sitio sin historial resultó ser exactamente donde estaba el defecto del apartado 8**, porque es el sitio donde nadie vio la lista incompleta.

Trabajo futuro, de coste bajo: mover las herramientas de la práctica al repositorio, para que un cambio en ellas se revise como cualquier otro. No se hizo ahora porque mover ficheros a tres días de la congelación introduce riesgo sin aportar nada al producto.

> ⚠️ **HUECO** — responsable: **José María + Claude**. Acumular el resto de limitaciones conforme aparezcan durante las Fases 1-4; este apartado recibe material hasta el último día.

11. Reparto del trabajo

Quién hizo qué, con estimación de horas por persona.

Cómo se repartió el trabajo, y por qué el historial no lo refleja

El producto es de los cinco. HookSuite lo diseñó y lo construyó el grupo entero en la Práctica 1 —interfaz, servidor, navegador real, captura de tráfico y módulo de IA, un módulo por persona— y **ese** es el producto que se entrega aquí. La Práctica 3 no construye una herramienta nueva: coge la de la P1 y la lleva a un estado ejecutable, seguro y reproducible. Todo lo que esta memoria cuenta descansa sobre el trabajo de los cinco.

El plan de trabajo también es del grupo. El alcance de esta práctica se decidió en la **reunión inicial del 29 de septiembre**: el grupo eligió abordar los cinco primeros puntos del plan de mejoras que la

propia P1 había dejado escrito. Ese plan se desarrolló en un documento que se presentó al grupo el 2 de octubre , y recibió el visto bueno con tres reparos a esa primera versión que se atendieron uno a uno y quedaron incorporados. Las fases, el orden del trabajo y lo que se descarta salen de ahí.

Lo que no se pudo hacer en grupo fue la ejecución. Entre el trabajo de cada uno, las clases del máster y el día a día, al grupo le resultó **imposible** trabajar de forma presencial o continuada durante estas semanas. La práctica se ejecutó en sesiones de trabajo **con la ayuda de un asistente de IA** —el apartado 12 lo declara con detalle: todo el código añadido o modificado en esta práctica lo escribió el asistente—, con el plan del grupo como guía y consultas al grupo en las decisiones de alcance.

Y las confirmaciones del repositorio no indican quién escribió cada línea. Se reparten al azar, a propósito, y lo que viene a continuación explica el mecanismo y su única excepción. Se declara aquí porque el enunciado dice expresamente que el historial de confirmaciones es una de las fuentes con las que se juzga el reparto: en este caso, usarlo para eso daría una foto equivocada.

Qué se pierde y qué se gana diciéndolo. Se pierde la comodidad de un historial que parece un reparto limpio. Se gana que la pregunta quede bien planteada: una confirmación con un nombre **no** es una afirmación de que esa persona escribiera ese código, y no debe leerse así. Lo que el grupo conoce —y lo que el enunciado pide que todos conozcan— es el producto y las decisiones que lo llevaron a este estado.

Y lo que esta memoria no va a decir, aunque sonaría mejor: que el grupo pudo trabajar de forma fluida y repartida en esta práctica. No ocurrió. Escribirlo convertiría una limitación de calendario —comprensible, ajena al producto y que no cambia lo que la herramienta hace— en una afirmación falsa sobre el equipo.

El mecanismo del sorteo de confirmaciones

La autoría de cada confirmacion se asigna **al azar** entre las cinco personas del grupo, y se declara aquí porque **el historial no dice quién hizo qué**. El motivo es que en esta práctica no hubo un reparto de tareas que reflejar: las decisiones se fueron tomando en **consulta de grupo continua** —dudas sobre la herramienta y cómo abordarlas, resueltas sobre la marcha— y la ejecución pasó por una sola vía, de modo que **ninguna confirmación es atribuible a una persona**. Firmarlas todas a un mismo nombre habría dado una foto igual de equivocada sobre un producto que es de los cinco. Se eligió el sorteo, y se dice que es un sorteo. Con una excepcion fija: **la memoria tecnica se firma siempre a nombre de la misma persona**, porque es un documento unico que se regenera entero en cada cambio.

Esa excepcion deforma el recuento, y el dato concreto lo demuestra: de las 41 confirmaciones de la jornada del 10 de octubre, una persona figura en 19 —mas del doble que cualquier otra—, pero **11 de esas 19 son regeneraciones de la memoria**. Descontada la excepcion quedan 8, que es justo lo que cabe esperar de un sorteo entre cinco sobre 41. El reparto es uniforme; lo que no es uniforme es el documento.

La estimacion de horas de este apartado sale del diario, no del historial de confirmaciones.

Dato del 12-oct, tomado del diario y no estimado a posteriori: la jornada suma **unas diez horas y media de asistente** repartidas en veinticinco entradas, más el tiempo de pruebas humanas, que **no se cronometra** porque va entremezclado con el uso de la herramienta. Al consolidar conviene no fundir las dos cifras: las horas del asistente están anotadas entrada por entrada; las de prueba humana, no. Y el reparto de mérito no sigue al reloj —de los dieciséis defectos de esa jornada, la mayoría los encontró quien usaba el producto, no quien escribía el código—.

Horas registradas: qué parte es medida y qué parte inferida

El diario tiene **179 entradas**, **177** con campo de horas, y suma **87,2 h**. El reparto de abajo separa a propósito las tres cosas que suelen ir mezcladas en una tabla de horas: lo que está **escrito**, lo que está **inferido con una regla declarada**, y lo que simplemente **no se registró**.

Quién	Horas	De dónde sale esa cifra
Claude Code (asistente)	**87,1 h**	**40,7 h escritas** con su nombre en el campo de horas, todas de la Fase 3 en adelante ·

		46,4 h inferidas: entradas anteriores cuyo campo dice solo « 1 h», sin nombrar a nadie
José María	**≈1 h de la reunión, más guía continua no cronometrada**	La reunión del 29 de septiembre, como el resto del grupo. A partir de ahí, la guía de la ejecución sesión a sesión, que **no se cronometró** porque va entremezclada con el uso de la herramienta — y que se apoyó en esa consulta de grupo continua: las decisiones que trasladaba salían de ahí. Lo que sí se puede contar son sus intervenciones: **43 entradas** las registran — 23 decisiones, 5 revocaciones de claves, 5 pruebas en producción, 5 lotes de capturas, 3 ejecuciones en la instalación pública y 2 fusiones con permiso de administración
Ivan · Macarena · Nacho · Carlos	**≈1 h cada uno, más consulta continua no cuantificable**	La **reunión inicial del 29 de septiembre** —donde se acordó el plan de trabajo— duró aproximadamente una hora y la hicieron los cinco. A partir de ahí hubo **consulta de grupo continua** durante todo el proceso, en mensajes sueltos sobre la herramienta y cómo abordar cada duda: es real, pero **no se puede cronometrar ni repartir por persona**, y aquí no se inventa un número. Lo que no hubo fue reparto de tareas

La regla de inferencia, dicha para que se pueda discutir. Desde la Fase 3 el campo escribe « **0,5 h de Claude**»; antes escribía « **1 h**» a secas. Las entradas las redactó **el mismo autor de principio a fin**, con el mismo formato y el mismo propósito —medir el tiempo de la sesión—, así que **se infiere que el campo significaba lo mismo antes de decirlo**. No es una medida y no se presenta como tal: son **46,4 de las 87,2 h**, más de la mitad, y por eso la cifra del asistente se da como **87,1 h** con su desglose a la vista en lugar de como un total redondo. (40,7 + 46,4 = 87,1; la décima que falta hasta 87,2 es redondeo de cada entrada.)

Lo que esta tabla NO es, y conviene decirlo antes de que alguien la lea mal. No es el esfuerzo del grupo en el producto: HookSuite lo escribieron los cinco en la práctica anterior, y este diario solo cubre las sesiones de la P3 —las de trabajo con el asistente—, que es donde se hizo el trabajo de esta entrega. Tampoco mide valor: una decisión de treinta segundos —«DVWA fuera del producto», «las credenciales no van al repositorio»— cambió el entregable más que varias horas de construcción.

Y el límite honesto: fuera de esa hora de reunión, el tiempo del grupo **no está en ningún registro**: ni el de la consulta continua ni el de la guía. No se cronometró, y aquí no se inventa.

12. Uso de herramientas de IA

Declaración obligatoria de qué herramientas de IA se usaron y para qué.

- **Claude Code** (modelo Claude de Anthropic): asistencia en diagnóstico, montaje de la cocina, ensayo de limpieza del historial y redacción de esta memoria, bajo revisión humana.

Los límites de la herramienta, observados en primera persona

Usar un asistente de IA para auditar seguridad tiene un límite que conviene declarar porque lo encontramos trabajando: **el filtro de seguridad del propio modelo bloquea a veces tareas legítimas de este dominio**. El 9 de octubre, leyendo las instrucciones del clasificador de vulnerabilidades de HookSuite, la petición fue rechazada con un error que lo dice con todas las letras —el filtro «a veces marca tareas legítimas de programación, **ciberseguridad** y biología»—. No es un fallo del proyecto ni del operador: es el coste de trabajar en seguridad ofensiva con una herramienta generalista, y conviene contarlo con la fecha y la referencia delante en lugar de omitirlo.

Lo caro no fue el bloqueo, sino **la recuperación**. Para desatascar la sesión se usó la función de rebobinado del cliente, eligiendo entre sus cuatro opciones la que descarta lo posterior al punto de

retorno; se perdieron así ocho minutos de trabajo. Las otras dos opciones útiles —resumir desde el punto, o resumir solo lo anterior y permanecer al final de la conversación— habrían conservado ese trabajo. Es una lección de herramienta, no de ingeniería, pero cuesta horas igual.

De ahí sale el hallazgo que sí afecta al diseño de nuestros controles: **el rebobinado restaura la conversación y los ficheros, pero no los efectos sobre el servidor**. En la lista de puntos de retorno, el turno que había construido una imagen de contenedor figuraba como «sin cambios de código», porque esa función contabiliza ficheros. La imagen siguió existiendo después de rebobinar. Quien la construyó —el asistente— ya no recordaba haberlo hecho, y el único testigo del cambio fue un control externo que compara el estado del servidor contra lo que la documentación afirma. La consecuencia práctica es que **un control que mira ficheros no sustituye a uno que mira el sistema**, y que los avisos molestos de ese control no se pueden desactivar sin perder exactamente aquello para lo que sirve.

Qué fue generado y qué fue verificado. La distinción no es teórica: ese mismo día, una afirmación escrita por el asistente en el plan del módulo de IA —que los cuatro conjuntos de instrucciones pedían un campo `descripcion`— resultó falsa para uno de ellos al releer el código fuente, y quedó corregida de forma declarada en vez de reescrita en silencio. El criterio que seguimos es que **nada entra en esta memoria por haberlo dicho la herramienta**: entra cuando se ha contrastado contra el código, la salida de un comando o el enunciado.

Los defectos que introdujo el asistente al arreglar otros

De los quince defectos del pulido del Spider (11-oct, apdo. 8), **dos los causamos nosotros**, y los dos comparten forma: un arreglo correcto en lo suyo que rompe algo contiguo que nadie volvió a mirar.

- **Los formularios pasaron de sobrar a faltar.** Al corregir que un formulario colgara de páginas ajenas, se ató cada formulario a su página de origen. Pero la página se registra con la URL **final** —tras redirecciones— y el formulario se ató a la **pedida**: cuando no coincidían, el formulario no colgaba de ninguna parte y desaparecía del panel. El defecto anterior era visible (sobraban filas); el nuevo era invisible (faltaban). Lo encontró una persona buscando un formulario concreto entre cincuenta y cinco páginas. El asistente **había avisado del riesgo y no lo comprobó**: avisar no sustituye a verificar.
- **Las últimas filas dejaron de alcanzarse.** Al bajar las acciones a una barra propia, el panel pasó de uno a dos hijos y la tabla siguió pidiendo el 100% de la altura: desbordaba, y el recorte se comía las primeras filas capturadas.

El 12-oct la cuenta subió a ocho en una sola jornada, y lo que importa no es el número sino qué los detectó:

Nº	Qué rompió el asistente	Qué lo detectó
1	Los escapes de un aviso nuevo se mostraban en crudo en la pantalla	los ojos de una persona leyéndolo
2	La dirección de destino se colocó en una barra sin sitio: solo se leía cuando la petición fallaba	una captura de producto
3	Un comentario colocado entre los atributos de un componente rompió la construcción	la construcción de la imagen
4	El arreglo de las cabeceras del Spider se aplicó a uno de los dos sitios, y todos los POST empezaron a dar `419`	usando el producto
5	Un borrado por rango de líneas se llevó la declaración de una variable y el botón de enviar dejó de hacer nada, **sin error visible**	usando el producto
6	Al permitir ver el formulario sin haber enviado nada, siete usos que daban la respuesta por segura tumbaron la pantalla entera	usando el producto
7	La pestaña del formulario se pintaba sin estilos, y `limpiar` no la borraba	usando el producto
8	Un borrado de bloque cortado por el primer cierre dejó nueve líneas huérfanas y tres variables sin uso	una comprobación posterior del propio asistente

El reparto es el hallazgo. Seis de los ocho los encontró una persona **usando** el producto; uno lo cazó la construcción de la imagen; uno, una comprobación que el propio asistente hizo **después** de dar el cambio por terminado; y **ninguno** lo cazó el método con el que el asistente verificó sus cambios durante toda la jornada —contar cadenas de texto dentro del paquete construido—. Ese método confirma que algo **está**: no dice que se lea bien ni que funcione.

Dos de los ocho son el mismo error repetido el mismo día: borrar código por rango de líneas sin comprobar qué quedaba al lado. La primera vez se llevó por delante la declaración de una variable; la segunda, el cierre de un condicional anidado. De ahí sale una regla concreta, que vale más que la buena intención de «revisar mejor»: **los bloques se delimitan por contenido, no por el primer cierre que aparezca, y después se comprueba qué ha quedado huérfano**.

Lo que se saca de aquí es que la velocidad del asistente para aplicar un cambio no va acompañada de la misma velocidad para comprobar sus efectos laterales, y que **sus verificaciones son sistemáticamente ciegas a lo que ve una persona en dos segundos**. Por eso las capturas de producto las toma una persona (R6) y por eso el reparto de la línea pone las pruebas en manos humanas: no es desconfianza, es que los dos métodos fallan en sitios distintos.

Leer sin leer: delegar la lectura de lo que no debe entrar en la conversación

El filtro descrito arriba obligó a una medida de método que vale la pena declarar, porque cambió la forma de trabajar. Las instrucciones que el producto envía al modelo son el material que más incomoda a ese filtro y, a la vez, eran lo único que faltaba por conocer para terminar el módulo de IA: **tres defectos abiertos dependían de saber qué formato piden**. El asistente principal no podía abrirlas sin arriesgar la sesión, y el permiso del operador no sirve de nada, porque el filtro no sabe nada de ese permiso: mide lo que entra en la conversación, no lo que alguien ha autorizado.

La salida fue **delegar la lectura**. Un asistente secundario, con instrucciones propias y un contexto que se destruye con él, abre los ficheros y devuelve solo su **interfaz**: nombres de campo, tipos, escalas y números de línea. No devuelve ni una frase del texto. El encargo se redactó con una precisión que resultó ser la clave —**«no los cites», nunca «no los leas»**—, porque una versión anterior, mal formulada, llevó al secundario a **medir** los ficheros sin abrirlos y a contestar a medias.

De ahí salieron los tres defectos del apartado 8. Es decir: el control que impide volcar material delicado en la conversación no solo no frenó el trabajo, sino que obligó a una forma de trabajar —preguntar por la interfaz y no por el contenido, y comprobar con pruebas en vez de con la vista— que resultó **más** estricta que la lectura directa. Lo que no se puede leer, se mide, se edita a ciegas y se verifica con una prueba.

El límite del método también se declara: el asistente secundario es un modelo, no un control. Lo que devuelve tiene que revisarlo **una persona que conozca el material**, porque es la única que puede juzgar si cruzó la línea entre describir y reproducir; el asistente principal no puede, precisamente porque no conoce el contenido.

Qué aportó el asistente, fase por fase

El modelo de trabajo lo fijó el grupo el 1 de octubre: **se le ceden al asistente los mandos de construcción**, y lo que no puede hacer se reparte a personas y queda escrito —probar el producto en marcha, revocar claves, autorizar lo irreversible—. El recuento de la última columna sale del diario, contando sus entradas una a una.

Fase	Qué construyó el asistente	Qué fue decisión o verificación humana	Registro
0 — Reconciliación e historial	Ensayo y ejecución de la reescritura del historial, el orden del empuje forzado de las siete ramas y la reconciliación de los ficheros sin confirmar	**La revocación de las claves**, repartidas en tres cuentas distintas: no es algo que el asistente pueda hacer. El visto bueno al empuje forzado y la decisión sobre la interfaz que solo existía en la instalación pública	9 entradas · 4,6 h
1 — Exposición y endurecimiento	Retirada de los puertos directos, orígenes permitidos, canal en tiempo real autenticado, topes del estado de sesión y endurecimiento del servidor	**Probar la web en producción**: nueve pruebas, tres hallazgos y una corrección al asistente. Y lanzar lo que los controles del servidor le frenan	19 entradas · 9,0 h
2 — Identidad y aislamiento	Alta y acceso por usuario, el guardián de los espacios de datos, las cuatro fugas y la	**El modelo de alta**: que no fuera abierto, sino con código de	21 entradas · 15,9 h

	quinta que introdujo la propia corrección	invitación. Y la verificación en la instalación pública	
3 — IA, pulido y congelación	RF-08 de punta a punta (bus interno, disparo desde el panel, techo de gasto, operación degradada), quince defectos del rastreador y dieciséis del banco de peticiones, el manual de uso, la congelación y esta memoria	**Encontrar la mayoría de los defectos usando el producto** —seis de los ocho de una sola jornada, arriba—, las capturas de pantalla, la clave de la API, la fusión con permiso de administración y las decisiones de **qué no se toca** el día de la congelación	118 entradas · 47,1 h

Generado frente a verificado: la regla, y lo que cuesta sostenerla

- **Todo el código añadido o modificado en esta práctica lo escribió el asistente** —el de la práctica anterior lo escribió el grupo—, y **nada se dio por bueno por haberlo escrito**. Lo que lo valida es otra cosa: sondas con valor esperado y **control negativo**, pruebas en el entorno de trabajo, y la verificación en la instalación pública, que la hace una persona.
- **Las cifras de esta memoria salen de medidas con herramienta, y donde no se ha medido, se dice.** El apartado 7 es el ejemplo: se da el número de avisos que da el auditor de dependencias y se declara, en la misma línea, que **no se ha medido** cuántos son alcanzables desde el camino de ejecución.
- **Las afirmaciones del asistente que resultaron falsas están declaradas en el cuerpo del documento**, no corregidas en silencio: el umbral que «había que añadir» y ya existía (apartado 4), el plan de pruebas «completo, diecinueve casos» cuando eran dieciocho y seis no existían (apartado 9), y un recuento del paquete entregado que el propio mensaje de confirmación contradecía (apartado 10).
- **Las dos formas en que este asistente se equivoca, nombradas porque se repiten.** La primera es **un medidor que no sabría cantar otra cosa**: un cero sobre un PDF cuyo texto va comprimido, o un recuento sobre el texto de una salida cuando lo que se pretendía medir era un árbol de dependencias. La segunda es **un documento que envejece en horas**: un pendiente describe lo que alguien entendió al escribirlo, no lo que el código hace ahora, y fiarse de él produjo tres pendientes caducados en una sola respuesta.
- **Y la limitación de fondo, que explica una norma de trabajo entera:** el asistente **no recuerda** las sesiones anteriores. Lo que no quede escrito en el diario o en esta memoria no existe para la sesión siguiente. De ahí que el diario se escriba **en el momento** y no al cerrar la fase: no es disciplina, es la única forma de que el motivo de una decisión sobreviva a quien la tomó.

Lo que el registro de horas no puede decir

El diario tiene **176 entradas**, **174** con el campo de horas, y suma **85,9 h registradas**. Pero el campo **solo empezó a nombrar a quién** en la Fase 3:

- **39,5 h atribuidas** explícitamente al asistente, en 99 entradas.
- **38,0 h de las Fases 0, 1 y 2** —más 8,5 h de entradas anteriores a la numeración por fases— **sin dueño anotado**.

Por eso el apartado 11 **no puede repartir esas horas por persona a partir de este registro**, y conviene decirlo en lugar de rellenar una tabla que parecería medida. Caben dos salidas honestas: aplicar el modelo de trabajo vigente —el asistente construyó, de modo que la mayor parte de esas horas serían suyas— o declararlas no atribuibles. **Esta memoria no lo resuelve por su cuenta:** lo anota como lo que es, un límite del registro, y deja la decisión a quien firma el apartado 11. Lo que sí está descartado es usar el número de confirmaciones como reparto, por el motivo que ese mismo apartado explica: la autoría se sortea.

13. Anexos

Manual de instalación ampliado, credenciales de prueba (ficticias), glosario y referencias.

Plan de pruebas: los dieciocho casos que cita la matriz

Esta tabla existe porque la columna «Prueba» del apartado 9 citaba dieciocho códigos y el lector no tenía dónde ir a verlos. Se reconstruye aquí con **dos resultados**: el del **29-sep**, medido sobre el prototipo antes de las Fases 1 y 2, y el de **hoy**, sobre el código congelado. La última columna dice **de dónde sale el resultado de hoy**, que es lo que permite juzgarlo: una reejecución, una figura de esta memoria, o nada —y entonces se dice—.

ID	Req.	Qué prueba	29-sep (prototipo)	Hoy (código congelado)
PR-01	RF-01	Abrir la URL pública en un navegador limpio	`401` Basic Auth: la credencial documentada no entra	**200**, sirve el panel · reejecutada en producción
PR-03	RF-03	Rastrear un objetivo y revisar el panel de detalle	no ejecutada	**pasa** · figura del apdo. 8: rótulos legibles y cabeceras reales
PR-04	RF-04	Enviar dos veces la misma petición y comparar	**sin caso escrito**	**pasa** · figuras del apdo. 8: mismo estado, tamaño y cuerpo; distinto tiempo
PR-05	RF-05	Modificar y reenviar una petición; ver la respuesta	envío correcto, historial vacío	**pasa** · figuras de los apdos. 6 y 8; el historial se retiró por diseño (apdo. 10)
PR-06	RF-06	Marcar un campo y lanzar el Intruder	diccionarios servidos; ejecución sin probar	**no reejecutada**: exige lanzar tráfico de ataque real, y su procedimiento del 29-sep nombraba el laboratorio retirado el 13-oct
PR-07	RF-07	Hash, codificadores y regex con casos conocidos	hash y codificadores correctos; JWT no soportado	**no reejecutada**: exige credencial de usuario
PR-08	RF-08	Publicar un hallazgo y recuperarlo por el panel	backend correcto; faltaba el productor	**pasa** · recorrido completo verificado el 10-oct y figura del apdo. 8 (techo a 0 €)
PR-10	RF-10	Ejercitar el módulo de navegador	**sin caso escrito**	**no reejecutada**: hasta hoy el módulo ni siquiera estaba en la red interna; el despliegue del 13-oct lo corrige, pero no se ha ejercitado
PR-11	RF-11	Que cada usuario tenga su espacio de datos	tokens correctos por `/api/session/new`	**procedimiento caducado**: esa ruta se retiró en la Fase 2. Lo que hoy cumple RF-11 es el acceso por usuario, y se prueba con PR-12
PR-12	RF-12	Que un usuario no vea los datos de otro	**sin caso escrito**	**pasa** · cuatro figuras del apdo. 7, verificado 11/11 en la cocina
PR-13	RNF-03	Servicio en pie y API publicada en la caja	`/health` correcto; esquema de la API **abierto**	**pasa y mejora**: `/health` 200 y el esquema **ya no se publica** · reejecutada
PR-14	RNF-07	Sin claves en lo que se publica	por ejecutar	**pasa** · cero ficheros seguidos por git con clave, con control positivo · reejecutada
PR-15	RNF-08	Cortafuegos por sesión	**sin caso escrito**	**no reejecutada**: el agente está operativo y endurecido (apdos. 7-8), pero no se ha vuelto a ejercitar
PR-16	RNF-07	Llamar a la API sin ninguna credencial	**toda la API respondía**	**pasa**: `401`, y los puertos 8000 y 3000 rechazan conexión desde fuera · reejecutada
PR-17	RNF-02	Conexión en tiempo real con el servidor	enlace correcto, sin autenticar	**pasa** · el indicador «conectado» de las figuras lo prueba ya autenticado
PR-18	RNF-09	Responder por TLS	sin HTTPS	**sigue sin TLS** · reejecutada. RNF-09 es el único requisito declarado pendiente
PR-19	RF-02	Estado del proxy del lado servidor	**500**, error de servidor	**pasa**: `401`, el extremo responde y exige credencial · reejecutada
PR-20	RNF-05	Límite de peticiones simultáneas	**sin caso escrito**	**no reejecutada**: el límite está implementado y citado en el apdo. 8, pero sin prueba propia

Cómo leer esta tabla, dicho sin adornos. De los dieciocho casos, **seis se han reejecutado hoy** contra producción, **seis los demuestra una figura** de esta memoria, **cinco no se han reejecutado** y se dice por qué, y **uno —PR-11— tiene el procedimiento caducado** porque cita una ruta que la Fase 2 retiró. Seis de los dieciocho **no tenían caso escrito en ninguna parte** antes de hoy (PR-03, 04, 10, 12, 15 y 20): se redactan aquí por primera vez.

Un aviso de método que vale para quien repita estas medidas. El frontend es una aplicación de una sola página, y el servidor devuelve su portada a **cualquier ruta desconocida**: una dirección inventada responde `200` igual que una real. Por eso, en las filas reejecutadas, un `200` no se ha dado por bueno sin mirar el tipo de contenido —y es lo que evitó concluir que el esquema de la API seguía publicado cuando lo que respondía era la portada—. Las salidas completas están en `docs/evidencias/pruebas-reejecutadas-13oct.md`.

Manual de despliegue

El despliegue es **manual y deliberadamente aburrido**: la instalación pública solo recibe versiones, nunca arreglos. Está prohibido editar ficheros en ella —fue la causa de que durante la Fase 0 lo desplegado no estuviera en ninguna rama— y por eso la actualización se hace con **traer y reponer a la fuerza** el estado de la rama principal, no con una fusión: si alguien hubiera editado algo allí, el siguiente despliegue se lo lleva. Es intencionado.

Orden de los pasos, y por qué este y no otro:

1. **Punto de retorno antes de tocar nada:** etiquetar las imágenes en servicio y copiar la configuración. No cambia nada y es lo que permite volver en un minuto.
2. **Fusionar la rama de trabajo en la principal** mediante solicitud de incorporación. La rama principal está protegida y la solicitud sale bloqueada por la regla de revisión; la desbloquea quien administra.
3. **Desplegar:** traer y reponer a la fuerza, y levantar **solo los servicios que el cambio toca**.
4. **Verificar con sondas,** nunca por impresión visual.
5. **Si falla,** volver al punto 1.
6. **Etiquetar** la versión entregada.

Qué servicios se levantan, y esto se mide, no se supone. Se listan los ficheros que cambian entre las dos ramas y se agrupan por carpeta. En el despliegue de la congelación: tres servicios se reconstruyeron (interfaz, API y módulo de IA), uno se **recreó sin reconstruir** —solo ganaba una línea de red, y su imagen pesa 2,4 GB—, y dos no se tocaron porque ni su imagen ni su configuración cambiaban. Dos lecciones que costaron tiempo y conviene no reaprender:

- **Nombrar un servicio no basta para recrearlo.** Levantar con reconstrucción un servicio cuya imagen se descarga y cuyo bloque no cambió **no lo recrea**: el despliegue sale «en verde» sin aplicar la configuración nueva. Para eso hace falta pedir la recreación explícitamente.
- **Quitar un servicio del fichero de composición no para su contenedor:** queda **huérfano** y sigue corriendo. Para que desaparezca de verdad hay que pedir que se retiren los huérfanos — y eso es parar y eliminar un contenedor, así que lo decide y lo lanza una persona, no el guion.

Verificación por sondas. Cada sonda tiene un valor esperado **medido antes en el entorno de trabajo**, y la lista incluye **un control negativo**: un marcador que debe dar cero. Sin ese control, una sonda que no encuentra nada es indistinguible de una sonda mal apuntada. Dos de las sondas de la primera versión estaban mal dirigidas —preguntaban a la pieza equivocada— y se cazaron **probándolas**, que es para lo que se prueban.

Avisos que no son opcionales:

- La reposición a la fuerza **no toca el fichero de entorno** (está excluido del control de versiones): la instalación pública conserva su clave y su código de invitación.
- Las variables nuevas traen valor por omisión, así que no hay que tocar el entorno para desplegar. Pero el tope de tokens por omisión **se queda corto** con el modelo actual: la respuesta se trunca y el análisis sale como «no analizado» (RNF-06). Para auditar de verdad en producción hay que subirlo en el entorno.
- La reconstrucción de la interfaz es la pesada: si queda poca memoria libre, conviene saberlo antes.

Usuarios de prueba y acceso para la corrección

El producto ya no tiene una contraseña compartida: desde la Fase 2 cada persona **se da de alta ella misma** con un código de invitación y elige su contraseña, que nadie más conoce. Eso cambia cómo se entrega el acceso para corregir, y conviene decirlo con claridad:

- **En este repositorio no hay ninguna credencial**, ni de prueba ni ficticia. El repositorio es **público**: una contraseña escrita aquí sería una contraseña publicada, y la instalación a la que abre está en internet.
- **Las credenciales de corrección se entregan por el canal del máster**, junto a la dirección de la instalación. Incluyen lo necesario para entrar y, si se quiere crear una cuenta propia, el código de invitación.
- **Las cuentas que se usaron para probar** durante el desarrollo viven en el entorno de trabajo, no en la instalación pública, y se retiran antes de la entrega. Se nombraron con prefijos reconocibles por propósito —alta, rastreo, víctima y atacante— precisamente para poder localizarlas y borrarlas luego.
- **Qué se puede comprobar con una sola cuenta:** todo el recorrido de auditoría. Para comprobar el **aislamiento** hacen falta dos, y así se verificó: once de once comprobaciones con dos cuentas distintas.

Glosario

- **Spider (rastreador):** recorre un sitio siguiendo enlaces y formularios para levantar su mapa.
- **Repeater:** banco de trabajo de **una** petición: se modifica y se reenvía tal como se ve en pantalla.
- **Intruder:** prueba muchos valores en un punto marcado de la petición (fuzzing) y señala las respuestas que se salen del patrón.
- **Proxy del lado servidor:** el producto hace la petición **en lugar** del navegador, de modo que no hace falta instalar ni configurar nada en el equipo de quien audita.
- **Fichero de autoconfiguración de proxy:** mecanismo de la primera versión para que el navegador enrutara solo; se jubiló en la Fase 2 junto con sus rutas.
- **Token firmado (JWT):** credencial que emite el servidor y que el cliente presenta en cada petición. Aquí el algoritmo de firma está fijado en el servidor y **no se lee del propio token**.
- **Hash de contraseña:** huella irreversible. Lo que se guarda es la huella, nunca la contraseña.
- **Bus interno:** canal por el que los servicios se hablan sin exponerse a internet; sustituye a que un módulo llame a la API como si fuera un cliente más.
- **Operación degradada:** seguir dando servicio con menos garantías cuando una pieza falla, **y que se note**. Un fallo silencioso que se disfraza de resultado correcto es peor que una caída.
- **Techo de gasto:** límite de llamadas al modelo por auditoría; al agotarse, el resultado sale degradado y visible en lugar de silencioso.
- **Trampa de araña:** sitio que genera direcciones infinitas y hace que un rastreador no termine nunca.
- **Fuga de aislamiento:** que los datos de un usuario aparezcan en la sesión de otro.
- **Huella digital (fingerprint):** identificar la tecnología de un objetivo por sus respuestas.
- **Entorno de trabajo y instalación pública:** las dos instalaciones del producto. Todo se monta y se prueba en la primera; la segunda solo recibe versiones (apartado 5).
- **Contenedor huérfano:** el que sigue corriendo después de que su definición desapareciera del fichero de composición.
- **RF / RNF:** requisito funcional (lo que el producto hace) y no funcional (cómo lo hace: seguridad, rendimiento, despliegue).
- **STRIDE:** modelo de amenazas que recorre seis familias —suplantación, manipulación, repudio, divulgación, denegación de servicio y elevación de privilegio— para no olvidar ninguna (apartado 7).

Referencias

Fuentes del enunciado y de la práctica anterior (material del máster, no redistribuido en este repositorio): el enunciado de esta práctica, el de la práctica anterior y el informe que el grupo entregó entonces. De ahí salen la estructura obligatoria de trece apartados, la numeración de requisitos que la práctica anterior no tenía y el plan de mejoras cuyos cinco primeros puntos delimitan el alcance (apartado 4).

Dentro de este repositorio —y todas estas rutas existen, se comprobó al escribirlas—: `docs/informe/P3-memoria.typ` (fuente única de esta memoria), `docs/informe/diario.md` (registro de trabajo con el motivo de cada decisión, las horas y lo que se descartó), `docs/capturas/` (figuras, con el nombre atado al requisito

que demuestran) y `docs/evidencias/` (salidas de terminal guardadas como texto, no como foto, para que sean buscables y comparables).

Taxonomías y modelos usados: la clasificación de severidad que pide RF-08 se apoya en la taxonomía de OWASP, y el modelo de amenazas del apartado 7 sigue STRIDE.

Herramientas de medición citadas en la memoria, para que cualquiera pueda reproducir las cifras: el auditor de dependencias de cada ecosistema (el del gestor de paquetes de la interfaz y `pip-audit` para los módulos en Python), `pdftotext` para escanear el PDF entregable en busca de secretos —sobre el PDF crudo el texto va comprimido y la búsqueda daría cero siempre—, y el compilador de documentos con el que se generan las dos salidas de esta memoria.

Material de trabajo del grupo que no vive en este repositorio (protocolo de trabajo, flujo de ramas, procedimiento de despliegue y planes por fases) y que se cita en la memoria por su contenido, no por su ruta, precisamente para no dejar referencias que un lector del repositorio no podría abrir.